

FUJINET® COLOR COMPUTER PROGRAMMER'S HANDBOOK

EVERY COMMAND, IN C AND IN 6809



FUJINET®

COLOR COMPUTER

PROGRAMMER'S

HANDBOOK

FUJINET

THE FUJINET COMMUNITY

A WORLDWIDE FREE-SOFTWARE PROJECT — FUJINET.ONLINE

WARNING

Before inserting or removing the FujiNet — or any Program Pak™ — be sure the Computer is OFF. Otherwise, the FujiNet or the Computer could be damaged.

A WORD ABOUT THESE NUMBERS

Every address, command byte, parameter and reply length in this book was read out of the FujiNet sources and checked against a client that speaks to them. Firmware moves. When this book and the sources disagree, the sources are right and this book has a bug; the back cover says where to report it.

FujiNet Programmer's Handbook for the TRS-80® Color Computer.

First edition.

The FujiNet firmware, the fujinet-lib client library, and the example programs printed in this book are free software, published under the GNU General Public License. Copy them, change them, put them on a disk and give the disk away.

TRS-80, Color Computer, Program Pak and Radio Shack are trademarks of Tandy Corporation, used here the way one speaks of an old friend. OS-9 is a trademark of Microware. Neither company had anything to do with this book and neither should be blamed for it.

The DriveWire protocol is Boisy Pitre's, and the Color Computer owes him a drink. The bit-banger routines this book leans on all evening are Darren Atkinson's. CMOC is Pierre Sarrazin's. LW-TOOLS is William Astle's. None of this would run without them. Written by the FujiNet community for anyone who wants to put a Color Computer on the network and then write something that uses it.

10 9 8 7 6 5 4 3 2 1



TO OUR PROGRAMMERS



At the simplest level of operation, you do not need this book at all. The CONFIG program that comes up when you switch the Computer on will put a disk in a drive for you, and every program you already own will load from it and never know the difference. The Getting Started manual covers that, and covers it well.

At a slightly more involved level, you have written something in Extended Color BASIC and you would like it to fetch a score table, or the weather, or a line of text from a friend's machine three time zones away. You will want Chapters 1 through 5, and then Chapter 12 as a menu.

If, however, you already know what a devicespec is, and you have opened this book to find out whether SET_DIRECTORY_POSITION takes its argument high byte first — it does — then start at Chapter 12 and use the index. Everything in the reference chapters was transcribed from the firmware and checked against a client that talks to it, and every card carries the same command written twice: once in 6809 assembly and once in C.

What is in here

Chapters 1 through 5 are the short road to a working program. They cover the tools, the two ROM vectors that carry everything, the shape of a FujiNet command, and a first program that asks the adapter who it is.

Chapters 6 through 11 are the libraries and the environments: a 6809 assembler library you can lift wholesale, the C side using fujinet-lib, the two devices you will spend all your time with, and what changes when you leave Disk BASIC for OS-9.

Chapters 12 through 15 are the reference. Every command the Color Computer can send is in there, one card each, with its frame laid out byte by byte and an example in both languages. The commands that

are **not** answered are named too, at the end of each chapter, so that you can tell a command that does nothing from one you spelled wrong. Chapters 16 through 19 are four complete programs, and Appendix C prints them in full. They are not sketches. They compile with the tools in Chapter 2 and the listings in this book are read off the same files at the moment the book is typeset, so a listing here cannot drift from one that builds.

What is not

This book documents the **bus**: how to get a command to the adapter and an answer back. It does not document what lives at the far end of a URL. The schemes the N: device speaks — TCP:, HTTP:, TNFS:, JSON:, SSH:, SMB: and the rest — have a book of their own, the **FujiNet Network Protocol Handbook**, and Chapter 20 says where to find it. It also assumes you can already write 6809 assembly or C for the Color Computer. There are good books on both and this is not trying to be one of them.

CONTENTS

To Our Programmers.	3
1. Welcome to FujiNet.	6
What is on the other end □ The devices □ What it is not	
2. Installing the Tools.	8
CMOC □ LWTOOLS □ Toolshed □ fujinet-lib □ A place to put it	
3. The DriveWire Connection.	10
Two vectors □ The calling convention □ Baud and the model switches □ Is anybody there? □ Why the vectors	
4. The Shape of a Command.	12
Two frames □ Big-endian, for once □ The three meta-commands □ The three-step dance □ Getting the length wrong □ One error byte for the whole bus	
5. First Contact.	14
The command □ In assembly □ In C □ What comes back □ Building and running it	
6. The 6809 Assembler Library.	16
fn.inc □ FNWAIT □ FNXACT and FNGERR □ FNGRSP □ FNCMD0 and FNCMD1 □ The network four □ Register discipline	
7. C on the Color Computer.	19
What CMOC gives you □ The library's shape □ Reading the return □ Dropping below the library □ Inline assembly □ Memory	
8. The Network Device.	22
Devicespecs and units □ Open modes □ Translation □ The five moves □ Reading what is there □ Prefixes □ Parsers □ Interrupts	
9. The Fuji Device.	24
Host slots □ Device slots □ Prefixes □ Walking a directory □ App keys □ The adapter's spare cycles	
10. Devices, Slots and Booting.	26
Getting a file into a drive □ Access modes □ Making a blank disk □ What an image can be □ Boot modes □ Living with HDB-DOS	
11. FujiNet under OS-9 and NitrOS-9.	28
What still works □ What stops working □ Supplying the two □ The stack above □ The bit-banger □ What this book has verified	
12. Command Reference: The Fuji Device.	30
The bus answers three □ WiFi and the adapter □ Hosts and prefixes □ Directories □ Device slots and mounting □ Booting and	
13. Command Reference: The Network Device.	60
The lifecycle five □ Line discipline and position □ Parsers and queries □ Filesystem operations □ Sockets and credentials □ Answered with an error	
14. Command Reference: The Other Devices.	75
The clock □ Setting the timezone □ CP/M □ The printer □ The modem, and why it is not here	
15. Command Reference: DriveWire Itself.	83
Reading a sector □ Writing a sector □ The clock, the short way □ Identification and reset □ Virtual serial channels □ Accepted and ignored	
16. Netcat.	90
What it does □ The loop □ The two halves □ Somewhere to point it	
17. Mounter.	92
The sequence □ Two frames by hand □ Reading the directory □ What CONFIG adds	
18. Weather.	94
Why a parser □ The sequence □ The frame the library gets wrong □ Asking a question	
19. fnstat under OS-9.	96
Three files □ The transport □ The bus □ Building it □ What is verified	
20. Always More To Come.	98
The other book □ Where the sources are □ Where the people are	
Appendix A. DriveWire Quick Reference.	99
Vectors □ Opcodes □ Frames □ The dance □ Modes	
Appendix B. Error and Status Codes.	101
The device error byte □ Sector status □ Network status □ WiFi status □ The library's codes	
Appendix C. Program Listings.	103
The equates □ The library □ The console □ Netcat □ Mounter □ Weather □ OS-9	
Appendix D. Troubleshooting.	125
Index.	127

resetting ☐ App keys ☐ Base64 ☐ Hashing ☐ QR codes ☐ Odds
and ends ☐ Answered with silence

1. WELCOME TO FUJINET

The FujiNet is an ESP32 with a WiFi radio, a microSD slot and a serial cable, in a case that sits beside your Color Computer. The cable goes to the four-pin DIN on the back of the Computer — the one Radio Shack meant for a printer — and from the Computer’s point of view what is on the other end is a DriveWire server.

That is the whole trick, and it is worth sitting with for a moment. Disk BASIC already knows how to talk to a DriveWire server: HDB-DOS is in the cartridge, the ROM routines are in memory, and `DIR` works. You did not have to teach the Computer anything. The FujiNet simply answers, and because it is answering over a protocol that already had room in it, the FujiNet’s own commands ride along in the same conversation.

What is on the other end

A DriveWire server answers an opcode byte and then reads or writes however many bytes that opcode implies. The published opcodes cover disks, printing, the clock and a set of virtual serial channels. FujiNet answers all of those, and then answers four more of its own:

\$E2..... the FujiNet control device — slots, hosts, WiFi, app keys
\$E3..... the N: network devices — one channel each
\$E4..... the CP/M console
\$E5..... the clock

Everything in Chapters 12 and 13 — every host slot, every mounted image, every socket you open — goes out as one of those two bytes followed by a command byte. There is no handshake to negotiate, no device to enumerate, and nothing to install. You write \$E2 and the adapter is listening.

The devices

Disk

Four drive slots, numbered 0 to 3. The Color Computer gets four where the Atari gets eight, because `MAX_DWDISK_DEVICES` is four. Mount an image into a slot and it answers `OP_READEX` and `OP_WRITE` as a 256-byte-block device — which is exactly what HDB-DOS expects, so `DRIVE #0` through `DRIVE #3` work with no further ceremony. Chapter 14 documents the block protocol, checksum and all.

The FujiNet control device

Device \$E2. Sixty-odd commands: join a network, read and write the eight host slots and the four drive slots, walk a host’s directory, mount and unmount, copy a file between hosts, keep a small blob of application state in an app key, and — because the adapter has a processor and you do not — compute a SHA-256, encode base64, or render a QR code.

The network device

Device \$E3, and the one you will spend your time with. Open a `devicespec` like

`N:TCP://fujinet.online:6502/`

and you have a socket. Twenty-nine commands hang off it: the five that matter (open, close, read, write, status) and twenty-four more for seeking, locking, making directories, accepting inbound connections and running a JSON, HTML or XML parser over whatever came back so that the Computer never has to hold the whole document.

The clock, the printer, CP/M

The clock will give you the time in seven formats, one of which is a six-byte reply to a single opcode with no command byte at all. The printer takes one byte per call and emulates a shelf full of hardware that has not been made in forty years. CP/M is a Z80 emulator with a console on the wire, and it is the only device in this book that is not available on every build.

What it is not

The FujiNet is not a modem, not a terminal program, and not a filesystem. It is a way to move bytes, and every interesting thing on the other side of those bytes is somebody else's protocol. This book stops at the point where your bytes leave the Computer.

One more honest note before we start. The adapter's own AT-command modem is implemented in the firmware and is **not** reachable on this bus: nothing constructs it for the Color Computer build. Chapter 14 says so again, in its place, because a command reference that quietly omits the dead ends is not a reference.



2. INSTALLING THE TOOLS

Everything in this book is built with two compilers and one disk tool, all three of them free, all three of them running on the machine you are reading this on rather than on the Color Computer. Cross-development is not cheating. The CoCo has 32K and no editor worth the name; your other computer has a screen you can see.

CMOC

CMOC is Pierre Sarrazin's C compiler for the 6809. It targets Disk Extended Color BASIC by default, which is what we want: it emits a `.BIN` file you load with `LOADM` and start with `EXEC`. It also targets OS-9, which Chapter 11 needs.

AT THE PROMPT

```
$ cmoc --version
cmoc (cmoc 0.1.100)
```

Debian and Ubuntu have it as `cmoc`; Arch has it in the AUR; otherwise build it from source. It wants a C preprocessor (it uses the system `cpc`) and it wants `LWTOOLS`, below, because it assembles and links with them.

LWTOOLS

William Astle's `LWTOOLS` is the 6809 assembler, linker and librarian: `lwasm`, `lwlink`, `lwar`. Every assembly listing in this book assembles with `lwasm`, and `CMOC` calls it for you on the C side.

AT THE PROMPT

```
$ lwasm --version
lwasm from lwtools 4.24
```

The two flags that matter here are `--6809`, because the Color Computer is a 6809 and not a 6309, and `--format=decb`, which writes the loader blocks `LOADM` expects.

AT THE PROMPT

```
$ lwasm --6809 --format=decb \
-o NETCAT.BIN netcat.asm
```

Toolshed

Toolshed gives you `decb`, which makes and fills Disk BASIC disk images, and `os9`, which does the same for OS-9. You need it to get a `.BIN` onto something the Computer can read.

AT THE PROMPT

```
$ decb dskini NETCAT.DSK
$ decb copy -2b NETCAT.BIN NETCAT.DSK,NETCAT.BIN
```

Put the resulting image somewhere your FujiNet can reach — the microSD card, or a TNFS share — mount it into a drive slot, and `LOADM` it. Chapter 17 writes a program that does the mounting for you.

A SHORTER LOOP

If you are running FujiNet-PC rather than hardware, or if you keep a TNFS server on the same machine you compile on, you can copy the built `.DSK` straight into the served directory and re-mount it from `CONFIG`. Edit, make, re-mount, `LOADM`. It is about fifteen seconds.

fujinet-lib

fujinet-lib is the C client library, and it has a Color Computer port built with CMOC. A release unpacks flat: the three headers, three assembler include files, and one library.

AT THE PROMPT

```
fujinet-fuji.h      fujinet-fuji.inc
fujinet-network.h  fujinet-network.inc
fujinet-clock.h    fujinet-clock.inc
fujinet-coco-4.7.9.lib
```

Point CMOC at the directory with `-I` and hand it the library on the link line. Some installations symlink the library to `libfujinet.a` in CMOC's own library directory, in which case `-lfujinet` finds it:

AT THE PROMPT

```
$ cmoc -Ifujinet -o WEATHER.BIN \
    weather.c fujinet/fujinet-coco-4.7.9.lib

$ cmoc -Ifujinet -o WEATHER.BIN \
    weather.c -lfujinet
```

The library covers most of the FujiNet control device and the five moves of the network device. It does not cover everything, and Chapters 12 and 13 say so command by command: each card names the fujinet-lib call when there is one, and shows the raw frame when there is not. Chapter 7 shows how to drop below the library without leaving it.

The assembler side

There is no equivalent library shipped for assembly, so this book brings one. Chapter 6 develops it and Appendix C prints it:

fn.inc... every opcode, command byte, mode and error code as an EQU
fnlow.asm... nine subroutines: the handshake, the verdict, the response
fnnet.asm..... the five moves of the N: device
cocoio.asm... screen and keyboard, so the examples can say something

Put them in a directory and point `lwasm` at it with `-I`. They assume nothing about where your program lives and allocate nothing but their own frames.

A place to put it

The layout the listings in this book use:

AT THE PROMPT

```
listings/
common.mk      the shared flags
fnlib/         fn.inc, fnlow.asm, ...
netcat/        netcat.asm, netcat.c
mounter/  weather/  os9/
```

and `common.mk` holds the one thing that varies between machines:

AT THE PROMPT

```
FNDIR ?= /usr/share/cmoc/include
CFLAGS := -I$(FNDIR)
ASFLAGS := --6809 --format=decB \
    -I../fnlib
```

NOTE

The listings in this book are read off disk when the book is typeset. If you are reading the PDF, the code you see in Appendix C is the same text that produced the .BIN files, character for character. It cannot have drifted, because there is nowhere for it to drift to.

3. THE DRIVEWIRE CONNECTION

Everything the Color Computer says to a FujiNet, and everything it hears back, goes through two subroutines. They are already in memory. Disk BASIC put them there.

Two vectors

Disk BASIC keeps the DriveWire entry points in a pair of indirect vectors near the top of the ROM:

\$D93F.....DWRead — the vector, not the routine
\$D941.....DWWrite

They are vectors, so you call through them and not to them:

6809 ASSEMBLER

```
DWREADV EQU    $D93F
DWWRITV EQU    $D941

        ldx     #BUFFER
        ldy     #LENGTH
        jsr     [DWWRITV]
```

On a Dragon the same two routines live at \$F9FE and \$FA00. fujinet-lib carries both and picks with a -DDRAGON at build time; so does this book's fn.inc.

The calling convention

Both routines take the buffer in X and the count in Y. What they give back is where people trip.

DWWrite

Sends Y bytes from X. On return X points one past the last byte sent and Y is zero. Everything else is preserved. There is no error return: the bit-banger writes into the dark.

DWRead

Reads Y bytes into X and gives up if the line goes quiet for about a second and a half. On return:

- **Z is set** if every byte arrived. This is the test you want — BNE branches on failure.
- **The carry is set** on a framing error, which is a different and rarer complaint.
- X still points at the start of the buffer.
- Y holds a 16-bit sum of the bytes received. It is the disk checksum, and it means your count register does not survive the call.
- All three accumulators are clobbered. U is preserved.

So the shortest correct read is:

6809 ASSEMBLER

```
        ldx     #BUF
        ldy     #7
        jsr     [DWREADV]
        bne     NOANSWER        ; Z clear
```

WATCH THE FLAG

It is the zero flag, not the carry, that says the read worked. fujinet-lib's C binding extracts it with `tfr cc,b` and two `lsrcs`, which is bit 2 of the condition codes — the Z flag. Testing the carry instead gives you a program that works perfectly until the network is slow.

Baud and the model switches

The bit-banger's speed is set by how many cycles the CPU spends between bits, so it depends on the machine. The FujiNet reads a pair of DIP switches at power-up and picks to match:

Model setting	Speed
Color Computer 1	38,400 baud
Color Computer 2	57,600 baud
Color Computer 3	115,200 baud
Dragon	57,600 baud

Cartridges built around the high-speed UART run at 921,600 baud to their own coprocessor instead, and FujiNet-PC over DriveWire-over-IP takes the rate from its configuration file and listens on TCP port 65504. You do not set any of this from a program, and you cannot ask what it is. If the adapter is silent, the switches are the first thing to check.

Why the vectors

You could bit-bang the port yourself. People have. The reason not to is that the routines behind those vectors are cycle-counted, tuned by people with oscilloscopes, and are the same ones HDB-DOS is using to serve the disk you booted from.

The deeper reason is that everything above them stops caring how the bytes move. Chapter 11 takes the vectors away — OS-9 does not map that ROM — and nothing above them changes.

Is anybody there?

There is a one-byte opcode whose only job is to prove something is listening. Send \$A5 and the adapter replies with the seven characters FUJINET — no length, no terminator, no command byte.

6809 ASSEMBLER

```
PROBE    ldx    #JFRM
         ldy    #1
         jsr    [DWWRTV]
         ldx    #JBUF
         ldy    #7
         jsr    [DWREADV]
         bne    NOFUJI    ; nothing there
         ldx    #JBUF
         ldy    #JSIG
         ldb    #7
CMP1     lda    ,x+
         cmpa   ,y+
         bne    NOFUJI
         decb
         bne    CMP1
         rts                ; Z set: a FujiNet

JFRM     fcb    $A5
JSIG     fcc    "FUJINET"
JBUF     rmb    7
```

CMOC C

```
/* fujinet-lib keeps these to itself,
   but they are in the library. */
extern byte dwread(byte *s, int l);
extern byte dwwrite(byte *s, int l);

byte fujinet_present(void)
{
    byte op = 0xA5;
    byte buf[8];

    dwwrite(&op, 1);
    if (!dwread(buf, 7))
        return 0;    /* silence */
    buf[7] = '\0';
    return !strcmp((char *) buf,
                  "FUJINET");
}
```

It costs eight bytes on the wire and it is the difference between a program that says `NO FUJINET FOUND` and one that hangs.

4. THE SHAPE OF A COMMAND

A FujiNet command is a few bytes written to the wire. There is no header, no length field, no checksum and no acknowledgement. The adapter reads an opcode, and from that opcode it knows exactly how many more bytes to take. If you and it disagree about that number, nothing complains: the conversation simply becomes nonsense from that point on, forever.

Which is why this chapter is short, and why it is the most important one.

Two frames

The FujiNet control device takes an opcode and a command byte, then whatever that command wants:

\$E2	command	aux bytes, 0 to 3	payload, if any
------	---------	-------------------	-----------------

The network device puts a unit number between them, and then **always** takes exactly two more bytes before any payload — whether the command wants them or not:

\$E3	unit	command	aux1	aux2	payload, if any
------	------	---------	------	------	-----------------

That “always two” is not a convention, it is the parser: the frame decoder for \$E3 consumes a two-byte parameter word before it will hand you a payload, and its destructor consumes it on the way out if nothing else did. A network command with no parameters still needs two zero bytes. Leave them off and the next command’s first two bytes become this command’s parameters.

The control device has no such rule. Its aux byte count is per command, and the reference cards in Chapter 12 give it for each one.

Big-endian, for once

Any parameter wider than a byte goes high byte first. So does the length in a SEND_RESPONSE on the network device, and so do the byte counts in READ and WRITE.

This is the one place where the Color Computer has it easier than the Atari or the Apple II. The 6809 stores the high byte first, so a CMOC structure can go straight out of the door:

```
CMOC C
struct
{
    uint8_t  opcode;
    uint8_t  unit;
    uint8_t  cmd;
    uint16_t len;      /* already big-endian */
} r;

r.opcode = 0xE3;
r.unit   = unit;
r.cmd    = 'R';
r.len    = 512;

dwwrite((uint8_t *) &r, sizeof(r));
```

No swapping, no packing directives, no helper macros. On every other FujiNet platform that structure needs work before it is legal on the wire. Here it is legal as written.

THREE EXCEPTIONS

Three replies come back little-endian, because the firmware sends a raw host integer rather than a declared big-endian one: GET_DIRECTORY_POSITION, RANDOM_NUMBER, and the network device’s TELL. Their cards say so. Everything else is high byte first.

The three meta-commands

Three command bytes never reach a device. The bus answers them itself, and they are the same three on \$E2 and on \$E3:

\$00 READY. . . . Writes back a single \$01. It means “I am here and I am listening.”

\$02 SEND_ERRORWrites back one byte: how the last command went. \$01 is success; everything else is in Appendix B.

\$01 SEND_RESPONSE. . Writes back whatever the last command parked. You must know how many bytes to expect.

On the network device all three take the unit byte and the two aux bytes like any other command, and SEND_RESPONSE reads its length out of that aux word and zero-pads the reply up to it.

The three-step dance

So a command that returns data is three exchanges, not one:

1. **Ask if it is ready.** Write \$E2 \$00 and read one byte. If nothing comes back, the adapter is busy; write it again. Loop until it answers. This is not politeness — a command sent into a busy adapter is a command that vanishes.
2. **Send the command.** Opcode, command byte, aux bytes, payload. Nothing comes back.
3. **Collect.** Write \$E2 \$02 and read the error byte. If the command had something to say, write \$E2 \$01 and read exactly as many bytes as it owes you.

Here is the whole of GET_WIFISTATUS as bytes on the wire. Three bytes out, one in; two out, one in; two out, one in.

AT THE PROMPT

```
CoCo -> E2 00      ready?
FN   -> 01         yes
CoCo -> E2 FA      GET_WIFISTATUS
CoCo -> E2 00      ready?
FN   -> 01         yes
CoCo -> E2 02      how did it go?
FN   -> 01         fine
CoCo -> E2 00      ready?
FN   -> 01         yes
CoCo -> E2 01      give it here
FN   -> 03         connected
```

Eleven bytes to learn one. It is not efficient and it does not need to be: at 115,200 baud that is under a millisecond, and the alternative — inline replies — is what makes the SIO bus on the Atari need a checksum on every frame.

Getting the length wrong

Say a command wants a 256-byte payload and you send 200. The adapter takes your 200, then keeps reading. The next 56 bytes it gets are the front of your next command. It uses them as payload, answers something meaningless, and now every byte for the rest of the session is one command out of step.

There is no recovery from this short of resetting the Computer. So:

- Pad fixed-size fields to their full size. A devicespec field is 256 bytes even when the spec is 20.
- Read exactly as many bytes as the card says a reply is. Not fewer.
- Send the aux bytes even when they are ignored.

One error byte for the whole bus

A caution that costs nothing to obey and an evening to discover. The error byte and the parked response are kept in one place for the whole bus, not one place per device. The network device’s dispatcher hands the control device’s error state to the meta-command handler.

In practice: finish one command before you start another. Do not send a control-device command between a network command and its SEND_RESPONSE, and do not interleave two N: units. The library does not protect you from this and neither does the adapter.

5. FIRST CONTACT

Here is a program that asks the adapter who it is and prints the answer. It is thirty lines of assembly or twenty of C, it uses one command, and when it works you know the cable, the switches, the ROM vectors, the handshake and your build are all correct. Nothing else you write will have as good a ratio of certainty to effort.

The command

GET_ADAPTERCONFIG_EXTENDED, command byte \$C4, no parameters, 240 bytes back. It is the right first command for three reasons: it takes nothing to get wrong, it answers before the WiFi is up, and its reply carries the IP address as text so you do not have to write a print-a-number routine before you can see anything.

The 240 bytes are one structure. The fields you want first are:

0.....ssid[33] — the network it joined
125.....fn_version[15] — the firmware
140.....sLocalIP[16] — the address, already dotted
and the rest — hostname, the binary forms of the four addresses, the MAC, the BSSID, and text forms of all of them — are laid out in full on the card in Chapter 12.

In assembly

Three calls. FNCMD0 sends a control-device command that takes no parameters and brings back the verdict; FNGRSP fetches what it parked. Both are from `fnlow.asm`, which Chapter 6 builds.

6809 ASSEMBLER

```
lda    #FCADCFX      ; $C4
jsr    FNCMD0
bne    FAILED        ; A was not $01

ldx    #CFG
ldy    #SZADCFX      ; 240
jsr    FNGRSP
bne    FAILED
```

and then it is just printing:

6809 ASSEMBLER

```
ldx    #MSSID
jsr    PUTS
ldx    #CFG+0        ; ssid[33]
jsr    PUTS
jsr    CRLF

ldx    #MADDR
jsr    PUTS
ldx    #CFG+140      ; sLocalIP[16]
jsr    PUTS
jsr    CRLF
```

Every field is NUL-terminated inside its own fixed-width slot, so the same PUTS that prints a message prints a field. Appendix C.2 has the whole file.

In C

fujinet-lib wraps this one, and the structure it fills is declared in fujinet-fuji.h, so the offsets above become field names:

```
CMOC C
#include <cmoc.h>
#include <coco.h>
#include <fujinet-fuji.h>

int main(void)
{
    AdapterConfigExtended ac;

    putchar(12);          /* clear */
    printf("FUJINET FIRST CONTACT\n");

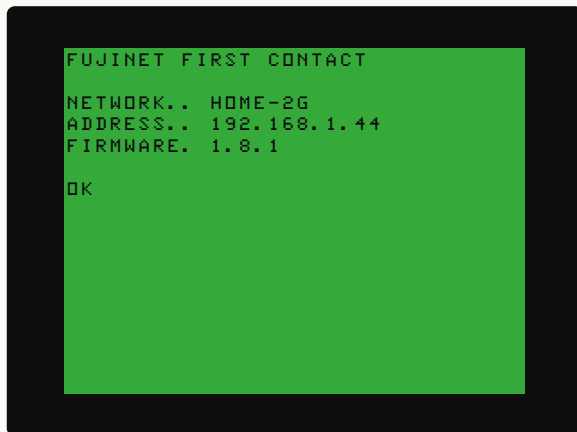
    if (!fuji_get_adapter_config_extended(&ac))
    {
        printf("NO ANSWER\n");
        return 1;
    }

    printf("NETWORK.. %s\n", ac.ssid);
    printf("ADDRESS.. %s\n", ac.sLocalIP);
    printf("FIRMWARE. %s\n", ac.fn_version);

    return 0;
}
```

putchar(12) clears the screen: Color BASIC's console-out routine treats a form feed that way, and it saves calling cls() for one character. Either is fine.

What comes back



Building and running it

AT THE PROMPT

```
$ lwasm --6809 --format=decb \
-I../fnlib -o FCDEMO.BIN fcdemo.asm

$ cmoc -Ifujinet -o FCDEMO.BIN \
fcdemo.c -lfujinet
```

Get the .BIN onto a disk image, mount the image, and:

AT THE PROMPT

```
LOADM"FCDEMO"
EXEC
```

IF IT SAYS NOTHING AT ALL

A program that hangs here has not reached the adapter. FNCMD0 begins with FNWAIT, which asks “are you there” forever and never gives up, because that is exactly what you want once you know something is out there. Before you know it, use the seven-byte probe from Chapter 3 — it fails in a second and a half and tells you so.

IF THE SSID IS EMPTY

Then you reached the adapter and it is not on a network. That is a CONFIG problem, not a program problem; the Getting Started manual has the screens. GET_WIFISTATUS will tell you the same thing in one byte: \$03 is connected and \$06 is not.

6. THE 6809 ASSEMBLER LIBRARY

There is no shipped assembler library for FujiNet on the Color Computer, so this book brings one. It is nine subroutines in about two hundred lines, it allocates nothing but its own frames, and every assembly example in the reference chapters calls into it. Appendix C.3 prints it whole.

fn.inc

Before the code, the names. `fn.inc` is two hundred EQUs: every DriveWire opcode, every command byte on both devices, the open and translation modes, the parser modes, the hash algorithms, every error code, and the sizes of the fixed-length replies.

6809 ASSEMBLER

```
DWREADV EQU    $D93F
DWWRITEV EQU    $D941
OPFUJI EQU      $E2
OPNET EQU       $E3
FNREADY EQU     $00
FNRESP EQU      $01
FNERROR EQU     $02
FCADCFX EQU     $C4
SZADCFX EQU     240
ESUCC EQU       1
```

Nothing in it generates a byte. Include it and use the names; the reference chapters use the same ones.

FNWAIT

The first routine and the one everything else starts with. Write `$E2 $00`, read one byte, and if nothing comes back, ask again.

6809 ASSEMBLER

```
FNWAIT pshs    d,x,y
FNWAIT1 ldx     #FNRDYF
        ldy     #2
        jsr     [DWWRITEV]
        ldx     #FNSCRIP
        ldy     #1
        jsr     [DWREADV]
        bne     FNWAIT1      ; nothing yet
        puls    d,x,y,pc

FNRDYF fcb      OPFUJI, FNREADY
FNSCRIP rmb     1
```

It loops forever by design. `DWRead` times out after a second and a half, so each pass is bounded; what is unbounded is how many passes it takes, and the adapter is allowed to be busy for as long as a directory listing needs. If you want a program that gives up, count the passes yourself — but do the seven-byte probe from Chapter 3 first and you will not need to.

FNXACT and FNGERR

`FNXACT` sends a whole frame and brings back the verdict. It does that by falling straight into `FNGERR`, which is the tidiest thing in the file:

6809 ASSEMBLER

```
FNXACT  jsr      FNWAIT
        jsr      [DWWRTIV]    ; X, Y consumed
*       fall into FNGERR

FNGERR  pshs     x,y
        jsr      FNWAIT
        ldx      #FNERRF
        ldy      #2
        jsr      [DWWRTIV]
        ldx      #FNSCRIP
        ldy      #1
        jsr      [DWREADV]
        bne      FNGE1        ; no answer at all
        lda      FNSCRIP
        bra      FNGE2
FNGE1   lda      #EGENERL     ; call silence fatal
FNGE2   puls     x,y
        cmpa     #ESUCC
        rts
```

Two things worth pointing at. The `CMPA #ESUCC` at the end sets the zero flag when the byte was `$01`, so every caller in this book ends `jsr FNXACT / bne FAILED` and reads naturally. And silence is folded into `$90` — a general error — rather than being reported as success, because a command whose verdict never arrived did not succeed.

FNGRSP

Fetch whatever the last command parked. You supply the buffer and the count; the count is not negotiable, it is on the command's card.

6809 ASSEMBLER

```
FNGRSP  pshs     x,y
        jsr      FNWAIT
        ldx      #FNRSPPF
        ldy      #2
        jsr      [DWWRTIV]
        puls     x,y
        jmp      [DWREADV]
```

The `JMP` at the end is not a shortcut, it is the return: whatever `DWRead` sets the zero flag to is what the caller tests.

FNCMD0 and FNCMD1

Most control-device commands take no parameters or one byte, so two shorthands cover a great deal of Chapter 12:

6809 ASSEMBLER

```
FNCMD0  sta      FNC0F+1
        ldx      #FNC0F
        ldy      #2
        bra      FNXACT

FNCMD1  sta      FNC1F+1
        stb      FNC1F+2
        ldx      #FNC1F
        ldy      #3
        bra      FNXACT

FNC0F   fcb      OPFUJI,0
FNC1F   fcb      OPFUJI,0,0
```

So mounting host slot 2 is:

6809 ASSEMBLER

```
        lda      #FCMNTHS    ; $F9
        ldb      #2
        jsr      FNCMD1
        bne      FAILED
```

COUNT THE PARAMETERS

FNCMD1 sends exactly one parameter byte. A command that takes two — READ_DIR_ENTRY and MOUNT_IMAGE among them — needs its own frame, because the adapter will read the second byte whether you sent it or not. Chapter 17's mounter builds a four-byte frame for exactly this reason.

The network four

`fnnet.asm` adds the five moves. They share one unit byte, `NTUNI`, which `NTOPEN` sets from the `devicespec`:

`NTOPEN`..... `X` = `devicespec`, `A` = mode, `B` = translation
`NTCLOS`..... no arguments
`NTSTAT`..... `X` = a four-byte buffer
`NTREAD`..... `X` = buffer, `Y` = count
`NTWRIT`..... `X` = buffer, `Y` = count

plus `NTXACT`, `NTGERR` and `NTGRSP` for the other twenty-four network commands, which take the unit in `B`. To drive two channels at once, keep your own unit bytes and call those three directly; the five convenience routines are single-channel on purpose.

Register discipline

The ROM routines are not gentle. `DWRead` returns a checksum in `Y` and clobbers all three accumulators; `DWWrite` consumes both index registers. Every routine in `fnlow.asm` therefore states in its header exactly what it keeps, and keeps it.

The one to remember: after any call that ends in `DWRead`, your count register is gone. Push it if you need it.



7. C ON THE COLOR COMPUTER

CMOC compiles C for the 6809 and, by default, for the Disk BASIC environment: the output is a `.BIN` that `LOADM` loads and `EXEC` starts. It has the standard library you would expect of a C for an eight-bit machine — `printf`, `strcpy`, `malloc` — and a header of Color Computer particulars in `<coco.h>`: `cls()`, `inkey()`, `waitkey()`, `width()`, `locate()`.

What CMOC gives you

Two things matter for this book.

The first is that the 6809 is big-endian, so a struct of `uint8_t` and `uint16_t` fields is already in wire order. Chapter 4 made this point and it is worth making twice: nowhere in a Color Computer FujiNet program do you byte-swap anything.

The second is that CMOC does not pad structures. A struct `{ uint8_t a, b, c; uint16_t d; }` is five bytes, in that order, always. Every frame in this book is built that way.

The library's shape

`fujinet-lib` is one call per command, named after the command, taking the parameters in the order the frame wants them:

CMOC C

```
bool fuji_mount_host_slot(uint8_t hs);
bool fuji_mount_disk_image(uint8_t ds,
                             uint8_t mode);
bool fuji_read_directory(uint8_t maxlen,
                         uint8_t aux2,
                         char *buffer);

uint8_t network_open(const char *spec,
                     uint8_t mode,
                     uint8_t trans);
int16_t network_read(const char *spec,
                    uint8_t *buf,
                    uint16_t len);
```

Note that the network calls take the **devicespec**, not a unit number. The library digs the unit out of the string each time — `N:` is unit 1, `N2:` is unit 2 — so you pass the same string you opened with and the library keeps the bookkeeping.

Reading the return

This is where people get caught, because the two halves of the library answer differently.

The `fuji_` calls return a `bool`: true if it worked. The raw byte from the adapter is left in the global `fn_device_error`, so that is what you print when something fails.

CMOC C

```
if (!fuji_mount_host_slot(0))
    printf("NO: %u\n", fn_device_error);
```

The `network_` calls return a `uint8_t` error code — and it is **not** the adapter's byte, it is a small device-agnostic code:

`FN_ERR_OK` 0 — it worked
`FN_ERR_IO_ERROR` 1 — the device complained

```

FN_ERR_BAD_CMD.....2 — bad arguments
FN_ERR_OFFLINE.....3 — no device
FN_ERR_WARNING.....4 — non-fatal
FN_ERR_NO_DEVICE.....5 — nothing there
FN_ERR_UNKNOWN.....$FF

```

so a success test is == FN_ERR_OK and not != 0-as-truth, and the adapter's real byte is again in fn_device_error. On this platform the mapping is blunt: exactly \$01 becomes FN_ERR_OK and everything else becomes FN_ERR_IO_ERROR. If you want to know that it was \$88 — end of file — rather than \$CA — connection refused — read fn_device_error yourself.

network_read() is the exception to the exception: it returns the number of bytes it got, or the **negative** of an error code.

CMOC C

```

n = network_read(spec, buf, want);
if (n < 0)
    fail(fn_device_error);

```

Dropping below the library

fujinet-lib does not wrap every command, and one or two of the calls it does provide disagree with the current firmware about which aux byte carries the parameter. When that happens you do not have to leave C: the bus primitives are in the library even though the headers keep them private.

CMOC C

```

extern void    bus_ready(void);
extern byte    dwwrite(byte *s, int l);
extern byte    dwread(byte *s, int l);
extern uint8_t network_get_error(uint8_t unit);
extern uint8_t network_get_response(uint8_t unit,
                                     uint8_t *buf,
                                     int len);

```

Declare the ones you need, build the frame as a structure, and send it. Here is the one from Chapter 18, which selects the JSON parser — a command whose mode the firmware reads out of aux2:

CMOC C

```

struct
{
    uint8_t op, unit, cmd, aux1, aux2;
} f;

f.op   = 0xE3;
f.unit = unit;
f.cmd  = 0xFC;           /* SET_PARSER */
f.aux1 = 0;
f.aux2 = 1;              /* JSON */

bus_ready();
dwwrite((byte *) &f, sizeof(f));
e = network_get_error(unit);

```

Five bytes, and it is the same five the card in Chapter 13 shows. There is no layer to fight: the library and your own frames are the same conversation.

Inline assembly

CMOC will take a block of 6809 wherever a statement can go, and inside it a parameter or local is named with a leading colon. This is how the library reaches the ROM vectors, and it is how you would reach your own:

CMOC C

```
byte dwwrite(byte *s, int l)
{
    asm
    {
        pshs x,y
        ldx :s
        ldy :l
        jsr [0xD941]
        tfr cc,d
        puls y,x
    }
}
```

The value left in B when the block ends is the function's return value. Labels inside a block want an @ prefix so that repeated inlining does not collide.

Memory

CMOC's default origin for a Disk BASIC program is set by --org=, and the default leaves room under it for BASIC. A FujiNet program's appetite is mostly buffers, and the two that matter are the 256-byte devicespec field in every network frame and whatever you read into. Neither is large, but both are easy to put on the stack by accident:

KEEP THE FRAMES STATIC

A network frame is 261 bytes. A local one eats a quarter of a small stack and CMOC will not warn you. Declare frame structures and read buffers static — every listing in this book does.

8. THE NETWORK DEVICE

Device \$E3 is a bank of channels. Each one is identified by a unit number, each unit is created the moment you first address it, and what a channel **is** depends entirely on the URL you opened it with. A TCP socket, a file on a TNFS share, an HTTP response, a directory listing — all of them are the same five commands over the same five bytes of frame.

Devicespecs and units

A devicespec is the N prefix, an optional unit digit, a colon, and a URL:

```
N:TCP://fujinet.online:6502/
N2:HTTPS://api.example.com/v1/now
N3:TNFS://tnfs.fujinet.online/COCO/
```

N: with no digit is unit 1. N2: through N8: are units 2 to 8 by convention — the firmware will create any unit from 0 to 255, but eight is what every client library parses and eight is what you should stay inside.

The unit is what goes in the frame's second byte. fujinet-lib works it out from the string every time you call; the assembly library works it out once, in NTOPEN, and remembers it.

Open modes

The first aux byte of OPEN says what you want to do:

Mode	Access
\$04	read
\$06	directory
\$07	directory, alternate form
\$08	write
\$09	append

Mode	Access
\$0C	read and write

For HTTP: and HTTPS: the same byte chooses the method, which is why the numbers are not contiguous:

Mode	Method
\$04	GET
\$05	DELETE
\$08	PUT
\$09	DELETE, collecting headers
\$0C	GET, collecting headers
\$0D	POST
\$0E	PUT, collecting headers

Translation

The second aux byte rewrites line endings on the way through:

```
$00..... none — the bytes are the bytes
$01..... CR — the Color Computer's own
$02..... LF
$03..... CR LF
$04..... PETSCII
```

The Color Computer's native line ending is a carriage return, \$0D, and the firmware knows it: when a protocol needs to emit a line ending and you have not said otherwise, that is what it emits. So \$00 is usually right for the CoCo and the translation modes are there for when you are reading something that insists on LF.

The five moves

\$4F OPEN. . . . mode and translation in the aux bytes, then a 256-byte devicespec field
\$43 CLOSE. nothing
\$53 STATUS. nothing; four bytes come back
\$52 READ. . . . the count in the aux word; the bytes come back through SEND_RESPONSE
\$57 WRITE. the count in the aux word, then the bytes
and the shape of a session is always the same:

6809 ASSEMBLER

```
ldx    #SPEC
lda    #OMRW
ldb    #TRNONE
jsr    NTOPEN
bne    BADOPEN
*      ... status, read, write ...
jsr    NTCLOS
```

Reading what is there

STATUS returns four bytes and they are the whole state of the channel:

bytes	waiting	bytes	waiting	connected	error
high		low			

The rule is: ask STATUS, then read **no more than** it said. Ask for more and the adapter pads the reply with zeros to the length you named, and you will spend an afternoon wondering where the zeros came from.

6809 ASSEMBLER

```
ldx    #STBUF
jsr    NTSTAT
ldd    STBUF          ; big-endian, so LDD
beq    NOTHING
cmpd   #RXMAX
bls    TAKEIT
ldd    #RXMAX          ; only what we can hold
TAKEIT ldx    #RXBUF
tfr    d,y
jsr    NTREAD
```

The third byte is the connection: while it is non-zero the far end is still there. The fourth is the channel's own error, and the one you will meet most is \$88 — 136, end of file. It is not a failure. It means this resource is finished; close the channel.

ZERO WAITING IS NOT AN ERROR

A TCP socket with nothing to say returns zero bytes waiting and a connected byte of one. That is the normal state of a network connection most of the time. The test for “we are done” is zero waiting and not connected, or an error byte of 136 — never zero waiting on its own.

Prefixes

Each unit keeps a current directory, and CHDIR sets it with the shell conventions you would hope for: . . goes up, a leading slash replaces the path but keeps the host, a bare scheme: resets everything, and anything else appends. An empty Nn: clears it. GETCWD reads it back.

Parsers

A channel can have a parser sitting on it — JSON, HTML with CSS selectors, or XML with a subset of XPath. You set the mode, tell it to parse, and then query it. The document stays on the adapter; only the answer to each query crosses the wire, which is the only reason a 64-kilobyte machine can read a modern API at all. Chapter 18 does this end to end.

Interrupts

The adapter will tell you when a channel has something, rather than making you poll: it drives the serial line's DCD on hardware, or DTR or RTS on FujiNet-PC, whenever bytes are waiting, the connection dropped, or an error other than success or end-of-file is pending. SET_INT_RATE sets how often it looks. It is not driven at all over DriveWire-over-IP, where there is no line to drive.

9. THE FUJI DEVICE

Device \$E2 is the adapter's own housekeeping: where your files live, which of them are in drives, what network you are on, and a handful of things the ESP32 can do that a 6809 would rather not.

Host slots

Eight of them, numbered 0 to 7. A host slot is a name — a TNFS server, or the literal SD for the card in the side of the cartridge — stored as 32 bytes.

READ_HOST_SLOTS returns all eight at once, 256 bytes, and WRITE_HOST_SLOTS takes all eight back. There is no read-one or write-one; you fetch the block, change a name, and put the block back.

CMOC C

```
HostSlot hosts[8];

fuji_get_host_slots((uint8_t *) hosts, 8);
strcpy((char *) hosts[2],
      "tnfs.example.com");
fuji_put_host_slots((HostSlot *) hosts, 8);
```

Naming a slot does not open it. MOUNT_HOST does that, and it is what turns a name into something you can list.

Device slots

Four of them on the Color Computer, numbered 0 to 3, and this is the one number where the CoCo differs from its cousins: the Atari has eight and the Apple II has ten. A device slot is 38 bytes:

host slot	mode	file name, 36 bytes
-----------	------	---------------------

READ_DEVICE_SLOTS returns all four, 152 bytes. The mode byte you get back is not the mode you set: the adapter ORs in \$40 when the drive is actually live, so

CMOC C

```
if (drives[i].mode & 0x40)
    /* something is mounted here */;
```

is the test for “is there a disk in it”, and the low bits — \$01 read, \$02 write — are how it was mounted.

Prefixes

Each host slot also carries a prefix, up to 256 bytes: the directory you are working in on that host. SET_HOST_PREFIX and GET_HOST_PREFIX take the slot number in the aux byte.

The prefix is what OPEN_DIRECTORY is relative to, so setting it once and then listing / repeatedly is how a file browser walks a tree without building paths.

Walking a directory

Three commands, and one of them has a parameter worth understanding.

1. `OPEN_DIRECTORY` takes the host slot in its aux byte and a 256-byte field holding the path. If there is room, a filename pattern can follow the path's terminating NUL: `"/" 00 "*.dsk" 00` lists only the disk images.
2. `READ_DIR_ENTRY` takes **two** aux bytes — the maximum length you will accept and a flags byte — and returns that many bytes. A directory's entries come back one per call, NUL-terminated inside the fixed width, with a trailing slash on the names that are directories. Two bytes of `$7F` mean the directory is finished.
3. `CLOSE_DIRECTORY` takes nothing.

The flags byte is where the detail lives. Set bit 7 and each entry is prefixed with a 13-byte block describing the file, in a layout that is the Color Computer's own:

yr	mo	dy	hr	mi	sec	size, 4 bytes, big-endian	dir	trunc	media
----	----	----	----	----	-----	---------------------------	-----	-------	-------

The year is `tm_year - 100`, so 26 is 2026. The size is four bytes high byte first — the one place in the firmware where a 32-bit value is deliberately put in the Color Computer's order. The flags byte is `$01` for a directory and `$02` for a name that had to be truncated to fit.

Set both bits 7 and 6 and you get block mode instead: the maximum-length byte becomes a count of 256-byte pages and the low six bits of the flags become a group size, and the adapter packs as many entries as will fit into one transfer. It is much faster for a long directory and the card in Chapter 12 gives the header layout.

App keys

An app key is 64 bytes of state the adapter will keep for you, filed under your creator ID, application ID and key number. It is how a game remembers a high score across a power cycle without owning a disk.

1. `OPEN_APPKEY` with a six-byte payload: creator (two bytes, high first), application, key, mode — 0 to read, 1 to write — and a reserved byte.
2. `READ_APPKEY` returns 66 bytes: a two-byte length, high byte first, and then the 64. `WRITE_APPKEY` takes the length in its aux word and then always the full 64 bytes, whatever the length says.
3. `CLOSE_APPKEY` when you are done.

WRITE ALL SIXTY-FOUR

The write path drains a full 64-byte block regardless of the length you declared. Send fewer and the adapter keeps reading into your next command. The firmware says so in a comment, which is how you know somebody found out the hard way.

The adapter's spare cycles

The ESP32 is a great deal faster than the 6809 and is sitting there anyway. Four families of command lend it out:

- **Base64**, in and out, encode and decode, four commands each way: feed input, compute, ask the length, take the output.
- **Hashing** — MD5, SHA-1, SHA-256, SHA-512 — with the same shape, plus a variant that keeps the input so you can hash a stream in pieces.
- **QR codes**, which will render to an ANSI or bitmap form you can put on a screen that has no business displaying a QR code.
- **A UUID and four random bytes**, for when you need either and would rather not write the code.

All four are in Chapter 12 with their exact frames.

10. DEVICES, SLOTS AND BOOTING

A drive slot has a file name in it and, separately, a state: mounted or not. Those are two commands, and running them in the wrong order is the most common way to get a drive that looks right and does nothing.

Getting a file into a drive

1. MOUNT_HOST the host slot the file lives on. Naming a host is not opening it.
2. SET_DEVICE_FULLPATH writes the file name into the drive slot. Its aux bytes are the drive slot, the host slot, and a mode; a mode of \$00 stores the name without touching the file.
3. MOUNT_IMAGE opens it. Its two aux bytes are the drive slot and the access mode.

CMOC C

```
fuji_mount_host_slot(hs);  
fuji_set_device_filename(0, hs, ds, name);  
fuji_mount_disk_image(ds, 2); /* r/w */
```

Chapter 17's mouter does exactly this with a menu on the front, and its assembly half builds the two payload frames by hand because neither command fits FNCMD1.

MOUNT_ALL does the lot in one call, from whatever the four slots already hold — which is how a saved configuration comes back after a power cycle.

Access modes

\$01..... read only
\$02..... read and write
\$40. . set by the adapter in a slot you read back: this drive is live. Never send it.

Mount an image \$01 and a write to it fails at the disk layer, which is what you want for something you are only booting.

Making a blank disk

NEW_DISK is one of the two commands on this bus with a Color-Computer-only payload. It takes no aux bytes and a 259-byte block:

disks	host	drive	file name, 256 bytes
-------	------	-------	----------------------

The first byte is a multiplier, not a size. Each unit is 315 blocks of 512 bytes — 161,280 bytes, which is exactly a 35-track, 18-sector, single-sided Disk BASIC diskette — and the whole thing is filled with \$FF. One gives you one diskette; four gives you a four-disk image for a drive that knows how to step through them.

The file must not already exist. If it does, the command fails rather than truncating something you wanted.

What an image can be

The extension decides what the adapter thinks it is serving:

.DSK..... a Disk BASIC diskette, 256-byte sectors
.VDK..... the Dragon format
.MRM, .RMM..... raw block images
.CAS..... a cassette image, served as blocks
.ROM, .CCC..... a cartridge image — see below

A .ROM is not served over DriveWire at all. On cartridges that have a coprocessor, mounting one pushes the whole image across to it at mount time, and it appears as a cartridge. On hardware that has no coprocessor, and over DriveWire-over-IP, it is not supported.

Boot modes

SET_BOOT_MODE takes one byte and chooses what the adapter serves as drive 0 when nothing else is mounted:

0..... /AUTORUN.DSK
1..... /MOUNT-AND-BOOT.DSK
2..... the Game Lobby, over TNFS
3..... /HISIDBOOT-FUJINET.DSK

and CONFIG_BOOT turns the CONFIG disk on and off with a single byte. Turn it off and the adapter stops inserting CONFIG ahead of your disk, which is what you want when your own program is the thing that should come up.

RESET PUTS CONFIG BACK

Any of the three reset opcodes — \$F8, \$FE, \$FF — re-arms the CONFIG disk and re-inserts the boot device. That is deliberate: it means there is always a way back to a working machine. It also means a program that turns CONFIG off and then resets has not achieved anything.

Living with HDB-DOS

The thing to keep in mind while writing any of this is that HDB-DOS is using the same wire. It thinks it has a disk; it does not know the disk is being swapped underneath it.

- Do not unmount or re-mount a drive image while a file is open on it from BASIC. Close the file first. The adapter will do exactly as it is told and DOS will be looking at a different disk than it remembers.

- A program that mounts its own next disk should do the mounting and then hand control over — not mount and keep running out of the disk it just replaced.
- Drive numbers in BASIC and drive slots in FujiNet are the same numbers: slot 0 is DRIVE #0. There is no offset to remember, which is a small mercy.

THE DISK IS NOT A DEVICE

There is no command language for the disk. It answers exactly two opcodes — OP_READEX and OP_WRITE — and everything about which disk it is answering as comes from the slot commands in this chapter. Chapter 14 documents the block protocol for the benefit of anyone writing their own DOS.

11. FUJINET UNDER OS-9 AND NITROS-9

OS-9 boots over DriveWire perfectly well, and a FujiNet is a DriveWire server, so a NitrOS-9 system running off a FujiNet is an ordinary thing that many people have. What is not ordinary is issuing \$E2 and \$E3 commands from inside it, and this chapter is about why and what to do.

What still works

The disk service. OP_READEX and OP_WRITE are the protocol the OS-9 DriveWire driver already speaks, and an image mounted in a FujiNet drive slot is a device descriptor away from being **done**. Nothing in this chapter is needed for that.

The adapter also answers OP_DWINIT — the feature enquiry OS-9's driver makes at startup — with \$04 every time, whatever its configured feature set says.

What stops working

Everything else in this book, and for one reason: under OS-9 the Disk BASIC ROM is not in the memory map. \$D93F and \$D941 are not vectors any more; they are whatever the MMU has put there. Every routine in Chapter 6, and the whole of fujinet-lib's Color Computer build, calls through them.

So fujinet-lib cannot be linked into an OS-9 module. It is not a packaging problem — the library is Disk BASIC code by construction.

Supplying the two

The fix is the shape of the design. Only two routines touch the hardware. Replace them and everything above is unchanged.

For the memory-mapped ports — the Becker port, the CoCo 3 FPGA's DriveWire window, the high-speed UART cartridge — that replacement is about a dozen instructions. Status at \$FF41, data at \$FF42, bit 1 of the status byte meaning a byte is waiting:

6809 ASSEMBLER

```
@byte    ldu      #$2000      ; a timeout
@wait     lda      $FF41
          bita     #$02
          bne      @got
          leau     -1,u
          bne      @wait
          bra      @timeout
@got      lda      $FF42
          sta      ,x+
          leay     -1,y
          bne      @byte
```

Two details that are easy to miss, and both of them cost real time to find:

- **Mask interrupts.** These ports hold one byte. An OS-9 clock tick between the poll and the read loses it. Frames are short and the mask is shorter than the transfer.
- **Keep the timeout.** The ROM's DWRead gives up after about a second and a half, and FNWAIT depends on that: it asks again when the adapter is busy. A poll loop with no timeout hangs there instead of retrying, and it will look exactly like a dead adapter.

The stack above

Chapter 6's library is assembly for Disk BASIC, so the OS-9 listing in this book rebuilds the same four routines in C, above the transport:

CMOC C

```
void fn_wait(void);
byte fn_error(void);
byte fn_send(byte *frame, int len);
byte fn_response(byte *buf, int len);
```

Put them beside FNWAIT, FNGERR, FNXACT and FNGRSP and they are the same routines in a different language. The handshake did not change, the frames did not change, and the commands in Chapters 12 to 15 are all still true. Only the bottom two calls moved.

CMOC C

```
byte fn_send(byte *frame, int len)
{
    fn_wait();
    dwwrite(frame, len);
    return fn_error();
}
```

Chapter 19's fnstat is thirty lines on top of that, and prints the same three fields Chapter 5 printed.

AT THE PROMPT

```
$ cmoc --os9 -o fnstat \
    fnstat.c fnbus.c dwport.c
$ os9 copy fnstat /dd/CMDS/fnstat
```

The bit-banger

Which leaves the case that matters most, and the one this book will not print: a FujiNet cartridge on a stock Color Computer hangs off the bit-banger port, and a correct bit-banger DWRead is a cycle-counted routine with the loop timing chosen per CPU speed.

It exists, it is good, and it is not ours. It lives in the DriveWire and NitrOS-9 sources as DWRead and DWWrite, maintained by people with oscilloscopes, and it has been retuned more than once. Reprinting it here would be copying somebody else's careful work into a book that cannot be updated when they improve it.

Take it from your NitrOS-9 or DriveWire source tree, put it behind the two names in dwport.h, and nothing else in Chapter 19 changes. That is the whole point of having built on the vectors.

OR ASK NITROS-9 FOR ITS COPY

NitrOS-9 already has those routines resident — they are how your boot disk arrived. They live in a subroutine module which a program can F\Link and call at fixed offsets from its execution entry. That is the tidiest route on a running system, and the offsets must come from your own NitrOS-9 build rather than from this book.

What this book has verified

Being plain about it, because the rest of this book is:

- The C stack in Chapter 19 compiles with `cmoc --os9` and is printed in Appendix C from the file that compiled.
- The memory-mapped transport is correct by inspection of the register map and has not been run against hardware for this book.
- The bit-banger route is described, not supplied.
- Everything from `fn_wait()` upward is the same protocol as the rest of the book, and that part is verified everywhere else in it.

12. COMMAND REFERENCE: THE FUJI DEVICE

Every command device \$E2 answers on the Color Computer, verified against the firmware's dispatch tables — the shared handler map in `fujiDevice.cpp`, the four mixins beside it, and the two commands `drivewireFuji.cpp` adds — and cross-checked against the `fujinet-lib` Color Computer port, whose structures fix the byte layout of every frame it sends.

Conventions. *Frame* gives the bytes in order. *Aux* are the parameter bytes that follow the command byte; the FujiNet control device takes as many as the command wants and no more. A *byte* parameter is one byte; a *word* is two, high byte first, unless the card says otherwise. *Payload* is the block that follows the aux bytes. *Reply* is what `SEND_RESPONSE` will give you and exactly how many bytes to ask for.

Every card carries the command written twice: once in 6809 assembly using the library in Chapter 6, once in C using `fujinet-lib` where it wraps the command and raw frames where it does not. Any command not in this chapter is not answered on this bus; the closing section names them all.

12.1 The bus answers three

These three never reach the device. The bus intercepts them, on \$E2 and on \$E3 alike, and they are the handshake every other card assumes has already happened.

DEVICE_READY

\$00

FRAME	E2 00
AUX	none
PAYLOAD	none
REPLY	one byte, always \$01
FUJINET-LIB	<code>bus_ready()</code>

Are you listening? The adapter answers when it is free and says nothing when it is not, so the only correct use is a loop. Send every command behind one of these; a command sent into a busy adapter is simply lost.

6809 ASSEMBLER

```
FNWAIT pshs    d,x,y
FNWAIT1 ldx     #FNRDYF
        ldy     #2
        jsr     [DWWRITV]
        ldx     #FNSCRIP
        ldy     #1
        jsr     [DWREADV]
        bne     FNWAIT1
        puls    d,x,y,pc
FNRDYF fcb     OPFUJI,FNREADY
```

CMOC C

```
void bus_ready(void)
{
    struct { uint8_t op, cmd; } rc;
    uint8_t r;

    rc.op = 0xE2;
    rc.cmd = 0x00;

    do
        dwwrite((uint8_t *) &rc, sizeof(rc));
    while (!dwwread(&r, 1));
}
```

SEND_ERROR

\$02

FRAME	E2 02
AUX	none
PAYLOAD	none
REPLY	one byte: \$01 for success, otherwise a code from Appendix B
FUJINET-LIB	fuji_get_error(), leaving the byte in fn_device_error

How did the last command go? Treat silence as a failure and not as a success: a verdict that never arrived is not a good verdict. Note that the error state belongs to the whole bus and not to one device, so fetch it before you start anything else.

6809 ASSEMBLER

```
        jsr     FNWAIT
        ldx     #FNERRF
        ldy     #2
        jsr     [DWWRITV]
        ldx     #FNSCRIP
        ldy     #1
        jsr     [DWREADV]
        bne     NOANSWER
        lda     FNSCRIP
        cmpa    #ESUCC
FNERRF fcb     OPFUJI,FNERROR
```

CMOC C

```
bool fuji_get_error(void)
{
    struct { uint8_t op, cmd; } sec;
    uint8_t err;

    sec.op = 0xE2;
    sec.cmd = 0x02;

    bus_ready();
    dwwrite((uint8_t *) &sec, sizeof(sec));

    return dwwread(&err, 1) ? err != 1 : 1;
}
```

SEND_RESPONSE

\$01

FRAME	E2 01
AUX	none
PAYLOAD	none
REPLY	as many bytes as the last command parked — you must know the count
FUJINET-LIB	fuji_get_response(buf, len)

Hand over what the last command left waiting. There is no length on the wire: the count on each card is the count, and reading fewer bytes than the adapter has queued leaves the rest in the pipe for the next command to trip over.

6809 ASSEMBLER

```
FNGRSP  pshs    x,y
        jsr     FNWAIT
        ldx     #FNRSPPF
        ldy     #2
        jsr     [DWWRITV]
        puls    x,y
        jmp     [DWREADV]
FNRSPPF  fcb     OPFUJI,FNRESP
```

CMOC C

```
bool fuji_get_response(uint8_t *buf, int len)
{
    struct { uint8_t op, cmd; } grc;

    grc.op = 0xE2;
    grc.cmd = 0x01;

    bus_ready();
    dwwrite((uint8_t *) &grc, sizeof(grc));
    return dwread(buf, len) != 0;
}
```

12.2 WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED

\$C4

FRAME	E2 C4
AUX	none
PAYLOAD	none
REPLY	240 bytes — ssid[33] hostname[64] then the binary localIP gateway netmask dnsIP four bytes each, mac[6] bssid[6] fn_version[15] at offset 125, then the text forms: sLocalIP[16] at 140, sGateway[16], sNetmask[16], sDnsIP[16], sMacAddress[18] at 204, sBssid[18]
FUJINET-LIB	fuji_get_adapter_config_extended(&ac)

The first command to send and the one every status screen wants. It takes nothing, it answers before the WiFi is up, and its text fields spare you writing a print-an-address routine before you can see anything. The shorter GET_ADAPTERCONFIG returns the first 140 bytes only, with the addresses in binary.

6809 ASSEMBLER

```
    lda    #FCADCFX
    jsr    FNCMD0
    bne    FAILED
    ldx    #CFG
    ldy    #SZADCFX      ; 240
    jsr    FNGRSP
    bne    FAILED
; ssid  CFG+0
; fw    CFG+125
; ip    CFG+140
```

CMOC C

```
AdapterConfigExtended ac;

if (fuji_get_adapter_config_extended(&ac)) {
    printf("%s\n", ac.ssid);
    printf("%s\n", ac.sLocalIP);
    printf("%s\n", ac.fn_version);
}
```

GET_ADAPTERCONFIG

\$E8

FRAME	E2 E8
AUX	none
PAYLOAD	none
REPLY	140 bytes — the first 140 of the extended form: ssid[33] hostname[64] localIP[4] gateway[4] netmask[4] dnsIP[4] mac[6] bssid[6] fn_version[15]
FUJINET-LIB	fuji_get_adapter_config(&ac)

The compact form, for when you want the addresses as numbers rather than as text — to compare two of them, say. A hundred bytes cheaper than \$C4 and a print routine more expensive.

6809 ASSEMBLER

```
    lda    #FCADCFG
    jsr    FNCMD0
    bne    FAILED
    ldx    #CFG
    ldy    #SZADCFG      ; 140
    jsr    FNGRSP
```

CMOC C

```
AdapterConfig ac;

if (fuji_get_adapter_config(&ac))
    printf("%u.%u.%u.%u\n",
           ac.localIP[0], ac.localIP[1],
           ac.localIP[2], ac.localIP[3]);
```

GET_WIFISTATUS

\$FA

FRAME	E2 FA
AUX	none
PAYLOAD	none
REPLY	one byte: \$03 connected, \$06 disconnected
FUJINET-LIB	fuji_get_wifi_status(&s)

One byte, and the fastest way to tell “the cable is fine but you are not on a network” from “there is no adapter”. The other values in the firmware’s status enumeration — \$01 no SSID available, \$04 connect failed, \$05 connection lost — belong to the radio and you will normally see only the two above.

6809 ASSEMBLER

```
lda    #FCWIFST
jsr    FNCMD0
bne    FAILED
ldx    #WST
ldy    #1
jsr    FNGRSP
lda    WST
cmpa   #WFCONN      ; $03
bne    OFFLINE
```

CMOC C

```
uint8_t s;

if (fuji_get_wifi_status(&s) && s == 3)
    /* on the network */;
```

GET_WIFI_ENABLED

\$EA

FRAME	E2 EA
AUX	none
PAYLOAD	none
REPLY	one byte: non-zero if the radio is switched on
FUJINET-LIB	fuji_get_wifi_enabled()

Whether the radio is enabled at all, which is a different question from whether it has joined anything. A FujiNet with WiFi switched off is still a perfectly good disk server.

6809 ASSEMBLER

```
lda    #FCWIFEN
jsr    FNCMD0
bne    FAILED
ldx    #WEN
ldy    #1
jsr    FNGRSP
```

CMOC C

```
if (!fuji_get_wifi_enabled())
    printf("RADIO IS OFF\n");
```

SCAN_NETWORKS

\$FD

FRAME	E2 FD
AUX	none
PAYLOAD	none
REPLY	one byte: how many networks were found
FUJINET-LIB	fuji_scan_for_networks(&n)

Sweeps the band and returns the count. This one takes real time — the radio has to listen on every channel — so FNWAIT will spin for a while before the verdict comes back. That is the loop working, not the loop failing.

6809 ASSEMBLER

```
lda    #FCSCAN
jsr    FNCMD0
bne    FAILED
ldx    #NCOUNT
ldy    #1
jsr    FNGRSP
```

CMOC C

```
uint8_t n;

if (fuji_scan_for_networks(&n))
    printf("%u NETWORKS\n", n);
```

GET_SCAN_RESULT

\$FC

FRAME	E2 FC n
AUX	<i>byte</i> : which result, 0 to count-1
PAYLOAD	none
REPLY	34 bytes: ssid[33] then a signed rssi byte
FUJINET-LIB	fuji_get_scan_result(n, &info)

One result at a time. The signal strength is a signed byte in dBm, so it is negative and bigger is worse: -45 is across the room and -85 is through two walls. The results live until the next scan.

6809 ASSEMBLER

```
lda    #FCSCANR
ldb    #0                ; first result
jsr    FNCMD1
bne    FAILED
ldx    #SSINFO
ldy    #SZSCANR          ; 34
jsr    FNGRSP
ldx    #SSINFO
jsr    PUTS              ; the name
lda    SSINFO+33          ; the strength
```

CMOC C

```
SSIDInfo info;
uint8_t i, n;

fuji_scan_for_networks(&n);
for (i = 0; i < n; i++)
    if (fuji_get_scan_result(i, &info))
        printf("%s %d\n", info.ssid,
                (signed char) info.rssi);
```

SET_SSID

\$FB

FRAME	E2 FB + 97
AUX	none
PAYLOAD	exactly 97 bytes : ssid[33] then password[64], both NUL-padded
REPLY	none
FUJINET-LIB	fuji_set_ssid(&nc)

Joins a network and stores it as the default. The payload is a fixed 97 bytes whatever the two strings are; clear the buffer first. A short payload leaves the adapter reading into your next command and the bus never recovers. Unlike some platforms, this one takes no dummy parameter byte.

6809 ASSEMBLER

```
    ldx    #SSFRM
    ldy    #99          ; 2 + 97
    jsr    FNXACT
    bne    FAILED

SSFRM    fcb    OPFUJI,FCSSSID
SSSSID   rmb    33
SSPASS   rmb    64
```

CMOC C

```
NetConfig nc;

memset(&nc, 0, sizeof(nc)); /* NUL-pad */
strcpy(nc.ssid, "HOME-2G");
strcpy(nc.password, "correcthorse");

if (fuji_set_ssid(&nc))
    /* joined, and saved as the default */;
```

GET_SSID

\$FE

```
FRAME    E2 FE
AUX       none
PAYLOAD   none
REPLY     97 bytes: ssid[33] then password[64]
FUJINET-LIB fuji_get_ssid(&nc)
```

Reads back the stored network — the one the adapter joins at power-up. Yes, it hands you the password as well. It is your adapter and your wire; treat the buffer accordingly.

6809 ASSEMBLER

```
    lda    #FCGSSID
    jsr    FNCMD0
    bne    FAILED
    ldx    #NETCFG
    ldy    #SZSSID      ; 97
    jsr    FNGRSP
```

CMOC C

```
NetConfig nc;

if (fuji_get_ssid(&nc))
    printf("DEFAULT: %s\n", nc.ssid);
```

12.3 Hosts and prefixes

READ_HOST_SLOTS

\$F4

```
FRAME    E2 F4
AUX       none
PAYLOAD   none
REPLY     256 bytes: eight slots of 32, each a NUL-terminated name
FUJINET-LIB fuji_get_host_slots(h, 8)
```

All eight at once. There is no read-one. A slot holding a TNFS server has its hostname; a slot pointing at the card in the side of the cartridge holds the literal SD; an unused slot's first byte is zero.

6809 ASSEMBLER

```
    lda    #FCRDHS
    jsr    FNCMD0
    bne    FAILED
    ldx    #HOSTS
    ldy    #SZHOSTS      ; 256
    jsr    FNGRSP
; slot n is at HOSTS + 32*n
```

CMOC C

```
HostSlot hosts[8];

if (fuji_get_host_slots((uint8_t *) hosts, 8))
    for (i = 0; i < 8; i++)
        printf("%u %s\n", i,
               hosts[i][0] ? (char *) hosts[i]
                           : "EMPTY");
```

WRITE_HOST_SLOTS

\$F3

FRAME E2 F3 + 256
AUX none
PAYLOAD 256 bytes: all eight slots, 32 each
REPLY none
FUJINET-LIB fuji_put_host_slots(h, 8)

And all eight go back. Read, change one, write: there is no other way to edit a single slot, and skipping the read means writing seven empty slots over somebody's configuration. Naming a slot does not open it — that is MOUNT_HOST.

6809 ASSEMBLER

```
    ldx    #WHFRM
    ldy    #258          ; 2 + 256
    jsr    FNXACT
    bne    FAILED

WHFRM fcb OPFUJI,FCWRHS
WHSLOT rmb 256
```

CMOC C

```
HostSlot hosts[8];

fuji_get_host_slots((uint8_t *) hosts, 8);
memset(hosts[2], 0, 32);
strcpy((char *) hosts[2], "tnfs.example.com");
fuji_put_host_slots(hosts, 8);
```

MOUNT_HOST

\$F9

FRAME E2 F9 n
AUX *byte*: the host slot, 0 to 7
PAYLOAD none
REPLY none
FUJINET-LIB fuji_mount_host_slot(hs)

Opens the host named in the slot: connects to the TNFS server, or mounts the card. Everything that reads a directory or mounts an image from that host needs this first, and it is the step people forget.

6809 ASSEMBLER

```
    lda    #FCMNTHS
    ldb    #0            ; host slot 0
    jsr    FNCMD1
    bne    FAILED
```

CMOC C

```
if (!fuji_mount_host_slot(0))
    printf("NO: %u\n", fn_device_error);
```

UNMOUNT_HOST

\$E6

FRAME E2 E6 n
AUX *byte*: the host slot, 0 to 7
PAYLOAD none
REPLY none
FUJINET-LIB **none that is correct** — see below

Closes the host and any files open on it. Worth doing before you pull a card out or change a server's name.

The Color Computer build of fujinet-lib has a copy-and-paste error here: `fuji_unmount_host_slot()` sends `FUJICMD_MOUNT_HOST` rather than `FUJICMD_UNMOUNT_HOST`, so it re-mounts the slot instead. The frame above is three bytes and is right.

6809 ASSEMBLER

```
lda    #FCUNMHS      ; $E6
ldb    #0
jsr    FNCMD1
bne    FAILED
```

CMOC C

```
/* fujinet-lib's fuji_unmount_host_slot()
   sends $F9 --- MOUNT --- not $E6.  Until
   that is fixed, send the frame. */
struct { uint8_t op, cmd, hs; } u;

u.op = 0xE2; u.cmd = 0xE6; u.hs = 0;
bus_ready();
dwwrite((uint8_t *) &u, sizeof(u));
ok = !fuji_get_error();
```

SET_HOST_PREFIX

\$E1

FRAME E2 E1 n + 256
AUX *byte*: the host slot, 0 to 7
PAYLOAD 256 bytes: the prefix, NUL-padded
REPLY none
FUJINET-LIB `fuji_set_host_prefix(hs, prefix)`

Sets the working directory on that host. Everything `OPEN_DIRECTORY` does afterwards is relative to it, which is how a file browser walks a tree without ever building a full path.

6809 ASSEMBLER

```
ldx    #SPFRM
ldy    #259           ; 3 + 256
jsr    FNXACT
bne    FAILED
```

```
SPFRM  fcb    OPFUJI,FCSHPFIX,0
SPPATH rmb    256
```

CMOC C

```
fuji_set_host_prefix(0, "/COCO/GAMES");
```

GET_HOST_PREFIX

\$E0

FRAME	E2 E0 n
AUX	<i>byte</i> : the host slot, 0 to 7
PAYLOAD	none
REPLY	256 bytes: the prefix, NUL-terminated inside
FUJINET-LIB	fuji_get_host_prefix(hs, buf)

Reads it back. Always 256 bytes on the wire whatever the string's length — ask for fewer and the rest stays in the pipe.

6809 ASSEMBLER

```
lda    #FCGHPFX
ldb    #0
jsr    FNCMD1
bne    FAILED
ldx    #PFX
ldy    #SZSPEC      ; 256
jsr    FNGRSP
```

CMOC C

```
char prefix[256];

if (fuji_get_host_prefix(0, prefix))
    printf("%s\n", prefix);
```

12.4 Directories

OPEN_DIRECTORY

\$F7

FRAME	E2 F7 n + 256
AUX	<i>byte</i> : the host slot, 0 to 7
PAYLOAD	256 bytes: the path, and optionally a NUL then a filename pattern then another NUL
REPLY	none
FUJINET-LIB	fuji_open_directory(hs, path) or fuji_open_directory2(hs, path, filter)

Opens a directory for reading. The pattern is optional and goes after the path's terminating NUL inside the same 256-byte field, with its own NUL after it; the adapter only looks for one if there is room. Clear the field before you build it.

6809 ASSEMBLER

```
*      "/" 00 "*.dsk" 00, padded to 256
      ldy    #ODPATH
      ldx    #MROOT
      jsr    SCOPY
      leay   1,y          ; over the NUL
      ldx    #MFILT
      jsr    SCOPY
      lda    HS
      sta    ODFRM+2
      ldx    #ODFRM
      ldy    #259
      jsr    FNXACT
```

```
ODFRM   fcb    OPFUJI,FCOPNDR,0
ODPATH  rmb    256
```

CMOC C

```
/* path and pattern, in one field */
fuji_open_directory2(hs, "/", "*.dsk");

/* or the whole field verbatim */
fuji_open_directory(hs, path_and_pattern);
```

READ_DIR_ENTRY

\$F6

FRAME E2 F6 len flags
AUX *byte*: the most bytes you will accept, usually 36.
 byte: flags — see below
PAYLOAD none
REPLY as many bytes as the first aux byte said
FUJINET-LIB fuji_read_directory(maxlen, flags, buf)

One entry per call, NUL-terminated inside the width you asked for. Directory names come back with a trailing slash. Two bytes of \$7F mean there is nothing left.

The flags byte: set bit 7 and the entry is prefixed with a 13-byte detail block — year (tm_year–100), month, day, hour, minute, second, then a four-byte size **high byte first**, then a directory flag (\$01), a truncated flag (\$02) and a media type. Set bits 7 and 6 together and the command switches to block mode: the length byte becomes a count of 256-byte pages, the low six bits of the flags become a group size, and the adapter packs as many entries as will fit into one transfer, behind a small header beginning 'M' 'F'.

6809 ASSEMBLER

```
*      two parameters, so FNCMD1 will not do
DIR1    ldx    #RDFRM
        ldy    #4
        jsr    FNXACT
        bne    DONE
        ldx    #ENTRY
        ldy    #36
        jsr    FNGRSP
        bne    DONE
        lda    ENTRY
        cmpa   #$7F          ; end of directory
        beq    DONE
```

```
RDFRM   fcb    OPFUJI,FCRDDIR,36,0
```

CMOC C

```
char entry[37];

while (fuji_read_directory(36, 0, entry))
{
    if ((uint8_t) entry[0] == 0x7F)
        break;          /* finished */
    printf("%s\n", entry);
}
```

CLOSE_DIRECTORY

\$F5

FRAME	E2 F5
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_close_directory()

Finishes with the directory. There is one directory handle for the whole adapter, so closing is not optional housekeeping — the next OPEN_DIRECTORY from anywhere depends on it.

6809 ASSEMBLER

```
lda    #FCCLSDR
jsr    FNCMD0
```

CMOC C

```
fuji_close_directory();
```

GET_DIRECTORY_POSITION

\$E5

FRAME	E2 E5
AUX	none
PAYLOAD	none
REPLY	two bytes — low byte first
FUJINET-LIB	fuji_get_directory_position(&pos)

Where you are in the open directory, so that you can come back to it.

This reply is little-endian, alone among the control device's two-byte values, because the firmware sends a raw host integer rather than a declared big-endian one. Its partner SET_DIRECTORY_POSITION reads big-endian. Swap it on the way in or on the way out, but do not do both.

6809 ASSEMBLER

```
lda    #FCGDIRP
jsr    FNCMD0
bne    FAILED
ldx    #POS
ldy    #2
jsr    FNGRSP
*      little-endian: POS is the low byte
ldb    POS
lda    POS+1
```

CMOC C

```
uint16_t pos;

fuji_get_directory_position(&pos);
/* CMOC reads this big-endian: swap it */
pos = (pos >> 8) | (pos << 8);
```

SET_DIRECTORY_POSITION

\$E4

FRAME E2 E4 hi lo
AUX *word*: the position, **high byte first**
PAYLOAD none
REPLY none
FUJINET-LIB fuji_set_directory_position(pos)

Puts the read position back where GET_DIRECTORY_POSITION found it. Big-endian here, little-endian there; the firmware is inconsistent and this card is the only warning you get.

6809 ASSEMBLER

```
        ldd     #17
        std     SDPFRM+2
        ldx     #SDPFRM
        ldy     #4
        jsr     FNXACT

SDPFRM  fcb     OPFUJI,FCSDIRP
        fdb     0
```

CMOC C

```
fuji_set_directory_position(17);
```

12.5 Device slots and mounting

READ_DEVICE_SLOTS

\$F2

FRAME E2 F2
AUX none
PAYLOAD none
REPLY 152 bytes: four slots of 38 — host slot, mode, file[36]
FUJINET-LIB fuji_get_device_slots(d, 4)

All four drives at once. Four, not eight — the Color Computer build declares four disk devices where the Atari declares eight. The mode byte you read back is not the mode you set: the adapter ORs in \$40 when the drive is actually live and is not the CONFIG disk. That bit is the test for “is there a disk in it”; the low bits say how it was mounted.

6809 ASSEMBLER

```
        lda     #FCRDDSD
        jsr     FNCMD0
        bne     FAILED
        ldx     #DRIVES
        ldy     #SZDEVS          ; 152
        jsr     FNGRSP
; slot n is at DRIVES + 38*n
```

CMOC C

```
DeviceSlot drives[4];

if (fuji_get_device_slots(drives, 4))
    for (i = 0; i < 4; i++)
        printf("%u%c %s\n", i,
            (drives[i].mode & 0x40) ? '*' : ' ',
            drives[i].file);
```

WRITE_DEVICE_SLOTS

\$F1

FRAME	E2 F1 + 152
AUX	none
PAYLOAD	152 bytes: all four slots, 38 each
REPLY	none
FUJINET-LIB	fuji_put_device_slots(d, 4)

Writes the configuration of all four back. This records what **should** be mounted; it does not mount anything. MOUNT_ALL acts on what this left behind, which is how a configuration survives a power cycle. Clear the \$40 bit before writing a slot back — it is the adapter's to set.

6809 ASSEMBLER

	ldx	#WDFRM	
	ldy	#154	; 2 + 152
	jsr	FNXACT	
WDFRM	fcb	OPFUJI,FCWRDS	
WDSLOT	rmb	152	

CMOC C

```
DeviceSlot drives[4];

fuji_get_device_slots(drives, 4);
drives[1].hostSlot = 0;
drives[1].mode = 2;
strcpy((char *) drives[1].file, "GAME.DSK");
fuji_put_device_slots(drives, 4);
```

SET_DEVICE_FULLPATH

\$E2

FRAME	E2 E2 ds hs mode + 256
AUX	<i>byte</i> : the drive slot, 0 to 3. <i>byte</i> : the host slot, 0 to 7. <i>byte</i> : mode — \$00 stores the name without opening it
PAYLOAD	256 bytes: the file name, NUL-padded
REPLY	none
FUJINET-LIB	fuji_set_device_filename(mode, hs, ds, name)

Puts a file name into a drive slot. Note the three aux bytes are drive, host, mode — in that order on the wire, which is not the order fujinet-lib's function takes them. The library reorders them for you; if you build the frame yourself, this card is the order that matters.

6809 ASSEMBLER

	lda	DS	
	sta	SDFRM+2	
	lda	HS	
	sta	SDFRM+3	
	clr	SDFRM+4	; mode 0
	ldx	#SDFRM	
	ldy	#261	; 5 + 256
	jsr	FNXACT	
SDFRM	fcb	OPFUJI,FCSDVFP,0,0,0	
SDNAME	rmb	256	

CMOC C

```
/* note the argument order: mode first */
fuji_set_device_filename(0, hs, ds,
    "GAMES/ZAXXON.DSK");
```

GET_DEVICE_FULLPATH

\$DA

FRAME E2 DA n
AUX *byte*: the drive slot, 0 to 3
PAYLOAD none
REPLY 256 bytes: the full path, NUL-terminated inside
FUJINET-LIB fuji_get_device_filename(ds, buf)

The whole path of what is in a drive, host prefix and all — where the 36-byte field in READ_DEVICE_SLOTS gives you only the name, and truncated at that.

6809 ASSEMBLER

```
lda    #FCGDVFP
ldb    #0
jsr    FNCMD1
bne    FAILED
ldx    #PATH
ldy    #SZSPEC      ; 256
jsr    FNGRSP
```

CMOC C

```
char path[256];

if (fuji_get_device_filename(0, path))
    printf("%s\n", path);
```

MOUNT_IMAGE

\$F8

FRAME E2 F8 ds mode
AUX *byte*: the drive slot, 0 to 3.
 byte: access — \$01 read, \$02 read and write
PAYLOAD none
REPLY none
FUJINET-LIB fuji_mount_disk_image(ds, mode)

Opens the file named in the slot and starts serving it as a disk. Two parameters, so it needs its own frame in assembly. Mount read-only and a write fails at the disk layer, which is what you want for anything you are only booting from.

6809 ASSEMBLER

```
lda    DS
sta    MIFRM+2
lda    #2          ; read and write
sta    MIFRM+3
ldx    #MIFRM
ldy    #4
jsr    FNXACT
bne    FAILED

MIFRM  fcb    OPFUJI,FCMNTIM,0,0
```

CMOC C

```
if (!fuji_mount_disk_image(ds, 2))
    printf("NO: %u\n", fn_device_error);
```

UNMOUNT_IMAGE

\$E9

FRAME E2 E9 n
AUX *byte*: the drive slot, 0 to 3
PAYLOAD none
REPLY none
FUJINET-LIB fuji_unmount_disk_image(ds)

Closes the image and empties the drive. Close any BASIC file open on that drive first: the adapter will do as it is told, and DOS will go on believing there is a disk there.

6809 ASSEMBLER

```
lda    #FCUNMIM
ldb    DS
jsr    FNCMD1
```

CMOC C

```
fuji_unmount_disk_image(ds);
```

MOUNT_ALL

\$D7

FRAME E2 D7
AUX none
PAYLOAD none
REPLY none
FUJINET-LIB fuji_mount_all()

Mounts everything the four device slots already name, using each slot's own access mode. One call to put a saved configuration back, and the usual last step of a program that has just written the slots.

6809 ASSEMBLER

```
lda    #FCMNTAL
jsr    FNCMD0
bne    FAILED
```

CMOC C

```
if (!fuji_mount_all())
    printf("SOMETHING DID NOT MOUNT\n");
```

NEW_DISK

\$E7

FRAME E2 E7 + 259
AUX none
PAYLOAD 259 bytes: disks, host slot, drive slot, then filename[256]
REPLY none
FUJINET-LIB fuji_create_new(&nd)

Creates a blank image and mounts it read-write. This is one of the two commands on this bus with a Color-Computer-only payload: the Atari's version sends a sector count and a sector size.

The first byte is a multiplier, not a size. Each unit is 315 blocks of 512 bytes — 161,280 bytes, exactly a 35-track, 18-sector, single-sided Disk BASIC diskette — filled with \$FF. The command fails rather than truncating if the file already exists.

6809 ASSEMBLER

```

        ldx    #NDFRM
        ldy    #261          ; 2 + 259
        jsr    FNXACT
        bne    FAILED

NDFRM   fcb    OPFUJI,FCNEWDK
NDNUM   fcb    1              ; one diskette
NDHOST   fcb    0
NDDRIVE  fcb    1
NDNAME   rmb    256

```

CMOC C

```

NewDisk nd;

memset(&nd, 0, sizeof(nd));
nd.numDisks = 1;
nd.hostSlot = 0;
nd.deviceSlot = 1;
strcpy(nd.filename, "NEW.DSK");

if (!fuji_create_new(&nd))
    printf("IT ALREADY EXISTS\n");

```

COPY_FILE

\$D8

```

FRAME      E2 D8 src dst + 256
AUX         byte: the source host slot.
            byte: the destination host slot

PAYLOAD     256 bytes: sourcepath NUL destpath NUL
REPLY       none
FUJINET-LIB fuji_copy_file(src, dst, spec)

```

Copies a file from one host to another without it passing through the Computer.

This command is broken on the DriveWire bus and will wedge it. The handler asks for the payload as a string but nothing ever tells the frame decoder how long that payload is, so the adapter sees an empty copy-spec, gives up — and leaves your 256 bytes sitting in its receive buffer, where they become the next command. There is no recovery short of a reset. fuji-net-lib sends the payload faithfully, which is precisely the problem. Leave this one alone until the firmware calls setDataLength() for it.

6809 ASSEMBLER

```

*      see the caution --- do not send this
*      to a current adapter
        ldx    #CFFRM
        ldy    #260
        jsr    FNXACT

CFFRM   fcb    OPFUJI,FCCOPYF,0,0
CFSPEC  rmb    256

```

CMOC C

```

/* fuji_copy_file(0, 1, "A.DSK\0B.DSK"); */

```

12.6 Booting and resetting

SET_BOOT_MODE

\$D6

FRAME	E2 D6 n
AUX	byte: 0 <code>/AUTORUN.DSK</code> · 1 <code>/MOUNT-AND-BOOT.DSK</code> · 2 the Game Lobby over TNFS · 3 <code>/HISIOBOOT-FUJINET.DSK</code>
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_set_boot_mode(mode)

Chooses what the adapter serves as drive 0 when nothing else is mounted. Mode 2 fetches the Game Lobby from `tnfs://tnfs.fujinet.online/COCO/lobby.dsk`, so it needs a network. On a Dragon it additionally arms the named-object path to `/DGNLOBBY.DWL`.

6809 ASSEMBLER

```
lda    #FCSBOOT
ldb    #2          ; the lobby
jsr    FNCMD1
```

CMOC C

```
fuji_set_boot_mode(2);
```

CONFIG_BOOT

\$D9

FRAME	E2 D9 n
AUX	byte: non-zero to keep the CONFIG disk, zero to stop inserting it
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_set_boot_config(toggle)

Turns the CONFIG disk on and off. Turn it off when your own program should come up instead. Note that any of the three reset opcodes turns it straight back on — deliberately, so that there is always a way back to a working machine.

6809 ASSEMBLER

```
lda    #FCCFGBT
clrb                   ; stop inserting CONFIG
jsr    FNCMD1
```

CMOC C

```
fuji_set_boot_config(0);
```

RESET

\$FF

FRAME	E2 FF
AUX	none
PAYLOAD	none
REPLY	none, and no acknowledgement — the adapter reboots
FUJINET-LIB	fuji_reset()

Reboots the adapter. The handler calls the reboot directly without bracketing the transaction, so the verdict you politely ask for afterwards will never come. Send it and move on; give the adapter a few seconds and probe with \$A5 before you assume anything.

Note that this is not the same as the bus-level reset opcodes \$F8, \$FE and \$FF from Chapter 15 — those re-arm the CONFIG disk without re-booting.

6809 ASSEMBLER

```
        lda    #FCRESET
        jsr    FNCMD0
*       do not expect the verdict to arrive
```

CMOC C

```
fuji_reset();
/* everything after this is on a new adapter */
```

12.7 App keys

An app key is 64 bytes the adapter keeps for you between power cycles, filed under a creator ID, an application ID and a key number. The files land on the microSD card as /FujiNet/ccccaakk.key, so a cartridge with no card in it cannot do this at all.

OPEN_APPKEY

\$DC

FRAME	E2 DC + 6
AUX	none
PAYLOAD	6 bytes: creator (word, high byte first), app, key, mode — 0 read, 1 write, 2 read 256 — and a reserved byte
REPLY	none
FUJINET-LIB	folded into fuji_read_appkey() and fuji_write_appkey()

Names the key and says which way you are going. The creator word is big-endian on the wire, which on a 6809 means you simply store it.

6809 ASSEMBLER

```
        ldd    #$4321            ; creator
        std    AKFRM+2
        lda    #1                ; app
        sta    AKFRM+4
        lda    #7                ; key
        sta    AKFRM+5
        clr    AKFRM+6            ; mode 0: read
        clr    AKFRM+7
        ldx    #AKFRM
        ldy    #8                ; 2 + 6
        jsr    FNXACT

AKFRM   fcb    OPFUJI,FCOPAPK
        fdb    0                ; creator
        fcb    0,0,0,0          ; app,key,mode,rsvd
```

CMOC C

```
fuji_set_appkey_details(0x4321, 1, DEFAULT);
/* the library opens and closes around
   each read or write */
```

READ_APPKEY

\$DD

FRAME	E2 DD
AUX	none
PAYLOAD	none
REPLY	66 bytes: a length (word, high byte first) then 64 bytes of data
FUJINET-LIB	fuji_read_appkey(key, &count, data)

Reads the open key. Always 66 bytes on the wire; the length tells you how many of the 64 mean anything. The two-byte header is the Color Computer build's own addition — other buses return the data alone.

6809 ASSEMBLER

```
lda    #FCRDAPK
jsr    FNCMD0
bne    FAILED
ldx    #AKBUF
ldy    #66
jsr    FNGRSP
ldd    AKBUF          ; the length
leax    2,x           ; the data
```

CMOC C

```
uint16_t count;
uint8_t data[66];

fuji_set_appkey_details(0x4321, 1, DEFAULT);
if (fuji_read_appkey(7, &count, data))
    /* count bytes are meaningful */;
```

WRITE_APPKEY

\$DE

FRAME	E2 DE hi lo + 64
AUX	<i>word</i> : how many bytes are meaningful, high byte first
PAYLOAD	always 64 bytes , whatever the length says
REPLY	none
FUJINET-LIB	fuji_write_appkey(key, count, data)

Writes the open key.

Send all sixty-four bytes. The handler drains a full block regardless of the length you declared, and the firmware carries a comment saying so, which is how you know somebody found out the hard way. Send fewer and the adapter reads on into your next command.

6809 ASSEMBLER

```
ldd    #12             ; meaningful bytes
std    WKFRM+2
ldx    #WKFRM
ldy    #68             ; 4 + 64
jsr    FNXACT

WKFRM   fcb    OPFUJI,FCWRAPK
        fdb    0             ; length
WKDATA  rmb    64
```

CMOC C

```
uint8_t data[64];

memset(data, 0, sizeof(data));
memcpy(data, "HIGH SCORE 4820", 15);
fuji_write_appkey(7, 15, data);
```

CLOSE_APPKEY

\$DB

FRAME	E2 DB
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	folded into the read and write calls

Finishes with the key. One key is open at a time for the whole adapter.

6809 ASSEMBLER

```
lda    #FCCLAPK
jsr    FNCMD0
```

CMOC C

```
/* the library closes for you */
```

12.8 Base64

Four commands each way, and the same shape both ways: feed the input, tell it to compute, ask how long the answer is, take the answer. The buffer lives on the adapter, so a 6809 never has to hold both forms of a string at once.

BASE64_ENCODE_INPUT / DECODE_INPUT

\$D0 / \$CC

FRAME	E2 D0 hi lo + len
AUX	<i>word</i> : how many bytes follow, high byte first. Zero is refused
PAYLOAD	that many bytes
REPLY	none
FUJINET-LIB	fuji_base64_encode_input(s, len) · fuji_base64_decode_input(s, len)

Appends to the adapter's buffer, so call it as many times as you like to build a long input a piece at a time.

6809 ASSEMBLER

```
ldd    #TEXTLEN
std    B64FRM+2
ldx    #B64FRM
ldy    #4
jsr    FNWAIT
jsr    [DWWRITV]    ; the header
ldx    #TEXT
ldy    #TEXTLEN
jsr    [DWWRITV]    ; the payload
jsr    FNGERR

B64FRM fcb    OPFUJI,FCB64EI
      fdb    0
```

CMOC C

```
fuji_base64_encode_input(text, strlen(text));
```

BASE64_ENCODE_COMPUTE / DECODE_COMPUTE

\$CF / \$CB

FRAME	E2 CF
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_base64_encode_compute() · fuji_base64_decode_compute()

Transforms the buffer in place. Input becomes output; there is one buffer and this is the moment it changes meaning.

6809 ASSEMBLER

```
lda    #FCB64EC
jsr    FNCMD0
bne    FAILED
```

CMOC C

```
if (!fuji_base64_encode_compute())
    fail();
```

BASE64_ENCODE_LENGTH / DECODE_LENGTH

\$CE / \$CA

FRAME	E2 CE
AUX	none
PAYLOAD	none
REPLY	4 bytes, high byte first
FUJINET-LIB	fuji_base64_encode_length(&len) · fuji_base64_decode_length(&len)

How long the result is. Four bytes because the adapter's buffer can be bigger than a Color Computer's memory; the top two will be zero for anything you can hold.

6809 ASSEMBLER

```
lda    #FCB64EL
jsr    FNCMD0
bne    FAILED
ldx    #B64LEN
ldy    #4
jsr    FNGRSP
*      B64LEN+2,+3 is the useful half
```

CMOC C

```
unsigned long len;

fuji_base64_encode_length(&len);
```

BASE64_ENCODE_OUTPUT / DECODE_OUTPUT

\$CD / \$C9

FRAME	E2 CD hi lo
AUX	<i>word</i> : how many bytes to take, high byte first. Zero, or more than the buffer holds, is refused
PAYLOAD	none
REPLY	that many bytes
FUJINET-LIB	fuji_base64_encode_output(s, len) — but see below

Takes bytes off the front of the buffer and removes them, so repeated calls walk a long result through a small Computer.

The Color Computer build of fujinet-lib does not send the aux word on either output call. The firmware reads it anyway, so two bytes of whatever you send next are consumed as the length and the bus goes out of step. Build the four-byte frame yourself until that is fixed; the same applies to QRCODE_OUTPUT.

6809 ASSEMBLER

```

        ldd     #64             ; the aux word
        std     B64OUT+2
        ldx     #B64OUT
        ldy     #4             ; 2 + 2 --- not 2
        jsr     FNXACT
        bne     FAILED
        ldx     #BUF
        ldy     #64
        jsr     FNGRSP

B64OUT  fcb     OPFUJI,FCB64E0
        fdb     0

```

CMOC C

```

/* fujinet-lib's output calls send no aux
   word; the firmware reads two bytes
   regardless and eats the front of your
   next command. Send the frame. */
struct { uint8_t op, cmd; uint16_t len; } f;

f.op = 0xE2; f.cmd = 0xCD; f.len = 64;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
if (!fuji_get_error())
    fuji_get_response(buf, 64);

```

12.9 Hashing

HASH_INPUT

\$C8

FRAME	E2 C8 hi lo + len
AUX	<i>word</i> : how many bytes follow, high byte first. Zero is refused
PAYLOAD	that many bytes
REPLY	none
FUJINET-LIB	fuji_hash_input(s, len)

Adds to the data to be hashed. Call it as often as you like to hash something larger than your buffers — a whole disk, if you have the patience.

6809 ASSEMBLER

```

        ldd     #LEN
        std     HIFRM+2
        ldx     #HIFRM
        ldy     #4
        jsr     FNWAIT
        jsr     [DWWRTV]
        ldx     #DATA
        ldy     #LEN
        jsr     [DWWRTV]
        jsr     FNGERR

HIFRM  fcb     OPFUJI,FCHASHI
        fdb     0

```

CMOC C

```
fuji_hash_input(buf, len); /* repeatable */
```

HASH_COMPUTE

\$C7

FRAME E2 C7 algo
AUX *byte*: 0 MD5 · 1 SHA-1 · 2 SHA-256 · 3 SHA-512
PAYLOAD none
REPLY none
FUJINET-LIB fuji_hash_compute(type)

Hashes what you fed it and clears the input. The algorithm you name here is remembered, and HASH_LENGTH and HASH_OUTPUT use it.

6809 ASSEMBLER

```
lda    #FCHASHC
ldb    #2          ; SHA-256
jsr    FNCMD1
bne    FAILED
```

CMOC C

```
fuji_hash_compute(2);    /* SHA-256 */
```

HASH_COMPUTE_NO_CLEAR

\$C3

FRAME E2 C3 algo
AUX *byte*: the algorithm, as above
PAYLOAD none
REPLY none
FUJINET-LIB fuji_hash_compute_no_clear(type)

The same, but keeps the input. Hash a growing message at several points without feeding it again — and the reason it is a separate opcode rather than a flag is that the handler decides by looking at which command byte arrived.

6809 ASSEMBLER

```
lda    #FCHASHN
ldb    #2
jsr    FNCMD1
```

CMOC C

```
fuji_hash_compute_no_clear(2);
```

HASH_LENGTH

\$C6

FRAME E2 C6 mode
AUX *byte*: 1 for hexadecimal text, anything else for raw bytes
PAYLOAD none
REPLY one byte: how long the output will be
FUJINET-LIB **none** — fuji_hash_length() returns false without sending anything

How many bytes HASH_OUTPUT will give you: 16 for MD5 raw and 32 as hex, 20 and 40 for SHA-1, 32 and 64 for SHA-256, 64 and 128 for SHA-512. You can compute these yourself, which is presumably why the library's version is a stub that returns false without sending anything. The three-byte frame above works.

6809 ASSEMBLER

```
lda    #FCHASHL
ldb    #1                ; hex text
jsr    FNCMD1
bne    FAILED
ldx    #HLEN
ldy    #1
jsr    FNGRSP
```

CMOC C

```
struct { uint8_t op, cmd, mode; } f;
uint8_t len;

f.op = 0xE2; f.cmd = 0xC6; f.mode = 1;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
if (!fuji_get_error())
    fuji_get_response(&len, 1);
```

HASH_OUTPUT

\$C5

FRAME	E2 C5 mode
AUX	<i>byte</i> : 1 for hexadecimal text, anything else for raw bytes
PAYLOAD	none
REPLY	the hash, as many bytes as HASH_LENGTH said
FUJINET-LIB	fuji_hash_output(mode, s, len)

The result. Ask for hexadecimal unless you are going to compare it rather than print it: the Color Computer's screen has no way to show a raw \$00.

6809 ASSEMBLER

```
lda    #FCHASHO
ldb    #1                ; hex text
jsr    FNCMD1
bne    FAILED
ldx    #HASH
ldy    #64               ; SHA-256 as hex
jsr    FNGRSP
```

CMOC C

```
char hash[65];

fuji_hash_output(1, hash, 64);
hash[64] = '\\0';
printf("%s\\n", hash);
```

HASH_CLEAR

\$C2

FRAME	E2 C2
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_hash_clear()

Throws away the accumulated input. Start again.

6809 ASSEMBLER

```
lda    #FCHASHZ
jsr    FNCMD0
```

CMOC C

```
fuji_hash_clear();
```

12.10 QR codes

QRCODE_INPUT

\$BC

FRAME	E2 BC hi lo + len
AUX	<i>word</i> : how many bytes follow, high byte first
PAYLOAD	that many bytes — the text to encode
REPLY	none
FUJINET-LIB	fuji_qrcode_input(s, len)

The text the code will carry. Appends, like the other input commands.

6809 ASSEMBLER

```
ldd    #URLLEN
std    QIFRM+2
ldx    #QIFRM
ldy    #4
jsr    FNWAIT
jsr    [DWWRTV]
ldx    #URL
ldy    #URLLEN
jsr    [DWWRTV]
jsr    FNGERR

QIFRM  fcb    OPFUJI,FCQRINP
      fdb    0
```

CMOC C

```
fuji_qrcode_input(url, strlen(url));
```

QRCODE_ENCODE

\$BD

FRAME	E2 BD ver ecc short
AUX	<i>byte</i> : version, masked to 0–127; 0 chooses one. <i>byte</i> : error correction, masked to 0–3 — low, medium, quartile, high. <i>byte</i> : non-zero to shorten the text through the adapter's own URL shortener first
PAYLOAD	none
REPLY	none
FUJINET-LIB	fuji_qrcode_encode(version, ecc, shorten)

Renders the matrix. The output mode starts as binary; asking QRCODE_LENGTH for a different one re-renders from the same matrix, so you do not have to encode twice to get two formats.

6809 ASSEMBLER

```
clr    QEFRM+2      ; pick a version
lda    #1           ; medium ecc
sta    QEFRM+3
clr    QEFRM+4      ; do not shorten
ldx    #QEFRM
ldy    #5
jsr    FNXACT

QEFRM  fcb    OPFUJI,FCQRENC,0,0,0
```

CMOC C

```
fuji_qrcode_encode(0, 1, false);
```

QRCODE_LENGTH

\$BE

FRAME E2 BE mode
AUX *byte*: output mode — 0 binary · 1 ANSI · 2 bitmap · 3 SVG · 4 ATASCII · 5 PETSCII
PAYLOAD none
REPLY 4 bytes, high byte first
FUJINET-LIB fuji_qrcode_length(mode, &len)

How big the rendered code is in the mode you asked for — and asking changes the mode. Binary gives you one byte per module, which is what you want if you are going to paint it into a PMODE screen yourself.

6809 ASSEMBLER

```
lda    #FCQRLen
ldb    #2           ; bitmap
jsr    FNCMD1
bne    FAILED
ldx    #QLEN
ldy    #4
jsr    FNGRSP
```

CMOC C

```
unsigned long len;

fuji_qrcode_length(2, &len);  /* bitmap */
```

QRCODE_OUTPUT

\$BF

FRAME E2 BF hi lo
AUX *word*: how many bytes to take, high byte first
PAYLOAD none
REPLY that many bytes
FUJINET-LIB fuji_qrcode_output(s, len) — **sends no aux word**

Takes the rendered code. The same aux-word problem as the base64 output commands: the library sends no length, the firmware reads two bytes anyway, and your next command loses its front. Build the frame.

6809 ASSEMBLER

```
ldd    QLEN+2
std    QOFRM+2
ldx    #QOFRM
ldy    #4
jsr    FNXACT
bne    FAILED
ldx    #QRBUF
ldy    QLEN+2
jsr    FNGRSP
```

```
QOFRM  fcb    OPFUJI,FCQROUT
       fdb    0
```

CMOC C

```
struct { uint8_t op, cmd; uint16_t len; } f;

f.op = 0xE2; f.cmd = 0xBF; f.len = qlen;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
if (!fuji_get_error())
    fuji_get_response(qrbuf, qlen);
```

12.11 Odds and ends

RANDOM_NUMBER

\$D3

```
FRAME      E2 D3
AUX         none
PAYLOAD     none
REPLY       4 bytes — low byte first
FUJINET-LIB none
```

Four bytes of entropy from the ESP32's hardware generator, which is a better source than anything you can synthesise on a machine with no clock. **This reply is little-endian** — the firmware sends a raw host integer. If you only want a die roll, take one byte and the question does not arise.

6809 ASSEMBLER

```
lda    #FCRANDM
jsr    FNCMD0
bne    FAILED
ldx    #RND
ldy    #4
jsr    FNGRSP
*      little-endian: RND+0 is the low byte
```

CMOC C

```
struct { uint8_t op, cmd; } f;
uint8_t r[4];

f.op = 0xE2; f.cmd = 0xD3;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
if (!fuji_get_error())
    fuji_get_response(r, 4);
/* r[0] is the low byte */
```

GENERATE_GUID

\$BB

FRAME	E2 BB
AUX	none
PAYLOAD	none
REPLY	37 bytes: a UUIDv4 as text, with its terminator
FUJINET-LIB	fuji_generate_guid(buf)

A fresh UUID, in the usual dashed text form. Thirty-six characters and a NUL, and it takes four lines of 6809 instead of four hundred.

6809 ASSEMBLER

```
lda    #FCGUID
jsr    FNCMD0
bne    FAILED
ldx    #GUID
ldy    #SZGUID      ; 37
jsr    FNGRSP
ldx    #GUID
jsr    PUTS
```

CMOC C

```
char guid[37];

if (fuji_generate_guid(guid))
    printf("%s\n", guid);
```

STATUS

\$53 'S'

FRAME	E2 53
AUX	none
PAYLOAD	none
REPLY	4 bytes, all zero
FUJINET-LIB	fuji_status(&s), which sends nothing and returns true

Reserved. It is dispatched, it answers, and the four bytes it sends are always zero — the handler is a stub on every bus. It is documented here because it is dispatched, and a reference that lists only the commands that do something is a reference you cannot trust.

6809 ASSEMBLER

```
lda    #FCSTAT
jsr    FNCMD0
bne    FAILED
ldx    #ST
ldy    #4
jsr    FNGRSP
```

CMOC C

```
/* it answers, but it has nothing to say */
```

12.12 Answered with silence on this platform

These command bytes exist in the firmware's command table and are **not** dispatched on the DriveWire bus. None of them needs an example, and every one of them is a trap, for a reason worth setting out plainly.

A command that reaches no handler does nothing at all: the adapter sends no reply, raises no error, and — because the Color Computer's dispatcher clears the error code to *success* before it looks for a handler — a SEND_ERROR afterwards answers \$01. The command that did nothing reports that it worked.

Worse, if the command had aux bytes or a payload, the adapter never read them. They are still in its receive buffer, and they will be read as the front of whatever you send next. So an unrecognised command with a payload does not merely fail quietly; it desynchronises the bus for the rest of the session.

The list, so that you can tell a command that is not implemented from one you spelled wrong:

\$F0	ENABLE_UDPSTREAM.....	the Atari's MIDI-over-UDP mode
\$EB	SET_BAUDRATE.....	SIO bus tuning
\$E3	SET_HSIO_INDEX.....	high-speed SIO tuning
\$DF	SET_SIO_EXTERNAL_CLOCK.....	SIO clocking
\$D5	ENABLE_DEVICE.....	device rotation, a no-op stub on this bus
\$D4	DISABLE_DEVICE.....	the same
\$D2	GET_TIME.....	use device \$E5 or the \$23 opcode — Chapter 14
\$D1	DEVICE_ENABLE_STATUS.....	device rotation again
\$C1	GET_HEAP.....	a diagnostic, on other buses
\$A0-\$A9	GET_DEVICE1..10_FULLPATH.....	use \$DA with the slot in its aux byte
\$90	UPDATE_FIRMWARE.....	over the web interface, not the wire
\$3F	HSIO_INDEX.....	SIO again
\$06 / \$15	ACK / NAK.....	SIO frame markers; this bus has no frame markers

fujinet-lib ships Color Computer stubs for several of these — `fuji_enable_device()`, `fuji_disable_device()`, `fuji_get_device_enabled_status()`, `fuji_get_hsio_index()`, `fuji_set_hsio_index()`, `fuji_enable_udpstream()`, `fuji_status()` — which return true without sending anything. That is the right behaviour for a portable library and a confusing one if you have not read this page: your program will believe it enabled a device. It did not; there was nothing to enable.

13. COMMAND REFERENCE: THE NETWORK DEVICE

Every command device \$E3 answers, verified against the unified network device in `NDevice.cpp` and cross-checked against the `fujinet-lib` Color Computer port.

Conventions as in Chapter 12, with one addition that applies to every card here: **the frame always carries a unit byte and exactly two aux bytes**, whether the command uses them or not. A command with no parameters is still five bytes. The frame decoder consumes that pair before it will hand a handler any payload, and consumes it on the way out if nothing else did.

Unlike the control device, an unrecognised command here is a clean failure: the dispatcher calls the error path, so `SEND_ERROR` answers \$90 and the bus stays in step.

13.1 The lifecycle five

OPEN

\$4F '0'

FRAME	E3 unit 4F mode trans + 256
AUX	<i>byte</i> : the open mode — \$04 read, \$06 directory, \$07 directory alternate, \$08 write, \$09 append, \$0C read and write; for HTTP the same byte is the method. <i>byte</i> : translation — \$00 none, \$01 CR, \$02 LF, \$03 CR LF, \$04 PETSCII
PAYLOAD	256 bytes: the devicespec, NUL-padded
REPLY	none
FUJINET-LIB	<code>network_open(spec, mode, trans)</code>

Instantiates the protocol named by the scheme and connects it. If a channel is already open on this unit it is closed first, so a second OPEN is a reconnect rather than an error.

The devicespec field is a fixed 256 bytes whatever the string's length. Pad it. A short field leaves the adapter reading on into your next command and there is no recovery.

6809 ASSEMBLER

```
ldx    #SPEC
lda     #OMRW
ldb     #TRNONE
jsr     NTOPEN
bne     BADOPEN
*       NTOPEN parses the unit out of SPEC,
*       clears the 256-byte field and pads
```

CMOC C

```
if (network_open("N:TCP://host:6502/",
                OPEN_MODE_RW,
                OPEN_TRANS_NONE) != FN_ERR_OK)
    fail(fn_device_error);
```

CLOSE

\$43 'C'

FRAME E3 unit 43 00 00
AUX two bytes, ignored — send zeros
PAYLOAD none
REPLY none
FUJINET-LIB network_close(spec)

Closes the channel and destroys the protocol. Always close: a channel left open holds a socket on the adapter, and the adapter has a finite number of them.

6809 ASSEMBLER

```
jsr     NTCLOS
*       or, for a unit you are tracking:
ldb     UNIT
stb     CFRM+1
ldx     #CFRM
ldy     #5
jsr     NTXACT
CFRM     fcb    OPNET,0,NCCLOSE,0,0
```

CMOC C

```
network_close(spec);
```

STATUS

\$53 'S'

FRAME E3 unit 53 00 00
AUX two bytes, ignored
PAYLOAD none
REPLY 4 bytes: bytes waiting (word, **high byte first**), connected, the channel's error byte
FUJINET-LIB network_status(spec, &bw, &conn, &err)

The whole state of a channel in four bytes, and the command you call most. Ask it before every read and read no more than it said. Zero waiting is not an error — it is the usual state of a quiet socket. The end of a resource is an error byte of 136, or zero waiting with the connected byte clear. With no protocol open at all this answers 207, not connected.

6809 ASSEMBLER

```
ldx    #STBUF
jsr    NTSTAT
bne    BADIO
ldd    STBUF          ; big-endian: LDD
lda    STBUF+2        ; connected
lda    STBUF+3        ; error
```

CMOC C

```
uint16_t bw;
uint8_t conn, err;

network_status(spec, &bw, &conn, &err);
if (err == 136) /* end of file */
    done();
```

READ

\$52 'R'

FRAME E3 unit 52 hi lo
AUX *word*: how many bytes to read, high byte first
PAYLOAD none
REPLY that many bytes, through SEND_RESPONSE
FUJINET-LIB network_read(spec, buf, len) — blocking, or network_read_nb()

Takes bytes out of the channel. Ask for no more than STATUS said was waiting: the adapter pads the reply with zeros up to the length you named, and the zeros are indistinguishable from data.

The length goes out twice — once in this command's aux word and again in the SEND_RESPONSE that collects the bytes. The library's blocking network_read() loops on STATUS until it has everything you asked for or hits end of file, so it is the wrong call for an interactive program; ask STATUS yourself and read what is there.

6809 ASSEMBLER

```
ldx    #RXBUF
ldy    #64
jsr    NTREAD
bne    BADIO
```

CMOC C

```
int16_t n = network_read(spec, buf, want);

if (n < 0)
    fail(fn_device_error); /* negative */
```

WRITE

\$57 'W'

FRAME E3 unit 57 hi lo + len
AUX *word*: how many bytes follow, high byte first
PAYLOAD that many bytes
REPLY none
FUJINET-LIB network_write(spec, buf, len)

Puts bytes into the channel. The header goes out first and the payload follows as a separate write — the adapter is already counting, so there is no need to build one buffer out of two.

6809 ASSEMBLER

```
ldx    #TXBUF
ldy    #TXLEN
jsr    NTWRIT
bne    BADIO
*      NTWRIT sends the header, then the
*      payload as a second DriveWire write
```

CMOC C

```
network_write(spec, (uint8_t *) line,
               strlen(line));
```

13.2 Line discipline and position

TRANSLATION

\$54 'T'

```
FRAME      E3 unit 54 00 mode
AUX         byte, ignored.
            byte: the translation mode — aux2, not aux1
PAYLOAD     none
REPLY       none
FUJINET-LIB none
```

Overrides the translation mode set at OPEN, for the life of the channel. **The mode is read from aux2.** A value of \$FF is the escape hatch that suppresses the override entirely.

6809 ASSEMBLER

```
ldb     NTUNI
stb     TRFRM+1
ldx     #TRFRM
ldy     #5
jsr     NTXACT

TRFRM    fcb    OPNET,0,NCTrans,0,TRCR
```

CMOC C

```
struct { uint8_t op,unit,cmd,a1,a2; } f;

f.op = 0xE3; f.unit = unit;
f.cmd = 'T'; f.a1 = 0;
f.a2 = 1; /* CR */
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
e = network_get_error(unit);
```

SET_EOL

\$4C 'L'

```
FRAME      E3 unit 4C hi lo + len
AUX         word: how many bytes of line ending follow, high byte first
PAYLOAD     that many bytes
REPLY       none
FUJINET-LIB none
```

Sets the byte or bytes the protocol should treat as a line ending on this channel. A zero length clears the override and puts the platform's own ending back — on the Color Computer that is a single carriage return, \$0D, which is usually what you wanted anyway.

6809 ASSEMBLER

```
    ldd    #1
    std    ELFRM+3
    ldb    NTUNI
    stb    ELFRM+1
    ldx    #ELFRM
    ldy    #5
    jsr    FNWAIT
    jsr    [DWWRITEV]
    ldx    #ELBYTE
    ldy    #1
    jsr    [DWWRITEV]
    ldb    NTUNI
    jsr    NTGERR

ELFRM   fcb    OPNET,0,NCSETEL,0,0
ELBYTE  fcb    13
```

CMOC C

```
struct { uint8_t op,unit,cmd;
        uint16_t len; } f;
uint8_t eol = 13;

f.op = 0xE3; f.unit = unit;
f.cmd = 'L'; f.len = 1;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
dwwrite(&eol, 1);
e = network_get_error(unit);
```

SET_INT_RATE

\$5A 'Z'

FRAME E3 unit 5A rate 00
AUX *byte*: milliseconds between checks
PAYLOAD none
REPLY none
FUJINET-LIB none

How often the adapter looks to see whether this channel has anything, for the purpose of raising the serial line. The default is 100 milliseconds on hardware and 20 on FujiNet-PC. It has no effect at all over DriveWire-over-IP, where there is no line to raise.

6809 ASSEMBLER

```
    ldb    NTUNI
    stb    IRFRM+1
    ldx    #IRFRM
    ldy    #5
    jsr    NTXACT

IRFRM   fcb    OPNET,0,NCINTRT,50,0
```

CMOC C

```
f.op = 0xE3; f.unit = unit;
f.cmd = 'Z'; f.a1 = 50; f.a2 = 0;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
```

SEEK

\$25 '%'

FRAME E3 unit 25 + 4
AUX *a four-byte offset*, high byte first — **not the usual two**
PAYLOAD none
REPLY none
FUJINET-LIB none

Moves the read position within whatever the channel's parser is holding.

This is the exception to the two-aux-byte rule. The handler asks for a 32-bit parameter, so the decoder reads four bytes here and not two, and the frame is seven bytes rather than five. Send five and the adapter takes the front of your next command as the rest of the offset.

6809 ASSEMBLER

```
*      the one command on this device whose
*      parameter is four bytes, not two
      ldb     NTUNI
      stb     SKFRM+1
      ldd     #0
      std     SKFRM+3
      ldd     #1024
      std     SKFRM+5
      ldx     #SKFRM
      ldy     #7
      jsr     NTXACT

SKFRM   fcb     OPNET,0,NCSEEK
        fqb     0
```

CMOC C

```
struct { uint8_t op,unit,cmd;
        uint32_t off; } f;

f.op = 0xE3; f.unit = unit;
f.cmd = '%'; f.off = 1024;      /* big-endian */
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
e = network_get_error(unit);
```

TELL

\$26 '&'

```
FRAME      E3 unit 26 00 00
AUX         two bytes, ignored
PAYLOAD     none
REPLY       4 bytes — low byte first
FUJINET-LIB none
```

Where the read position is. **The reply is little-endian** — the firmware declares it that way explicitly — while SEEK takes its offset big-endian. The two do not match and this card is the only warning.

With no parser on the channel the offset is zero.

6809 ASSEMBLER

```
      ldb     NTUNI
      stb     TLFRM+1
      ldx     #TLFRM
      ldy     #5
      jsr     NTXACT
      bne     BADIO
      ldb     NTUNI
      ldx     #POS
      ldy     #4
      jsr     NTGRSP
*      POS+0 is the low byte

TLFRM   fcb     OPNET,0,NCTELL,0,0
```

CMOC C

```
uint8_t pos[4];
/* ... send the frame ... */
network_get_response(unit, pos, 4);
/* pos[0] is the low byte */
```

13.3 Parsers and queries

SET_PARSER

\$FC

FRAME E3 unit FC 00 mode
AUX *byte, ignored.*
byte: 0 none · 1 JSON · 2 HTML · 3 XML — aux2, not aux1
PAYLOAD none
REPLY none
FUJINET-LIB network_json_parse() sends this — **in the wrong aux byte**

Puts a parser on the channel. HTML is queried with a CSS selector and XML with a subset of XPath; JSON is the one you will use.

The mode is read from aux2. fujinet-lib's network_json_parse() puts it in aux1 and leaves aux2 zero, which selects the null parser — so on a current adapter that call appears to succeed and then every query comes back empty. Send the five bytes above instead. Chapter 18 does.

6809 ASSEMBLER

```
lda NTUNI
sta SPFRM+1
ldx #SPFRM
ldy #5
ldb NTUNI
jsr NTXACT

SPFRM fcb OPNET,0,NCSPARS,0,PMJSON
```

CMOC C

```
struct { uint8_t op,unit,cmd,a1,a2; } f;

f.op = 0xE3; f.unit = unit;
f.cmd = 0xFC;
f.a1 = 0;
f.a2 = 1; /* JSON */
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
e = network_get_error(unit);
```

PARSE

\$50 'P'

FRAME E3 unit 50 00 00
AUX two bytes, ignored
PAYLOAD none
REPLY none
FUJINET-LIB the second half of network_json_parse()

Reads the channel to the end and hands the lot to the parser. After this the document lives on the adapter and the channel's contents are whatever the last query selected. It is the moment a 64-kilobyte machine stops caring how big the response was.

Errors here are real: 213 means the document would not parse.

6809 ASSEMBLER

```
lda NTUNI
sta PAFRM+1
ldx #PAFRM
ldy #5
ldb NTUNI
jsr NTXACT

PAFRM fcb OPNET,0,NCPARSE,0,0
```

CMOC C

```
f.cmd = 'P'; f.a1 = f.a2 = 0;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
e = network_get_error(unit);
```

QUERY

\$51 'Q'

FRAME	E3 unit 51 00 00 + 256
AUX	two bytes, ignored
PAYLOAD	256 bytes: the query, NUL-padded
REPLY	none directly — the result becomes the channel's contents, so ask STATUS and then READ
FUJINET-LIB	network_json_query(spec, query, buf)

Selects part of the parsed document. A JSON query is a path like `/current/temperature_2m`; the result replaces the channel's contents, so the sequence is query, status, read.

Note the order of checks in the firmware: the 256-byte payload is drained *before* it complains that there is no parser, precisely so that a query sent too early fails cleanly instead of wedging the bus.

6809 ASSEMBLER

```
        ldx    #QYSTR          ; cleared, then
        ...      ; the query copied in
        lda    NTUNI
        sta    QYFRM+1
        ldx    #QYFRM
        ldy    #QYLEN          ; 5 + 256
        ldb    NTUNI
        jsr    NTXACT
*       then NTSTAT and NTREAD

QYFRM   fcb    OPNET,0,NCQUERY,0,0
QYSTR   rmb    256
```

CMOC C

```
char answer[64];
int16_t n = network_json_query(spec,
                                "/current/temperature_2m",
                                answer);

if (n > 0)
    printf("%s\n", answer);
```

SET_PARAMETER

\$FB

FRAME	E3 unit FB type value
AUX	<i>byte</i> : 0 sets the query flags, 1 sets the parser's line ending. <i>byte</i> : the value
PAYLOAD	none
REPLY	none
FUJINET-LIB	none

Tunes the parser. With a type of 0 the value is a flags byte: the low three bits ask for character remapping (\$01), international ATASCII remapping (\$02) and the stripping of SGML tags (\$04, JSON only), and bits 4 and 5 choose the output form — \$00 verbatim or \$10 ASCII. \$20 and \$30 are reserved and are rejected.

With a type of 1 the value is a single byte to use as the parser's line ending. Any other type is an error. This command needs a parser already set; without one it fails.

6809 ASSEMBLER

```
lda    NTUNI
sta    SQFRM+1
ldx    #SQFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
```

```
SQFRM  fcb    OPNET,0,NCSPARM,0,$10
```

CMOC C

```
f.cmd = 0xFB;
f.a1 = 0;           /* query flags */
f.a2 = 0x10;        /* ASCII output */
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
```

SET_HTTP_MODE

\$4D 'M'

FRAME E3 unit 4D 00 mode

AUX *byte*, ignored.

byte: 0 body · 1 collect response headers · 2 read the collected headers · 3 set a request header · 4 set the POST data — **aux2**

PAYLOAD none

REPLY none

FUJINET-LIB network_http_set_channel_mode(spec, mode)

Changes what reads and writes on an HTTP channel mean. In mode 3 a write sets a request header; in mode 1 the adapter collects the response headers and in mode 2 you read them back; in mode 4 a write becomes the POST body. Mode 0 puts you back on the body.

This one is only valid on an HTTP: or HTTPS: channel and errors on anything else. The library gets the aux byte right here.

6809 ASSEMBLER

```
lda    NTUNI
sta    HMFRM+1
ldx    #HMFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
```

```
HMFRM  fcb    OPNET,0,NCHTTPM,0,HMSETH
```

CMOC C

```
network_http_set_channel_mode(spec, 3);
network_write(spec,
    (uint8_t *) "Accept: application/json", 24);
network_http_set_channel_mode(spec, 0);
```

13.4 Filesystem operations

These six work on a *one-shot* channel: the adapter parses the devicespec you hand it, does the operation, and throws the protocol away. You do not open first and you do not close afterwards. They need a filesystem behind them — TNFS:, SMB:, FTP: — and error on a socket.

All six share a frame: the operation's own command byte, a mode byte in aux1 that the filesystem layer interprets, an ignored aux2, and a 256-byte devicespec. fujinet-lib reaches them through network_ioctl(), which is why that function takes a devicespec as its last argument.

DELETE

\$21 '!'

FRAME E3 unit 21 mode 00 + 256
AUX *byte*: mode, passed to the filesystem.
 byte, ignored
PAYLOAD 256 bytes: the devicespec, NUL-padded
REPLY none
FUJINET-LIB network_fs_delete(spec)

Removes a file.

6809 ASSEMBLER

```
ldx    #SPEC
lda    #NCDELET
jsr    FSOP          ; see Appendix C
```

CMOC C

```
network_fs_delete("N:TNFS://server/OLD.DSK");
```

RENAME

\$20

FRAME E3 unit 20 mode 00 + 256
AUX *byte*: mode. *byte*, ignored
PAYLOAD 256 bytes: path, newname, NUL-padded
REPLY none
FUJINET-LIB network_fs_rename(spec)

Renames a file. The old and new names go in one devicespec separated by a comma — the whole thing is still one 256-byte field.

6809 ASSEMBLER

```
ldx    #SPEC
lda    #NCRENAM
jsr    FSOP
```

CMOC C

```
network_fs_rename("N:TNFS://s/A.DSK,B.DSK");
```

LOCK / UNLOCK

\$23 '#' / \$24 '\\$'

FRAME E3 unit 23 mode 00 + 256
AUX *byte*: mode. *byte*, ignored
PAYLOAD 256 bytes: the devicespec
REPLY none
FUJINET-LIB network_fs_lock(spec) · network_fs_unlock(spec)

Sets and clears a file's read-only attribute, where the filesystem has one. On TNFS this is the write permission bit.

6809 ASSEMBLER

```
ldx    #SPEC
lda     #NCLOCK
jsr     FSOP
```

CMOC C

```
network_fs_lock(spec);
/* ... */
network_fs_unlock(spec);
```

MKDIR / RMDIR

\$2A '*' / \$2B '+'

FRAME E3 unit 2A mode 00 + 256
AUX *byte: mode, ignored*
PAYLOAD 256 bytes: the devicespec
REPLY none
FUJINET-LIB network_fs_mkdir(spec) · network_fs_rmdir(spec)

Makes and removes a directory. RMDIR wants the directory to be empty; the filesystem decides, not the adapter.

6809 ASSEMBLER

```
ldx     #SPEC
lda      #NCMKDIR
jsr      FSOP
```

CMOC C

```
network_fs_mkdir("N:TNFS://server/SAVES/");
```

CHDIR

\$2C ', '

FRAME E3 unit 2C 00 00 + 256
AUX two bytes, ignored
PAYLOAD 256 bytes: the new prefix, NUL-padded
REPLY none
FUJINET-LIB network_fs_cd(spec)

Sets this unit's prefix — the directory every later devicespec is relative to. The conventions are the shell's: .. or < goes up one, / or > truncates back to the host, a leading slash replaces the path but keeps the host, a bare scheme: resets everything, and anything else is appended. An empty Nn: clears it.

6809 ASSEMBLER

```
ldx     #SPEC
lda      #NCCHDIR
jsr      FSOP
```

CMOC C

```
network_fs_cd("N:TNFS://server/COCO/");
```

GETCWD

\$30 '0'

FRAME	E3 unit 30 00 00
AUX	two bytes, ignored
PAYLOAD	none
REPLY	the prefix — ask for 256 bytes and it is zero-padded to fit
FUJINET-LIB	none

Reads the prefix back. The adapter sends the string's own length, but SEND_RESPONSE on this device pads the reply out to whatever you asked for — so ask for 256, and the tail is zeros.

6809 ASSEMBLER

```
lda    NTUNI
sta    CWFRM+1
ldx    #CWFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
bne    BADIO
ldb    NTUNI
ldx    #PFX
ldy    #256
jsr    NTGRSP
```

```
CWFRM  fcb    OPNET,0,NCGETCW,0,0
```

CMOC C

```
char prefix[257];
/* ... send the five-byte frame ... */
network_get_response(unit,
                      (uint8_t *) prefix, 256);
prefix[256] = '\0';
```

13.5 Sockets and credentials

ACCEPT

\$41 'A'

FRAME	E3 unit 41 00 00
AUX	two bytes, ignored
PAYLOAD	none
REPLY	none
FUJINET-LIB	none

Accepts a waiting inbound connection on a listening TCP: channel — one opened with a port and no host, as in N:TCP://:6502/. Errors on any other kind of channel.

There is no way to ask whether a connection is waiting other than to try: STATUS reports 209, no connection waiting, when there is none.

6809 ASSEMBLER

```
lda    NTUNI
sta    ACFRM+1
ldx    #ACFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
```

```
ACFRM  fcb    OPNET,0,NCACPT,0,0
```

CMOC C

```
f.cmd = 'A'; f.a1 = f.a2 = 0;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
e = network_get_error(unit);
```

CLOSE_CLIENT

\$63 'c'

```
FRAME      E3 unit 63 00 00
AUX        two bytes, ignored
PAYLOAD     none
REPLY       none
FUJINET-LIB none
```

Hangs up on the accepted client without closing the listening socket, so the next ACCEPT can take another. CLOSE would throw away the listener as well. TCP only.

6809 ASSEMBLER

```
lda    NTUNI
sta    CCFRM+1
ldx    #CCFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
```

```
CCFRM  fcb    OPNET,0,NCCLCLI,0,0
```

CMOC C

```
f.cmd = 'c'; f.a1 = f.a2 = 0;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
```

GET_REMOTE

\$72 'r'

```
FRAME      E3 unit 72 00 00
AUX        two bytes, ignored
PAYLOAD     none
REPLY       256 bytes: the remote address as text
FUJINET-LIB none
```

Who the last UDP datagram came from.

This one is compiled out of the ESP32 firmware. It is guarded `#ifndef ESP_PLATFORM`, so it works on FujiNet-PC and always errors on a hardware adapter. UDP only in either case.

6809 ASSEMBLER

```
lda    NTUNI
sta    GRFRM+1
ldx    #GRFRM
ldy    #5
ldb    NTUNI
jsr    NTXACT
bne    BADIO
ldb    NTUNI
ldx    #REMOTE
ldy    #256
jsr    NTGRSP
```

```
GRFRM  fcb    OPNET,0,NCGTREM,0,0
```

CMOC C

```
char remote[257];
/* ... send the frame ... */
network_get_response(unit,
                     (uint8_t *) remote, 256);
```

SET_DESTINATION

\$44 'D'

FRAME E3 unit 44 hi lo + len
AUX *word*: how many bytes follow, high byte first
PAYLOAD that many bytes: the destination, as text
REPLY none
FUJINET-LIB none

Points a UDP channel at somewhere to send. UDP only.

6809 ASSEMBLER

```
ldd    #DESTLEN
std     SDFRM+3
lda     NTUNI
sta     SDFRM+1
ldx     #SDFRM
ldy     #5
jsr     FNWAIT
jsr     [DWWRITEV]
ldx     #DEST
ldy     #DESTLEN
jsr     [DWWRITEV]
ldb     NTUNI
jsr     NTGERR
```

```
SDFRM  fcb    OPNET,0,NCSETDE,0,0
```

CMOC C

```
struct { uint8_t op,unit,cmd;
         uint16_t len; } f;
char dest[] = "192.168.1.9:5000";

f.op = 0xE3; f.unit = unit;
f.cmd = 'D'; f.len = strlen(dest);
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
dwwrite((uint8_t *) dest, f.len);
```

USERNAME / PASSWORD

\$FD / \$FE

FRAME	E3 unit FD 00 00 + 256
AUX	two bytes, ignored
PAYLOAD	256 bytes, NUL-padded
REPLY	none
FUJINET-LIB	none

Credentials for the next protocol that wants them — FTP:, SMB:, an authenticated HTTP:. Set them before OPEN; they live on the unit, not on the channel, so they survive a close.

They can also go inline in the devicespec as `scheme://user:pass@host/`, which is shorter and puts your password in a string constant. These two exist so that it does not have to be.

6809 ASSEMBLER

```
        lda     NTUNI
        sta     UNFRM+1
        ldx     #UNFRM
        ldy     #261
        ldb     NTUNI
        jsr     NTXACT

UNFRM   fcb     OPNET,0,NCUSER,0,0
UNSTR   rmb     256
```

CMOC C

```
/* the same frame with cmd 0xFE for the
   password. Both are 261 bytes. */
```

13.6 Answered with an error

These bytes are in the command table and are not in the network device's dispatch table. Unlike the control device, this one fails properly: the dispatcher takes the error path, `SEND_ERROR` answers \$90, and the bus stays in step. The two aux bytes are still consumed, so a five-byte frame leaves nothing behind.

\$FF	GET_DSTATS_VALUE.....	never implemented on any bus
\$FA	SET_UNIT.....	the unit is in every frame here
\$E3	SET_HSIO_INDEX.....	SIO bus tuning
\$81	QUERY_ALT.....	the Atari's alternate query entry
\$80	PARSE_ALT.....	likewise
\$45	GET_ERROR 'E'.....	use the bus's own \$02
\$3F	HSIO_INDEX '?'.....	SIO again

The third of those is worth a second look. Other FujiNet platforms fetch a channel's error with `NET_GET_ERROR`, command \$45. On this bus that command does not exist: the error comes from the bus meta-command \$02, addressed to the unit, exactly as Chapter 4 described. If you are porting code from an Atari or an Apple II, that is the substitution to make.

14. COMMAND REFERENCE: THE OTHER DEVICES

14.1 The clock

Device \$E5, and it breaks the rule that every other device follows. The bus runs the clock's handler *first* and then rewrites the command to SEND_RESPONSE, so the answer comes straight back: no DEVICE_READY, no SEND_ERROR, no second exchange. Write the frame, read the reply. Most of the read commands take an optional third byte. Send \$01 and the reply uses the alternate timezone set with \$99; send \$00, or nothing at all, and it uses the adapter's own. Strings come back with a terminating NUL.

There is a simpler road still, and it is the one fujinet-lib takes: the bus-level opcode \$23 in Chapter 15 is a single byte out and six bytes back, with no device and no command at all.

GET_TIME (simple)

\$54 'T'

FRAME	E5 54 [alt]
AUX	byte, optional: \$01 to use the alternate timezone
PAYLOAD	none
REPLY	7 bytes: century, year, month (1–12), day, hour, minute, second
FUJINET-LIB	clock_get_time(buf, SIMPLE_BINARY) — which actually uses the \$23 opcode

The most convenient form: a full four-digit year split across two bytes, then the fields in the order you would write them. This opcode is in both the clock's read table and its write table — it is SETTZ on buses where reads and writes have separate namespaces — and the read table is consulted first. On this bus \$54 gets you the time. To set a timezone, use \$99 or \$74.

6809 ASSEMBLER

```
ldx    #CKFRM
ldy    #3
jsr    [DWWRITEV]
ldx    #TIMBUF
ldy    #7
jsr    [DWREADV]
bne    NOCLOCK

CKFRM  fcb    OPCLOCK,$54,0
```

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x54, 0 };
uint8_t t[7];

dwwrite(frame, 3);
if (dwread(t, 7))
    printf("%u%02u-%02u-%02u %02u:%02u:%02u\n",
           t[0], t[1], t[2], t[3],
           t[4], t[5], t[6]);
```

GET_TIME (APETIME)

\$93

FRAME	E5 93 [alt]
AUX	byte, optional: the alternate-timezone flag
PAYLOAD	none
REPLY	6 bytes: day, month (1–12), year (two digits), hour, minute, second
FUJINET-LIB	none

The Atari's APETIME order, day first, and a two-digit year. It is here because it is dispatched and because porting code from an Atari is a thing people do. \$9A is the same reply with the alternate timezone forced on regardless of the flag byte.

6809 ASSEMBLER

```
ldx    #APFRM
ldy    #3
jsr    [DWWRTV]
ldx    #TIMBUF
ldy    #6
jsr    [DWREADV]

APFRM  fcb    OPCLOCK,$93,0
```

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x93, 0 };
uint8_t t[6];

dwwrite(frame, 3);
dwread(t, 6);      /* dd mm yy hh mm ss */
```

GET_TIME (hundredths)

\$4D 'M'

FRAME	E5 4D [alt]
AUX	byte, optional: the alternate-timezone flag
PAYLOAD	none
REPLY	8 bytes: the seven of the simple form, then hundredths (0–99)
FUJINET-LIB	none

The simple form with a hundredths byte on the end. Useful for timing something across a network round trip, where the Color Computer's own sixty-times-a-second interrupt is the least of your errors.

6809 ASSEMBLER

```
ldx    #HUFRM
ldy    #3
jsr    [DWWRTV]
ldx    #TIMBUF
ldy    #8
jsr    [DWREADV]

HUFRM  fcb    OPCLOCK,$4D,0
```

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x4D, 0 };
uint8_t t[8];

dwwrite(frame, 3);
dwread(t, 8);
```

GET_TIME (ProDOS)

\$50 'P'

FRAME E5 50 [alt]
AUX *byte, optional*
PAYLOAD none
REPLY 4 bytes in the ProDOS date and time format
FUJINET-LIB clock_get_time(buf, PRODOS_BINARY) — not implemented on this platform

The Apple II's packed four-byte date. Dispatched here as well, because the clock is shared code across every bus. You will not want it on a Color Computer unless you are writing something that stores dates in that format on purpose.

6809 ASSEMBLER

```
ldx    #PDFRM
ldy    #3
jsr    [DWWRTV]
ldx    #TIMBUF
ldy    #4
jsr    [DWREADV]

PDFRM  fcb    OPCLOCK,$50,0
```

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x50, 0 };
uint8_t t[4];

dwwrite(frame, 3);
dwread(t, 4);
```

GET_TIME (SOS)

\$53 'S'

FRAME E5 53 [alt]
AUX *byte, optional*
PAYLOAD none
REPLY the string YYYYMMDD0HHMMSS000 and a NUL — 19 bytes
FUJINET-LIB none

The Apple III's SOS format, as text. Eighteen digits and a terminator.

6809 ASSEMBLER

```
ldx    #SOFRM
ldy    #3
jsr    [DWWRTV]
ldx    #TIMBUF
ldy    #19
jsr    [DWREADV]

SOFRM  fcb    OPCLOCK,$53,0
```

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x53, 0 };
char t[20];

dwwrite(frame, 3);
dwread((uint8_t *) t, 19);
```

GET_TIME (ISO)

\$49 'I' / \$5A 'Z'

FRAME	E5 49 [alt]
AUX	<i>byte</i> , optional — ignored by \$5A
PAYLOAD	none
REPLY	YYYY-MM-DDTHH:MM:SS+HHMM and a NUL — 25 bytes
FUJINET-LIB	none

ISO 8601, which is the one to use if you are going to put a timestamp in something that leaves the machine. \$49 is local time with the offset filled in; \$5A is the same thing at UTC, with an offset of +0000, and ignores the alternate-timezone flag.

6809 ASSEMBLER

```
ldx    #ISFRM
ldy    #3
jsr    [DWWRITV]
ldx    #TIMBUF
ldy    #25
jsr    [DWREADV]
ldx    #TIMBUF
jsr    PUTS
```

ISFRM fcb OPCLOCK,\$49,0

CMOC C

```
uint8_t frame[3] = { 0xE5, 0x49, 0 };
char t[26];

dwwrite(frame, 3);
if (dwread((uint8_t *) t, 25))
    printf("%s\n", t);
```

GET_TZ

\$47 'G'

FRAME	E5 47
AUX	none — this one takes no flag byte
PAYLOAD	none
REPLY	the adapter's timezone string and a NUL
FUJINET-LIB	clock_get_tz(buf)

Which timezone the adapter is keeping. Ask \$4C first: it answers with one byte, the string's length including its terminator, and then \$47 sends exactly that many. Reading a variable-length reply on a bus with no length field is the one thing that needs two commands, and this is the pair that does it.

6809 ASSEMBLER

```
lda    #$4C          ; ask the length first
sta    TZFRM+1
ldx    #TZFRM
ldy    #2
jsr    [DWWRTV]
ldx    #TZLEN
ldy    #1
jsr    [DWREADV]
lda    #$47
sta    TZFRM+1
ldx    #TZFRM
ldy    #2
jsr    [DWWRTV]
ldx    #TZBUF
ldb    TZLEN
clra
tfr    d,y
jsr    [DWREADV]
```

```
TZFRM  fcb    OPCLOCK,0
```

CMOC C

```
uint8_t f[2] = { 0xE5, 0x4C };
uint8_t len;
char tz[64];

dwwrite(f, 2);  dwread(&len, 1);
f[1] = 0x47;
dwwrite(f, 2);  dwread((uint8_t *) tz, len);
```

SET_TZ

\$74 't' / \$99

FRAME	E5 74 hi lo + len
AUX	<i>word</i> : how many bytes of timezone follow, high byte first
PAYLOAD	that many bytes
REPLY	none
FUJINET-LIB	clock_set_tz(tz) — not implemented on this platform

\$74 sets the system timezone and \$99 sets the alternate one — the one the optional flag byte on every read command selects. The string is a POSIX timezone name such as America/Chicago.

KNOWN BUG

The Color Computer's implementation of this reads the length with an extra byte-swap: the frame decoder has already converted it from big-endian, and the clock handler swaps it again. On the adapter's little-endian processor a length of 3 therefore arrives as 768, and the adapter sits waiting for 768 bytes that will never come. Until that is fixed, set the timezone from the web interface.

6809 ASSEMBLER

```
*      see the caution below before using
      ldx    #STFRM
      ldy    #4
      jsr    [DWWRTIV]
      ldx    #TZSTR
      ldy    #TZLEN
      jsr    [DWWRTIV]

STFRM   fcb    OPCLOCK,$74
        fdb    0
```

CMOC C

```
/* $74 sets the adapter's own timezone;
   $99 sets the alternate one that the
   [alt] flag selects. */
```

14.2 CPM/M

Device \$E4 is a Z80 emulator running RunCPM on the adapter, with its console on the wire. It exists only in the ESP32 firmware — FujiNet-PC compiles it out — and it is the one device in this book that is not on every build. The three bus meta-commands work here as they do everywhere else.

CPM_BOOT

\$42 'B'

```
FRAME      E4 42
AUX         none
PAYLOAD     none
REPLY       none
FUJINET-LIB none
```

Starts the emulator on its own task with 64K of RAM. Give it a moment before you expect a prompt.

6809 ASSEMBLER

```
      ldx    #CBFRM
      ldy    #2
      jsr    [DWWRTIV]

CBFRM   fcb    OPCPM,'B'
```

CMOC C

```
uint8_t f[2] = { 0xE4, 'B' };
dwwrite(f, 2);
```

CPM_READ

\$52 'R'

```
FRAME      E4 52 hi lo
AUX         word: how many bytes to take, high byte first
PAYLOAD     none
REPLY       that many bytes from the console
FUJINET-LIB none
```

Takes bytes from the console output queue. Ask CPM_STATUS first: when the queue is empty this command sends nothing at all and the Color Computer waits for a reply that is not coming.

6809 ASSEMBLER

```
        ldd      #64
        std      CRFRM+2
        ldx      #CRFRM
        ldy      #4
        jsr      [DWWRTIV]
*       then the three-step fetch as usual

CRFRM   fcb      OPCPM,'R'
        fdb      0
```

CMOC C

```
struct { uint8_t op, cmd; uint16_t len; } f;

f.op = 0xE4; f.cmd = 'R'; f.len = 64;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
```

CPM_WRITE

\$57 'W'

FRAME E4 57 hi lo + len
AUX *word*: how many bytes follow
PAYLOAD that many bytes
REPLY none
FUJINET-LIB none

Puts keystrokes into the console input queue. Both queues hold 2048 bytes.

6809 ASSEMBLER

```
        ldd      #LEN
        std      CWFRM+2
        ldx      #CWFRM
        ldy      #4
        jsr      [DWWRTIV]
        ldx      #KEYS
        ldy      #LEN
        jsr      [DWWRTIV]

CWFRM   fcb      OPCPM,'W'
        fdb      0
```

CMOC C

```
f.op = 0xE4; f.cmd = 'W'; f.len = n;
bus_ready();
dwwrite((uint8_t *) &f, sizeof(f));
dwwrite(keys, n);
```

CPM_STATUS

\$53 'S'

FRAME E4 53
AUX none
PAYLOAD none
REPLY 2 bytes: how many are waiting, high byte first
FUJINET-LIB none

How much the console has to say. Poll this and read what it reports, exactly as with the network device.

6809 ASSEMBLER

```
ldx    #CSFRM
ldy    #2
jsr    [DWWRTV]
*      then the three-step fetch

CSFRM   fcb    OPCPM,'S
```

CMOC C

```
uint16_t waiting;
/* ... frame, then ... */
fuji_get_response((uint8_t *) &waiting, 2);
/* already big-endian on a 6809 */
```

14.3 The printer

PRINT

\$50 'P'

FRAME 50 byte
AUX *byte*: the character to print
PAYLOAD none
REPLY none
FUJINET-LIB none

One byte per call, straight into the printer emulation — and note that this is a *bus* opcode, not a device: there is no command byte, no hand-shake and no reply. The adapter can pretend to be anything from a Radio Shack line printer to a PNG file, and which one is a setting in the web interface rather than something you choose from here.

\$46 'F', print-flush, is accepted and ignored.

6809 ASSEMBLER

```
PRCHR   sta    PRFRM+1
        ldx    #PRFRM
        ldy    #2
        jmp    [DWWRTV]

PRFRM   fcb    OPPRINT,0
```

CMOC C

```
void print_char(uint8_t c)
{
    uint8_t f[2] = { 0x50, 0 };

    f[1] = c;
    dwwrite(f, 2);
}
```

14.4 The modem, and why it is not here

FujiNet has a full Hayes-style modem with an AT command set, and it is **not reachable from the Color Computer**. The code is in the firmware and compiles; nothing in the Color Computer build ever constructs it, so there is no device on the bus to address. The Atari build constructs one; this one does not.

What does exist is the raw virtual-serial channel machinery from Chapter 15 — sixteen queues each way, addressed by OP_SERREAD, OP_SERWRITE and their relatives. On this adapter nothing consumes the inbound queues, so they are plumbing without anything plumbed to them.

For a terminal program, open a N:TELNET:// or N:TCP: channel and use the five moves. That is what Chapter 16 does, and it is the answer the platform actually has.

15. COMMAND REFERENCE: DRIVEWIRE IT-SELF

Below the four FujiNet devices is the DriveWire protocol proper. These opcodes are answered by the bus itself: no device, no command byte, no three-step handshake. They are how HDB-DOS gets its sectors, and they are the only part of this book that a program written in 1985 would recognise.

You will not usually send any of them — DOS is doing it for you — but two of them are the disk, so anyone writing their own DOS needs this chapter, and \$23 is by a wide margin the easiest way to ask the time.

15.1 Reading a sector

OP_READEX

\$D2

FRAME	D2 drive lsn2 lsn1 lsn0
AUX	<i>byte</i> : the drive, 0 to 3. <i>three bytes</i> : the logical sector number, high byte first
PAYLOAD	none
REPLY	256 bytes, then you send a 16-bit checksum high byte first , then one status byte comes back
FUJINET-LIB	none — this is DOS's business

The disk read, and the only checksummed exchange on this bus. The adapter always sends the block — on an error it sends 256 zeros — and then you prove you received it by sending back the 16-bit sum of the bytes. DWRead computes that sum for you and leaves it in V, which is why it clobbers your count register.

Status bytes: \$00 success · \$F4 read error · \$F6 invalid or inactive drive · 211 past the end of the image · 243 your checksum did not match.

A drive number of 5 or more is reduced by 5 on a Dragon.

6809 ASSEMBLER

```
*      read drive 0, sector 18
      clr      RXFRM+1
      clr      RXFRM+2
      clr      RXFRM+3
      lda      #18
      sta      RXFRM+4
      ldx      #RXFRM
      ldy      #5
      jsr      [DWWRITV]
      ldx      #SECTOR
      ldy      #256
      jsr      [DWREADV]
      bne      NOANSWER
      sty      CHKSUM      ; DWRead left it in Y
      ldx      #CHKSUM
      ldy      #2
      jsr      [DWWRITV]
      ldx      #STATUS
      ldy      #1
      jsr      [DWREADV]

RXFRM  fcb      OPREADX,0,0,0,0
```

CMOC C

```
struct { uint8_t op, drive, l2, l1, l0; } f;
uint8_t sector[256], st;
uint16_t sum;

f.op = 0xD2; f.drive = 0;
f.l2 = 0; f.l1 = 0; f.l0 = 18;
dwwrite((uint8_t *) &f, sizeof(f));
if (!dwwread(sector, 256)) fail();
sum = checksum(sector, 256);
dwwrite((uint8_t *) &sum, 2); /* big-endian */
dwwread(&st, 1);
```

15.2 Writing a sector

OP_WRITE

\$57 'w'

FRAME	57 drive lsn2 lsn1 lsn0 + 256
AUX	byte: the drive. three bytes: the sector, high byte first
PAYLOAD	256 bytes of data, then a 16-bit checksum low byte first
REPLY	one status byte
FUJINET-LIB	none

The disk write, and two things about it will surprise you.

The checksum goes low byte first here and high byte first on the read. The firmware reads it in the opposite order from the way it reads the other one, and there is no reason for it beyond history.

The checksum is not checked. It is read, the block's sum is computed, and the comparison is commented out. A corrupted write is written.

Status bytes: \$00 success · \$F5 write error · \$F6 invalid or inactive drive. If you send fewer than 256 bytes of data the adapter discards its input and sends **no status byte at all**, which leaves you waiting on a read that will time out and a bus that is out of step.

\$77 'w', re-write, is the same command under another name.

6809 ASSEMBLER

```
ldx    #WXFRM
ldy    #5
jsr    [DWWRITV]
ldx    #SECTOR
ldy    #256
jsr    [DWWRITV]
ldx    #CHKSUM      ; low byte first
ldy    #2
jsr    [DWWRITV]
ldx    #STATUS
ldy    #1
jsr    [DWREADV]
```

WXFRM fcb OPWRITE,0,0,0,0

CMOC C

```
dwwrite((uint8_t *) &f, 5);
dwwrite(sector, 256);
dwwrite(sum_lo_first, 2);
dwread(&st, 1);
```

15.3 The clock, the short way

OP_TIME

\$23 '#'

FRAME 23
AUX none
PAYLOAD none
REPLY 6 bytes: year minus 1900, month (1–12), day, hour, minute, second
FUJINET-LIB clock_get_time(buf, SIMPLE_BINARY)

One byte out, six bytes back, no device and no handshake. It is the cheapest command in this book and it is what fujinet-lib's `clock_get_time()` actually uses.

The year is a `tm_year` — years since 1900 — so add 1900 and not 2000. The month has already been incremented to the 1-to-12 range you expect. The adapter's own timezone applies; there is no flag byte here.

6809 ASSEMBLER

```
DWTIME pshs    x
        jsr     FNWAIT
        ldx     #DWTF
        ldy     #1
        jsr     [DWWRITV]
        puls    x
        ldy     #6
        jmp     [DWREADV]
```

DWTF fcb OPTIME

CMOC C

```
uint8_t op = 0x23, t[6];

dwwrite(&op, 1);
if (dwread(t, 6))
    printf("%u-%02u-%02u %02u:%02u:%02u\n",
           t[0] + 1900, t[1], t[2],
           t[3], t[4], t[5]);
```

15.4 Identification and reset

OP_JEFF

\$A5

FRAME	A5
AUX	none
PAYLOAD	none
REPLY	the seven characters FUJINET, with no terminator
FUJINET-LIB	none

Is there a FujiNet on this wire? A plain DriveWire server does not answer this, so it distinguishes a FujiNet from any other thing that serves disks. Eight bytes, and it fails in the time DWRead takes to give up rather than hanging. Put it at the top of every program.

6809 ASSEMBLER

```
        ldx    #JFRM
        ldy    #1
        jsr    [DWWRITV]
        ldx    #JBUF
        ldy    #7
        jsr    [DWREADV]
        bne    NOFUJI
*        compare JBUF with "FUJINET"

JFRM    fcb    OPJEFF
JSIG    fcc    "FUJINET"
```

CMOC C

```
uint8_t op = 0xA5, buf[8];

dwwrite(&op, 1);
if (!dwread(buf, 7)) return 0;
buf[7] = '\0';
return !strcmp((char *) buf, "FUJINET");
```

OP_DWINIT

\$5A 'Z'

FRAME	5A
AUX	none
PAYLOAD	none
REPLY	one byte, always \$04
FUJINET-LIB	none

The feature enquiry that OS-9's DriveWire driver makes at startup. The firmware defines a whole set of feature bits — EMCEE, DLOAD, HDB-DOS, DOS-PLUS, printer, SSH, sound — and then returns \$04, HDB-DOS, unconditionally, because the branch that would return the real set is compiled out. Do not read anything into the answer.

6809 ASSEMBLER

```
        ldx    #DIFRM
        ldy    #1
        jsr    [DWWRITV]
        ldx    #FEAT
        ldy    #1
        jsr    [DWREADV]

DIFRM    fcb    OPDWINI
```

CMOC C

```
uint8_t op = 0x5A, feat;

dwwrite(&op, 1);
dwread(&feat, 1); /* always 0x04 */
```

OP_RESET

\$F8 / \$FE / \$FF

FRAME	FF
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	none

Three opcodes, one behaviour: re-arm the CONFIG disk and re-insert the boot device. This is a bus reset, not a reboot — the adapter stays up and the network stays connected. It is what the Computer's own reset line triggers, and it is why there is always a way back to CONFIG.

It also closes any named object the Dragon path had open.

6809 ASSEMBLER

```
ldx    #RSFRM
ldy    #1
jsr    [DWWRTIV]
```

```
RSFRM  fcb    OPRST1
```

CMOC C

```
uint8_t op = 0xFF;
dwwrite(&op, 1);
```

OP_NOP

\$00

FRAME	00
AUX	none
PAYLOAD	none
REPLY	none
FUJINET-LIB	none

Does nothing and says nothing. It is here because \$00 on the wire is not an error, which is worth knowing when you are debugging a desynchronised bus: a stream of zeros produces no complaint at all.

6809 ASSEMBLER

```
ldx    #NPFRM
ldy    #1
jmp    [DWWRTIV]
```

```
NPFRM  fcb    OPNOP
```

CMOC C

```
uint8_t op = 0x00;
dwwrite(&op, 1);
```

OP_NAMEOBJ_MNT

\$01

FRAME	01 len + name
AUX	<i>byte</i> : the length of the name
PAYLOAD	that many bytes
REPLY	one byte, \$01
FUJINET-LIB	none

Names a file to be streamed sequentially in place of sector-addressed disk reads, and then resets. It is the Dragon's cassette-style load path: once a named object is set, OP_READEX hands out the next 256 bytes of that file rather than the sector you asked for, falling back to /

AUTOLoad.DWL and, in lobby boot mode, to /DGNLOBBY.DWL.

The Color Computer build does not dispatch this opcode at all.

6809 ASSEMBLER

```
*      Dragon only
      lda    #NAMELEN
      sta    NOFRM+1
      ldx    #NOFRM
      ldy    #2
      jsr    [DWWRTV]
      ldx    #NAME
      ldy    #NAMELEN
      jsr    [DWWRTV]
      ldx    #ACKBYTE
      ldy    #1
      jsr    [DWREADV]

NOFRM  fcb    OPNAMOB,0
```

CMOC C

```
/* Dragon only; ignored on a CoCo. */
```

15.5 Virtual serial channels

DriveWire carries sixteen virtual serial channels in each direction, and the adapter implements the plumbing for all of them. Nothing on the FujiNet side currently fills the inbound queues, so these read as empty and write into a queue nobody drains. They are documented because they are dispatched, and because OS-9's scdwv driver speaks them; if a future firmware attaches the modem or a telnet service to a channel, this is the door it will come through.

\$43	SERREAD	'C'.....	returns two bytes: the first non-empty channel, and one byte from it
\$63	SERREADM	'c'.....	reads a channel number and a count
\$C3	SERWRITE.....	reads a channel number and one byte — with no bounds check on the channel	
\$64	SERWRITE.....	channel, one discarded byte, a count, then that many bytes	
\$80--\$8F	FASTWRITE.....	the channel is the low nybble of the opcode; one data byte follows	
\$45	SERINIT	'E'.....	reads one byte
\$C5	SERTERM.....	reads one byte	
\$44	SERGETSTAT	'D'.....	reads two bytes
\$C4	SERSETSTAT.....	reads a channel and a code; a code of \$28 drains a further 26 bytes	

ONE TO AVOID

OP_SERWRITE does not range-check the channel number where its multi-byte cousin does. A channel number of 16 or more writes past the end of the queue array. There is no reason to send it at all today; there is a good reason not to send it with a bad channel.

15.6 Accepted and ignored

These are dispatched, consume the bytes the protocol says they should, and do nothing else. They exist so that a DriveWire client that sends them does not desynchronise the bus.

\$46	PRINTFLUSH	'F'.....	taken and discarded
\$47	GETSTAT	'G'.....	reads two bytes
\$53	SETSTAT	'S'.....	reads two bytes
\$49	INIT	'I'.....	nothing
\$54	TERM	'T'.....	nothing

And two opcodes in the table are *not* dispatched: \$52 'R', the non-extended read, and \$72 'r', re-read. Only the extended forms \$D2 and \$F2 are answered. Anything else at all falls to the unhandled path, which drains and discards the whole receive buffer — which is, at least, a way back from a desynchronised bus, if a lossy one.

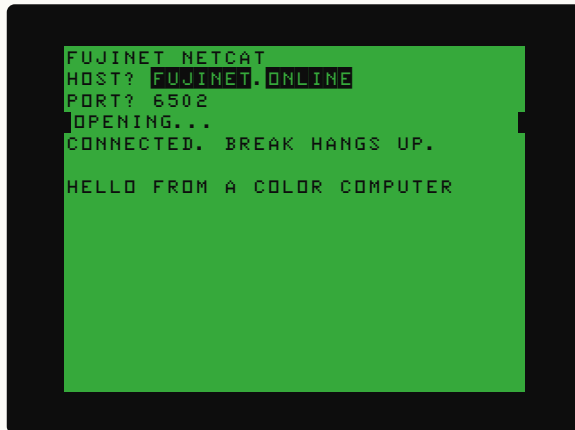


16. NETCAT

Every one of these handbooks ends with a netcat, and for a good reason: it is the smallest program that uses all five moves of the network device, and if it works then the cable, the switches, the vectors, the handshake, the frames and your build are all correct at once.

What it does

Asks for a host and a port, opens a TCP channel, and then sits in a loop: anything you type goes up the wire, anything that arrives goes on the screen. BREAK hangs up.



The loop

Four things happen on every pass, and the order of them is the whole program:

1. Poll the keyboard. If there is a key and it is not BREAK, write that one byte to the channel.

2. Ask STATUS.
3. If bytes are waiting, read *no more than* that many — and no more than the buffer holds — and print them.
4. If nothing is waiting and the connected byte is clear, the far end has gone. Say so and close.

6809 ASSEMBLER

```
LOOP    jsr    [POLCAT]
        tsta
        beq    LRECV
        cmpa   #BREAK
        beq    BYE
        sta    KEYB
        ldx    #KEYB
        ldy    #1
        jsr    NTWRIT

LRECV    ldx    #STBUF
        jsr    NTSTAT
        ldd    STBUF
        beq    LCONN
        cmpd   #RXMAX
        bls    LREAD
        ldd    #RXMAX

LREAD    pshs   d
        ldx    #RXBUF
        tfr    d,y
        jsr    NTREAD
        puls   d
        ldx    #RXBUF
        jsr    PUTBUF
        bra    LOOP

LCONN    lda    STBUF+2
        bne    LOOP
        ldx    #MDROP
        jsr    PUTS
```

The two halves

The C half is the same program with the frames hidden:

CMOC C

```
for (;;)
{
    k = inkey();
    if (k == KBREAK)
        break;
    if (k)
        if (network_write(spec, &k, 1)
            != FN_ERR_OK)
            goto ioerr;

    if (network_status(spec, &bw, &conn, &err)
        != FN_ERR_OK)
        goto ioerr;

    if (bw)
    {
        if (bw > RXMAX)
            bw = RXMAX;
        n = network_read(spec, rx, bw);
        if (n < 0)
            goto ioerr;
        for (i = 0; i < (uint16_t) n; i++)
            putchar(rx[i]);
    }
    else if (!conn)
    {
        printf("\nTHE OTHER END CLOSED.\n");
        break;
    }
}
```

Note that it calls `network_read()` with exactly the count `STATUS` gave it. The library's blocking read will wait until it has everything you asked for, so asking for more than is there turns an interactive program into a hung one. Ask for what is waiting.

ONE BYTE AT A TIME

Writing each keystroke as its own `WRITE` is five bytes of frame for one byte of payload, which is wasteful and exactly right: a terminal that buffers a line cannot be used to talk to anything that echoes. If you are sending a file rather than a conversation, buffer.

Somewhere to point it

fujinet.online:6502..... the project's own echo and chat service
localhost:6502. with FujiNet-PC, a `nc -l 6502` on the same machine
towel.blinkenlights.nl:23..... it still works

Both halves are in Appendix C.4 in full. They are 180 lines of assembly and 110 of C, and between them they are the shortest complete statement of what this book is about.

17. MOUNTER

Netcat exercises the network device. This one exercises the other one: host slots, a directory, a device slot, a mount. It is CONFIG with the menus taken off, and it is about a hundred and fifty lines either way.

The sequence

1. READ_HOST_SLOTS and show the eight.
2. MOUNT_HOST the one the user picked.
3. OPEN_DIRECTORY on it, with *.dsk as the pattern, then READ_DIR_ENTRY until two \$7F bytes arrive, then CLOSE_DIRECTORY.
4. SET_DEVICE_FULLPATH to put the chosen name into a drive slot, then MOUNT_IMAGE to open it.
5. READ_DEVICE_SLOTS and show what the four drives now hold.

```
FUJINET MOUNTER
1 SD
2 TNFS.FUJINET.ONLINE
3 (EMPTY)

HOST? 2
1 COCOLOBBY.DSK
2 NETCAT.DSK

FILE? 2
DRIVE (1-4)? 1

READY ON DRIVE 0
```

Two frames by hand

Two of those commands do not fit FNCMD1, and the reason is the same both times: they take more than one parameter byte, and the adapter will read every byte the command declares whether you sent it or not. READ_DIR_ENTRY takes a length and a flags byte:

6809 ASSEMBLER

```
RDFRM    fcb      OPFUJI,FCRDDIR,36,0

DIR1     ldx      #RDFRM
         ldy      #4
         jsr      FNXACT
```

and SET_DEVICE_FULLPATH takes three, plus a 256-byte name:

6809 ASSEMBLER

```
SDFRM    fcb      OPFUJI,FCSDVFP,0,0,0
SDNAME   rmb      256

         lda      DS
         sta      SDFRM+2
         lda      HS
         sta      SDFRM+3
         clr      SDFRM+4          ; mode 0
         ldx      #SDFRM
         ldy      #261
         jsr      FNXACT
```

This is the pattern for every payload command in Chapter 12: declare the frame with its fixed head and its buffer, patch the parameters in, and send the whole thing as one write.

Reading the directory

The pattern goes in the same 256-byte field as the path, after the path's own terminator:

6809 ASSEMBLER

```
ldy    #ODPATH
ldx    #MROOT      ; "/"
jsr    SCOPY
leay   1,y          ; over the NUL
ldx    #MFILT      ; "*.dsk"
jsr    SCOPY

MROOT  fcc    "/"
       fcb    0
MFILT  fcc    "*.dsk"
       fcb    0
```

and the read loop watches for the two \$7F bytes:

CMOC C

```
while (n < MAXENT)
{
    if (!fuji_read_directory(36, 0, entry))
        break;
    if ((uint8_t) entry[0] == 0x7F)
        break;
    strcpy(names[n], entry);
    printf("%u %s\n", n + 1, names[n]);
    n++;
}
fuji_close_directory();
```

Note the pattern is lowercase. Host filesystems are frequently case-sensitive and the Color Computer's keyboard is not much help, so the program carries its pattern as a constant rather than asking for one.

CLOSE THE DIRECTORY

There is one directory handle for the whole adapter. A program that exits without closing leaves the next program — CONFIG, very possibly — unable to open one. The listing closes even on the paths where it found nothing.

What CONFIG adds

Not very much, and it is worth saying what: prefixes, so you can walk into a subdirectory; the \$40 bit on the mode byte, so a mounted drive shows a marker; WRITE_DEVICE_SLOTS and MOUNT_ALL, so the

configuration survives a power cycle; and a screen you can navigate with the arrow keys.

All four are commands in Chapter 12 and none of them changes the shape of this program. The full listings are in Appendix C.5.

18. WEATHER

The response from a modern web API is forty kilobytes of JSON and the Color Computer has thirty-two. This program reads three numbers out of one without ever holding more than sixty-four bytes of it.

Why a parser

Because the parsing happens on the adapter. You open the channel, tell the adapter to read the whole response and parse it, and from then on the document is the adapter's problem. Each query you send selects a value, and only that value crosses the wire.

It is the single most useful thing the network device does, and it is the reason a 1980 machine can talk to a 2026 service at all.

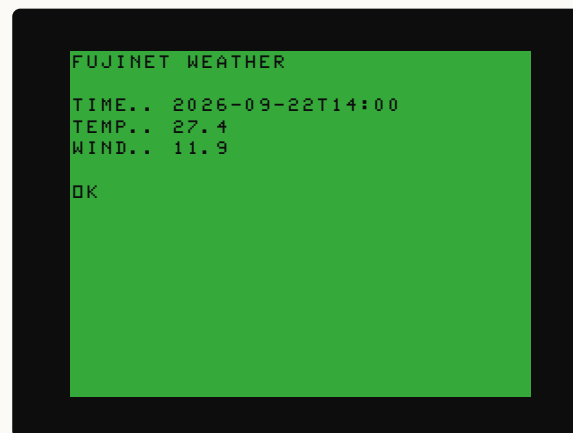
The sequence

1. OPEN the URL with mode \$04 — an HTTP GET.
2. SET_PARSER with the mode in **aux2**.
3. PARSE. The adapter reads the response to the end and builds the document.
4. For each question: QUERY with a path, then STATUS to find out how long the answer is, then READ.
5. CLOSE.

CMOC C

```
static char spec[] =
    "N:HTTPS://api.open-meteo.com/v1/forecast"
    "?latitude=39.10&longitude=-94.58"
    "&current=temperature_2m,wind_speed_10m";

network_open(spec, OPEN_MODE_HTTP_GET,
             OPEN_TRANS_NONE);
json_mode(network_unit(spec));
show("TIME.. ", "/current/time");
show("TEMP.. ", "/current/temperature_2m");
show("WIND.. ", "/current/wind_speed_10m");
network_close(spec);
```



The frame the library gets wrong

This is the program in this book that has to go below `fujinet-lib`, and it is worth understanding why rather than just copying the workaround. `network_json_parse()` sends `SET_PARSER` with the mode in `aux1` and zero in `aux2`. The firmware reads the mode out of `aux2`. So the call succeeds, selects the null parser, and every query afterwards comes back empty — which looks exactly like an API that has stopped answering.

Five bytes fix it:

```
CMOC C
static uint8_t json_mode(uint8_t unit)
{
    struct { uint8_t op, unit, cmd,
              aux1, aux2; } f;
    uint8_t e;

    f.op   = 0xE3;
    f.unit = unit;
    f.cmd  = 0xFC;      /* SET_PARSER */
    f.aux1 = 0;
    f.aux2 = 1;        /* JSON --- aux2 */

    bus_ready();
    dwwrite((byte *) &f, sizeof(f));
    e = network_get_error(unit);
    if (e != FN_ERR_OK)
        return e;

    f.cmd  = 'P';      /* PARSE */
    f.aux1 = f.aux2 = 0;

    bus_ready();
    dwwrite((byte *) &f, sizeof(f));
    return network_get_error(unit);
}
```

and the assembly half never had the problem, because it was building the frame by hand all along:

```
6809 ASSEMBLER
SPFRM   fcb   OPNET,0,NCSPARS,0,PMJSON
PAFRM   fcb   OPNET,0,NCPARSE,0,0
```

Asking a question

A query sets the channel's contents to whatever it selected, so the read after it is an ordinary read:

```
6809 ASSEMBLER
ASK      pshs   x,y
         jsr    PUTS          ; the label
         ...      ; clear QYSTR,
         ...      ; copy the query in
         lda     NTUNI
         sta     QYFRM+1
         ldx     #QYFRM
         ldy     #QYLEN      ; 5 + 256
         ldb     NTUNI
         jsr     NTXACT
         bne     ASKBAD
         ldx     #STBUF
         jsr     NTSTAT
         ldd     STBUF
         beq     ASKBAD
         ...      ; clamp, read, print
```

An empty answer is not an error — it means the path selected nothing, which usually means a typo in the path or an API that changed its shape. Print a question mark and carry on; the listing does.

Both halves are in Appendix C.6.

19. FNSTAT UNDER OS-9

The last program asks the same question as the first — who is on the other end of the wire — and answers it from inside OS-9, where none of the machinery the rest of this book uses is available.

It is thirty lines on top of two files that are the real content of this chapter.

Three files

dwport.c.dwread and dwwrite. Twenty lines of inline 6809 and the only part that touches hardware.

fnbus.c.fn_wait, fn_error, fn_send, fn_response. Chapter 6's library, in C.

fnstat.c..... the program.

Nothing above dwport.c knows what kind of wire it is on. That is the whole argument of Chapter 3, cashed in.

The transport

Memory-mapped: status at \$FF41, data at \$FF42. Interrupts masked, because the port holds one byte and an OS-9 clock tick will take it. A timeout, because fn_wait retries and a poll loop with no timeout would never give it the chance.

CMOC C

```
byte dwread(byte *s, int l)
{
    asm
    {
        pshs    cc,x,y,u
        orcc    #$50
        ldx     :s
        ldy     :l
        beq     @ok
        @byte   ldu     #$2000
        @wait   lda     $FF41
        bita    #$02
        bne     @got
        leau    -1,u
        bne     @wait
        bra     @timeout
        @got    lda     $FF42
        sta     ,x+
        leay    -1,y
        bne     @byte
        @ok     puls    cc,x,y,u
        ldb     #1
        bra     @exit
        @timeout puls    cc,x,y,u
        clrb
        @exit
    }
}
```

This is the Becker port, the CoCo 3 FPGA's DriveWire window and the high-speed UART cartridge. For the bit-banger — which is what a FujiNet cartridge actually plugs into — take DWRead and DWWrite from your NitrOS-9 or DriveWire sources and put them behind these two names. Chapter 11 says why this book does not reprint them.

The bus

Four functions, and they are recognisably the same four:

CMOC C

```
void fn_wait(void)
{
    byte frame[2];
    byte r;

    frame[0] = 0xE2;
    frame[1] = 0x00;

    do
        dwwrite(frame, sizeof(frame));
    while (!dwread(&r, 1));
}

byte fn_send(byte *frame, int len)
{
    fn_wait();
    dwwrite(frame, len);
    return fn_error();
}
```

And then the program is Chapter 5 again:

CMOC C

```
frame[0] = 0xE2;
frame[1] = 0xC4;          /* ADAPTERCONFIG_EXT */

e = fn_send(frame, sizeof(frame));
if (e != FN_OK) { ... }

e = fn_response(cfg, 240);
if (e != FN_OK) { ... }

printf("network  %s\n", &cfg[0]);
printf("address  %s\n", &cfg[140]);
printf("firmware %s\n", &cfg[125]);
```

with the offsets spelled out instead of a structure, because we are not linking fujinet-lib and so we do not have its headers.

Building it

AT THE PROMPT

```
$ cmoc --os9 -o fnstat \
    fnstat.c fnbus.c dwport.c
$ os9 copy fnstat /dd/CMDS/fnstat
$ chmod e+ /dd/CMDS/fnstat
```

and then it is an ordinary command:

AT THE PROMPT

```
$ fnstat
network  HOME-2G
address  192.168.1.44
firmware 1.8.1
```

Lowercase, because OS-9 has a screen that can show it.

What is verified

Plainly, because the rest of this book is:

- The three files compile with `cmoc --os9`, and Appendix C.7 prints them from the files that compiled.
- The transport is correct by inspection of the register map. It has not been run against hardware for this book.
- The bit-banger route is described, not supplied.
- Everything from `fn_wait()` upward is the same protocol as the rest of the book, and that part is verified throughout it.

20. ALWAYS MORE TO COME

This book documents the bus: how to get a command to the adapter and an answer back. It has said almost nothing about what is at the far end of a URL, and that is deliberate, because the far end has a book of its own.

The other book

The *FujiNet Network Protocol Handbook* is the companion volume, and it documents every scheme the N: device speaks: TCP:, TELNET:, UDP:, HTTP: and HTTPS:, TNFS:, SMB:, FTP:, SSH:, JSON:, GMAIL:, GCAL: and more — what each one does with the open modes, what a directory looks like on it, which of them support seeking, and what their error codes mean.

Everything in it applies to the Color Computer without change. A devicespec is a devicespec.

Where the sources are

Nothing in this book is a secret and all of it moves. The places to look when this book and the adapter disagree:

fujinet-firmware. the adapter. lib/bus/drivewire/ is this bus; lib/device/fujiDevice/ and lib/device/NDevice/ are the two devices in Chapters 12 and 13.
fujinet-lib. . . . the C client library. coco/src/ is this platform's half.
fujinet-apps. . . real programs, several of them with a coco/ directory.
fujinet-hardware. . the cartridge: schematics, board files and the case.

All of them are on GitHub under FujiNetWiFi, and all of them take patches. The firmware bugs this book names in Chapters 12, 13 and 14 are named precisely so that somebody can fix them; if you are that somebody, the fix is welcome and this book will be wrong in the next edition, happily.

Where the people are

fujinet.online. . . the project, the documentation and the disk images
The Discord. where the firmware is argued about and where somebody will answer a question about a desynchronised bus at two in the morning
github.com/FujiNetWiFi the sources, the issues and the wiki edition of this book

Every address, frame and listing in this book was transcribed from the live FujiNet sources and checked against a client that speaks to them. When in doubt, the sources win.

The FujiNet Project · fujinet.online

A LAST WORD

The Color Computer was sold as a machine you could program, in a store, with a manual in the box. That the same machine can now hold a JSON conversation with a weather service is not nostalgia; it is the machine doing exactly what it was sold to do, forty years later, because somebody wrote down how.

Write something. Put it on a disk image. Tell somebody where to find it.

APPENDIX A

DRIVEWIRE QUICK REFERENCE

The two vectors

\$D93F. . DWRead — X = buffer, Y = count. **Z set** when every byte arrived; carry set on a framing error. Clobbers Y (a checksum) and all accumulators; keeps X and U. Times out in about 1.5 seconds.

\$D941.DWWrite — X = buffer, Y = count. X ends one past the last byte, Y ends zero. No error return.

On a Dragon: \$F9FE and \$FA00.

Baud by model

Color Computer 1.....	38,400
Color Computer 2.....	57,600
Color Computer 3.....	115,200
Dragon.....	57,600
High-speed UART.....	921,600
DriveWire over IP.....	TCP port 65504

The four FujiNet devices

\$E2.....	the FujiNet control device — Chapter 12
\$E3.....	the N: network devices — Chapter 13
\$E4.....	CP/M — Chapter 14, ESP32 firmware only
\$E5.....	the clock — Chapter 14

Bus opcodes

\$00.....	NOP
\$01.....	named object mount (Dragon)
\$23.....	clock --- six bytes back
\$43.....	serial read

\$44.....	serial get status
\$45.....	serial init
\$46.....	print flush (ignored)
\$47.....	get status
\$49.....	init
\$50.....	print one byte
\$53.....	set status
\$54.....	terminate
\$57.....	write a sector
\$5A.....	DWINIT --- replies \$04
\$63.....	serial read multiple
\$64.....	serial write multiple
\$77.....	re-write a sector
\$80--\$8F.....	fast write, channel in the low nybble
\$A5.....	replies "FUJINET"
\$C3.....	serial write one byte
\$C4.....	serial set status
\$C5.....	serial terminate
\$D2.....	read a sector
\$F2.....	re-read a sector
\$F8 \$FE \$FF.....	reset

Frames

The FujiNet control device, \$E2:

\$E2	command	0 to 3 aux bytes	payload, if any
------	---------	------------------	-----------------

The network device, \$E3 — always two aux bytes:

\$E3	unit	command	aux1	aux2	payload
------	------	---------	------	------	---------

Multi-byte parameters and replies are **high byte first**, which is the 6809's own order. The four exceptions, all replies: GET_DIRECTORY_POSITION, RANDOM_NUMBER, the network device's TELL, and the write-sector checksum.

The three meta-commands

\$00..... DEVICE_READY — replies \$01; loop until it does
\$02..... SEND_ERROR — one byte, \$01 is success
\$01..... SEND_RESPONSE — the parked reply; you supply the count

The dance

- 1. \$E2 \$00, read one byte, loop until it comes.
 - 2. The command frame. Nothing comes back.
 - 3. \$E2 \$02, read the verdict.
 - 4. If it owes you data, \$E2 \$01 and read exactly the count on the card.
- On the network device every one of those carries the unit and two aux bytes, and SEND_RESPONSE takes the length in its aux word.
The clock, \$E5, skips the whole thing: write the frame, read the reply.

Open modes and translation

\$04 / \$06 / \$07..... read · directory · directory alt
\$08 / \$09 / \$0C..... write · append · read and write
\$05 / \$0D / \$0E..... HTTP DELETE · POST · PUT with headers
\$00..... no translation
\$01..... CR --- the Color Computer's own
\$02 / \$03..... LF · CR LF
\$04..... PETSCII

Sector protocol

read..... \$D2 drive lsn(3) → 256 bytes → checksum **hi lo**
..... → status: \$00 ok · \$F4 read · \$F6 drive · 211 EOF · 243 checksum

write..... \$57 drive lsn(3) 256 bytes checksum **lo hi**
..... → status: \$00 ok · \$F5 write · \$F6 drive

APPENDIX B

ERROR AND STATUS CODES

The device error byte

What SEND_ERROR — command \$02 on \$E2 or \$E3 — gives you. One is success; everything else is a complaint.

1.	success
131.	the channel is write-only
132.	invalid command
135.	the channel is read-only
136.	end of file --- not a failure
138.	general timeout
144.	a fatal error; also what the bus sets on any refused command
146.	not implemented
151.	the file exists
162.	no space left on the device
165.	the devicespec was not understood
166.	invalid seek position
167.	access denied
170.	file not found
200.	connection refused
201.	network unreachable
202.	socket timeout
203.	network down
204.	connection reset
205.	connection already in progress
206.	address in use
207.	not connected
208.	the server is not running
209.	no connection waiting
210.	service not available

211.	connection aborted
212.	invalid username or password
213.	could not parse the document
214.	general client error
215.	general server error
255.	could not allocate buffers

136 IS NOT A FAILURE

End of file is how a resource says it is finished. A program that treats it as an error will report a failure every time it successfully reads a whole file. Check for it before you check for anything else.

144 MEANS TWO THINGS

It is the firmware's general fatal error, and it is also what the bus writes when a command it does not recognise takes the error path. On the network device a 144 from a command you are sure exists usually means you misspelled it.

Sector status

The byte that ends an \$D2 or \$57 exchange:

\$00..... success
\$F4..... read error
\$F5..... write error
\$F6..... invalid or inactive drive
211..... past the end of the image
243..... your checksum did not match --- read only

Network status

The four bytes STATUS returns on \$E3:

waiting, high	waiting, low	connected	error
---------------	--------------	-----------	-------

bytes waiting.....how much you may read. Zero is normal.
connected..... non-zero while the far end is there.
errorthe channel's own code, from the table opposite. 1 is fine; 136 is the
end; 207 means nothing is open.

The end of a resource is **136**, or zero waiting with the connected byte
clear. Never zero waiting on its own.

WiFi status

The byte GET_WIFISTATUS returns:

1..... no SSID available
3..... connected
4..... connection failed
5..... connection lost
6..... disconnected

In practice you will see 3 and 6.

The library's codes

fujinet-lib's network_ calls return these instead of the adapter's byte.
The raw byte is left in fn_device_error.

FN_ERR_OK..... \$00
FN_ERR_IO_ERROR..... \$01
FN_ERR_BAD_CMD..... \$02
FN_ERR_OFFLINE..... \$03
FN_ERR_WARNING..... \$04
FN_ERR_NO_DEVICE..... \$05
FN_ERR_UNKNOWN..... \$FF

The Color Computer's mapping is blunt: exactly \$01 becomes
FN_ERR_OK and everything else becomes FN_ERR_IO_ERROR. To tell end-
of-file from connection-refused, read fn_device_error.

The fuji_ calls return a bool instead, and leave the byte in the
same place.



APPENDIX C

PROGRAM LISTINGS

Every file here is read off the disk when this book is typeset, so what is printed is the text that produced the binaries — character for character, with nothing retyped and nothing abridged. The Makefiles that build them are in the same directories.

The assembly files assemble with `lwasm 4.24` and the C files compile with `cmoc 0.1.100`. Neither has been run against hardware for this book; every byte layout in them was read out of the firmware and every one of them builds from clean.

C.1 The equates

fn.inc

```
1 *****
2 * FN.INC --- FujiNet equates for the TRS-80 Color Computer
3 *
4 * Every value here is transcribed from the FujiNet firmware:
5 *   opcodes ... lib/bus/drivewire/opcode.h
6 *   commands ... include/fujiCommandID.h
7 *   errors ... lib/network-protocol/status_error_codes.h
8 *
9 * Assemble with lwasm. Include this before FNLOW.ASM.
10 *****
11
12 * --- DriveWire ROM vectors -----
13 * Disk BASIC keeps the DriveWire entry points in two indirect
14 * vectors. X holds the buffer, Y holds the count.
15 DWREADV EQU    $D93F      READ  --- JSR [DWREADV]
16 DWWRITV EQU    $D941      WRITE --- JSR [DWWRITV]
17
18 * --- DriveWire opcodes -----
19 OPNOP EQU      $00        do nothing
20 OPNAMO EQU     $01        named object mount (Dragon)
21 OPTIME EQU     $23        '#' six-byte clock reply
22 OPSERR EQU     $43        'C' read a virtual serial byte
23 OPSGST EQU     $44        'D' serial get status
24 OPSERIN EQU     $45        'E' serial init
25 OPPRTFL EQU     $46        'F' printer flush (ignored)
26 OPGSTAT EQU     $47        'G' get status
27 OPINIT EQU     $49        'I' init
28 OPPRINT EQU     $50        'P' one byte to the printer
29 OPSSTAT EQU     $53        'S' set status
30 OPTERM EQU     $54        'T' terminate
31 OPWRITE EQU     $57        'W' write one 256-byte block
32 OPDWINI EQU     $5A        'Z' DWINIT --- replies $04
33 OPSERRM EQU     $63        'c' serial read multiple
34 OPSERWM EQU     $64        serial write multiple
35 OPFASTW EQU     $80        $80-$8F fast write, vchan in low nybble
36 OPJEFF EQU      $A5        replies with the string "FUJINET"
37 OPSERWR EQU     $C3        serial write one byte
38 OPSSTA EQU      $C4        serial set status
39 OPSERTM EQU     $C5        serial terminate
40 OPREADX EQU     $D2        READEX --- read one 256-byte block
41 OPFUJI EQU      $E2        the FujiNet control device
42 OPNET EQU       $E3        the N: network devices
43 OPCPM EQU       $E4        the CP/M console
44 OPCLOCK EQU     $E5        the clock device
45 OPRREDX EQU     $F2        re-read
46 OPRST3 EQU      $F8        reset
47 OPRST2 EQU      $FE        reset
48 OPRST1 EQU      $FF        reset
49
50 * --- the three bus meta-commands -----
51 * These are answered by the bus itself, ahead of any device.
52 FNREADY EQU      $00        are you there? --- replies $01
53 FNRESP EQU       $01        send the response
54 FNERROR EQU      $02        send the error byte
55
56 * --- FUJI device commands ($E2) -----
57 FCRESET EQU      $FF        reboot the adapter
58 FCSSID EQU       $FE        get the stored SSID and password
59 FCSCAN EQU       $FD        scan for networks
60 FCSCANR EQU      $FC        get one scan result
61 FCSSSID EQU      $FB        set SSID and password, and join
62 FCWIFST EQU      $FA        get WiFi status
63 FCMNTHS EQU      $F9        mount a host slot
64 FCMNTIM EQU      $F8        mount a disk image
65 FCOPNDR EQU      $F7        open a directory
66 FCRDIR EQU       $F6        read one directory entry
```

67	FCCLSDR	EQU	\$F5	close the directory	138	NCCLCLI	EQU	\$63	'c' close the accepted client
68	FCRDHS	EQU	\$F4	read the eight host slots	139	NCGTREM	EQU	\$72	'r' get the UDP remote address
69	FCWRHS	EQU	\$F3	write the eight host slots	140	NCSPARM	EQU	\$FB	set a parser parameter
70	FCRDDS	EQU	\$F2	read the four device slots	141	NCSPARS	EQU	\$FC	set the parser mode
71	FCWRDS	EQU	\$F1	write the four device slots	142				
72	FCWIFEN	EQU	\$EA	is WiFi enabled?	143	* --- open modes, aux1 of NCOPEN -----			
73	FCUNMIM	EQU	\$E9	unmount a disk image	144	OMREAD	EQU	4	read
74	FCADCFG	EQU	\$E8	get the adapter configuration	145	OMDIR	EQU	6	directory
75	FCNEWDK	EQU	\$E7	create a new blank disk	146	OMDIRA	EQU	7	directory, alternate form
76	FCUNMHS	EQU	\$E6	unmount a host slot	147	OMWRITE	EQU	8	write
77	FCGDIRP	EQU	\$E5	get the directory position	148	OMAPPND	EQU	9	append
78	FCSDIRP	EQU	\$E4	set the directory position	149	OMRW	EQU	12	read and write
79	FCSDVFP	EQU	\$E2	set a device slot's full path	150	* the same byte carries the HTTP method			
80	FCSHPPFX	EQU	\$E1	set a host prefix	151	OMHGET	EQU	4	GET
81	FCGHPFX	EQU	\$E0	get a host prefix	152	OMHDEL	EQU	5	DELETE
82	FCWRAPK	EQU	\$DE	write the open app key	153	OMHPUT	EQU	8	PUT
83	FCRDAPK	EQU	\$DD	read the open app key	154	OMHDELH	EQU	9	DELETE, collecting headers
84	FCOPAPK	EQU	\$DC	open an app key	155	OMHGETH	EQU	12	GET, collecting headers
85	FCCLAPK	EQU	\$DB	close the app key	156	OMHPOST	EQU	13	POST
86	FCGDIVFP	EQU	\$DA	get a device slot's full path	157	OMHPUTH	EQU	14	PUT, collecting headers
87	FCCFGBT	EQU	\$D9	enable or disable the CONFIG disk	158				
88	FCCOPYF	EQU	\$D8	copy a file between hosts	159	* --- translation modes, aux2 of NCOPEN -----			
89	FCMNTAL	EQU	\$D7	mount everything in the slots	160	TRNONE	EQU	0	leave the bytes alone
90	FCSB00T	EQU	\$D6	set the boot mode	161	TRCR	EQU	1	end lines with CR --- the CoCo's own
91	FCRANDM	EQU	\$D3	get four random bytes	162	TRLF	EQU	2	end lines with LF
92	FCB64EI	EQU	\$D0	base64 encode: input	163	TRCRLF	EQU	3	end lines with CR LF
93	FCB64EC	EQU	\$CF	base64 encode: compute	164	TRPET	EQU	4	PETSCII
94	FCB64EL	EQU	\$CE	base64 encode: length	165				
95	FCB64EO	EQU	\$CD	base64 encode: output	166	* --- parser modes, aux2 of NCSPARS -----			
96	FCB64DI	EQU	\$CC	base64 decode: input	167	PMNONE	EQU	0	no parser
97	FCB64DC	EQU	\$CB	base64 decode: compute	168	PMJSON	EQU	1	JSON
98	FCB64DL	EQU	\$CA	base64 decode: length	169	PMHTML	EQU	2	HTML, queried with a CSS selector
99	FCB64DO	EQU	\$C9	base64 decode: output	170	PMXML	EQU	3	XML, queried with an XPath subset
100	FCHASHI	EQU	\$C8	hash: input	171				
101	FCHASHC	EQU	\$C7	hash: compute and clear	172	* --- HTTP channel modes, aux2 of NCHTTPM -----			
102	FCHASHL	EQU	\$C6	hash: length	173	HMBODY	EQU	0	the response body
103	FCHASHO	EQU	\$C5	hash: output	174	HMCOLL	EQU	1	collect response headers
104	FCADCFX	EQU	\$C4	get the extended adapter config	175	HMGETH	EQU	2	read the collected headers
105	FCHASHN	EQU	\$C3	hash: compute, keep the input	176	HMSETH	EQU	3	set a request header
106	FCHASHZ	EQU	\$C2	hash: clear	177	HMPOSTD	EQU	4	set the POST data
107	FCQROUT	EQU	\$BF	QR code: output	178				
108	FCQRLN	EQU	\$BE	QR code: length	179	* --- hash algorithms, aux1 of FCHASHC -----			
109	FCQRENC	EQU	\$BD	QR code: encode	180	HAMD5	EQU	0	
110	FCQRINP	EQU	\$BC	QR code: input	181	HASHA1	EQU	1	
111	FCGUID	EQU	\$BB	generate a UUID	182	HASHA256	EQU	2	
112	FCSTAT	EQU	\$53	'S' status (a four-byte stub)	183	HASHA512	EQU	3	
113					184				
114	* --- NETWORK device commands (\$E3) -----				185	* --- error codes returned by FNERROR -----			
115	NCRENAM	EQU	\$20	' ' rename a file	186	ESUCC	EQU	1	success
116	NDELETE	EQU	\$21	'!' delete a file	187	EWONLY	EQU	131	the channel is write-only
117	NCLOCK	EQU	\$23	'#' lock a file	188	EBADCMD	EQU	132	invalid command
118	NCUNLCK	EQU	\$24	'\$' unlock a file	189	ERDONLY	EQU	135	the channel is read-only
119	NCSEEK	EQU	\$25	'%' seek	190	EOF	EQU	136	end of file
120	NCTELL	EQU	\$26	'&' tell	191	ETIMOUT	EQU	138	general timeout
121	NCKMDIR	EQU	\$2A	'*' make a directory	192	EGENERL	EQU	144	a fatal error
122	NCRMDIR	EQU	\$2B	'+' remove a directory	193	ENOTIMP	EQU	146	not implemented
123	NCCDIR	EQU	\$2C	',' change the prefix	194	EEXISTS	EQU	151	the file exists
124	NCGETCW	EQU	\$30	'0' get the prefix	195	ENOSPC	EQU	162	no space left on the device
125	NCACCP	EQU	\$41	'A' accept a TCP connection	196	EBADSPC	EQU	165	the devicespec was not understood
126	NCCLSE	EQU	\$43	'C' close the channel	197	EBADPT	EQU	166	invalid seek position
127	NCSETDE	EQU	\$44	'D' set the UDP destination	198	EACCESS	EQU	167	access denied
128	NCSETEL	EQU	\$4C	'L' set the end-of-line string	199	ENOFILE	EQU	170	file not found
129	NCHTTPM	EQU	\$4D	'M' set the HTTP channel mode	200	ECONREF	EQU	200	connection refused
130	NCOPEN	EQU	\$4F	'O' open a channel	201	ENETUNR	EQU	201	network unreachable
131	NCPARSE	EQU	\$50	'P' parse the channel contents	202	ESOCKTO	EQU	202	socket timeout
132	NCQUERY	EQU	\$51	'Q' query the parsed document	203	ENETDOWN	EQU	203	network down
133	NCREAD	EQU	\$52	'R' read from the channel	204	ECONRST	EQU	204	connection reset
134	NCSTAT	EQU	\$53	'S' status	205	ECONPRG	EQU	205	connection already in progress
135	NCTRANS	EQU	\$54	'T' set the translation mode	206	EADDRIN	EQU	206	address in use
136	NCWRITE	EQU	\$57	'W' write to the channel	207	ENOTCON	EQU	207	not connected
137	NCINTRT	EQU	\$5A	'Z' set the interrupt rate	208	ENOSERV	EQU	208	the server is not running

```

209 ENOWAIT EQU      209      no connection waiting
210 ENOSVC EQU      210      service not available
211 ECONABT EQU      211      connection aborted
212 EBADLOG EQU      212      invalid username or password
213 EBADJSN EQU      213      could not parse the JSON
214 ECLIENR EQU      214      general client error
215 ESERVER EQU      215      general server error
216 ENOBUFS EQU      255      could not allocate buffers
217
218 * --- Wifi status, the reply to FCWIFST -----
219 WFNOSID EQU      1        no SSID available
220 WFCNN EQU      3          connected
221 WFFAIL EQU      4          connection failed
222 WFLOST EQU      5          connection lost
223 WFDISC EQU      6          disconnected
224
225 * --- sizes you will need -----
226 SZHOSTS EQU      256      eight host slots, 32 bytes each
227 SZDEVS EQU      152      four device slots, 38 bytes each
228 SZADCFG EQU      140      AdapterConfig
229 SZADCFX EQU      240      AdapterConfigExtended
230 SZSSID EQU      97        ssid[33] + password[64]
231 SZSCANR EQU      34        ssid[33] + rssi
232 SZGUID EQU      37        a UUID and its terminator
233 SZAPKEY EQU      64        an app key's payload
234 SZSPEC EQU      256      a devicespec, a path, a prefix

```

C.2 The library

fnlow.asm — the handshake, the verdict, the response

```

1 *****
2 * FNLOW.ASM --- the low road to FujiNet from 6809 assembly
3 *
4 * Nine subroutines. Everything else in this book is built on
5 * them. INCLUDE FN.INC before this file.
6 *
7 * The Color Computer talks to FujiNet through the two DriveWire
8 * vectors Disk BASIC leaves at $D93F and $D941. Both take the
9 * buffer in X and the count in Y. DWRead returns with Z set
10 * when every byte arrived and the carry set on a framing error;
11 * it clobbers Y (it hands back a checksum) and the
12 * accumulators, but preserves X. DWWrite consumes both X and Y
13 * and preserves everything else. These routines put the
14 * registers back where you left them unless the header says
15 * otherwise.
16 *****
17
18 * -----
19 * FNWAIT --- block until the adapter answers.
20 *
21 * Writes the two-byte frame $E2 $00 and waits for the single
22 * byte $01 to come back. If nothing comes back it simply asks
23 * again. Every command in this library begins here, because
24 * the adapter is free to be busy and the bus has no other way
25 * of saying so.
26 *
27 * Entry: nothing. Exit: all registers preserved.
28 * -----
29 FNWAIT PSHS      D,X,Y
30 FNWAIT1 LDX      #FNRDYF
31          LDY      #2
32          JSR      [DWWRITV]
33          LDX      #FNSCRIP
34          LDY      #1
35          JSR      [DWREADV]
36
37          BNE      FNWAIT1      nothing yet --- ask again
38          PULS      D,X,Y,PC
39
40 * -----
41 * FNXACT --- send a FUJI request and collect the verdict.
42 *
43 * Entry: X = the request frame, whose first byte must be $E2
44 *        Y = the length of that frame
45 * Exit: A = the error byte, $01 on success
46 *        Z set when A is $01, so BNE branches on failure
47 *        X and Y are clobbered
48
49 FNXACT JSR      FNWAIT
50          JSR      [DWWRITV]      X and Y are consumed here
51          * fall into FNGERR
52
53 * -----
54 * FNGERR --- ask the FUJI device how the last command went.
55 *
56 * Entry: nothing.
57 * Exit: A = the error byte; Z set when it is $01; X, Y kept.
58
59 FNGERR PSHS      X,Y
60          JSR      FNWAIT
61          LDX      #FNERRF
62          LDY      #2
63          JSR      [DWWRITV]
64          LDX      #FNSCRIP
65          LDY      #1
66          JSR      [DWREADV]
67          BNE      FNGE1      no answer at all
68          LDA      FNSCRIP
69          BRA      FNGE2
70 FNGE1 LDA      #EGENERL      call silence a fatal error
71 FNGE2 PULS      X,Y

```

```

71          CMPA    #ESUCC
72          RTS
73
74 * -----
75 * FNGRSP --- fetch the response the last command left waiting.
76 *
77 * The adapter does not push data at you. A command that has
78 * something to say parks it, and you come back for it with this.
79 *
80 * Entry: X = where to put it
81 *        Y = how many bytes to take
82 * Exit:  Z set when every byte arrived; X preserved, Y clobbered
83 * -----
84 FNGRSP    PSHS    X,Y
85          JSR      FNWAIT
86          LDX      #FNRSFP
87          LDY      #2
88          JSR      [DWWRITV]
89          PULS     X,Y
90          JMP      [DWREADV]
91
92 * -----
93 * FNCMD0 --- issue a FUJI command that takes no parameters.
94 *
95 * Entry: A = the command byte
96 * Exit:  as FNXACT
97 * -----
98 FNCMD0    STA     FNC0F+1
99          LDX      #FNC0F
100         LDY      #2
101         BRA      FNXACT
102
103 * -----
104 * FNCMD1 --- issue a FUJI command that takes one byte.
105 *
106 * Entry: A = the command byte, B = the parameter
107 * Exit:  as FNXACT
108 * -----
109 FNCMD1    STA     FNC1F+1
110         STB      FNC1F+2
111         LDX      #FNC1F
112         LDY      #3
113         BRA      FNXACT
114
115 * -----
116 * NTXACT --- send an N: request and collect the verdict.
117 *
118 * Entry: X = the request frame, whose first byte must be $E3
119 *        and whose second must be the unit
120 *        Y = the length of that frame
121 *        B = the unit again, for the error fetch
122 * Exit:  A = the error byte; Z set when it is $01
123 * -----
124 NTXACT    JSR      FNWAIT
125         JSR      [DWWRITV]
126 *        fall into NTGERR
127
128 * -----
129 * NTGERR --- ask one N: unit how its last command went.
130 *
131 * Entry: B = the unit (1 to 255)
132 * Exit:  A = the error byte; Z set when it is $01; X, Y kept.
133 * -----
134 NTGERR    PSHS     X,Y
135         STB      NTERRF+1
136         JSR      FNWAIT
137         LDX      #NTERRF
138         LDY      #5
139         JSR      [DWWRITV]
140         LDX      #FNSCRPF
141         LDY      #1

```

```

142         JSR      [DWREADV]
143         BNE      NTGE1
144         LDA      FNSCRPF
145         BRA      NTGE2
146 NTGE1     LDA      #EGENERL
147 NTGE2     PULS     X,Y
148         CMPA     #ESUCC
149         RTS
150
151 * -----
152 * NTGRSP --- fetch an N: unit's pending response.
153 *
154 * The length goes out in the frame as well as governing the
155 * read: the firmware pads the reply out to it.
156 *
157 * Entry: B = the unit, X = where to put it, Y = how many bytes
158 * Exit:  Z set when every byte arrived; X preserved
159 * -----
160 NTGRSP    PSHS     X,Y
161         STB      NTRSPF+1
162         TFR      Y,D
163         STD      NTRSPF+3
164         JSR      FNWAIT
165         LDX      #NTRSPF
166         LDY      #5
167         JSR      [DWWRITV]
168         PULS     X,Y
169         JMP      [DWREADV]
170
171 * -----
172 * NTUNIT --- read the unit number out of a devicespec.
173 *
174 * "N:" is unit 1. "N2:" is unit 2, and so on to "N8:".
175 *
176 * Entry: X = the devicespec
177 * Exit:  B = the unit; A and X clobbered
178 * -----
179 NTUNIT    LDB      1,X          the character after the N
180         CMPB     #'.'
181         BEQ      NTU1          plain "N:" --- unit 1
182         SUBB     #'0'
183         CMPB     #1
184         BLO      NTU1          garbage --- use unit 1
185         CMPB     #8
186         BHI      NTU1
187         RTS
188 NTU1      LDB      #1
189         RTS
190
191 * -----
192 * DWTIME --- read the adapter's clock the short way.
193 *
194 * OP_TIME is answered by the bus itself, so there is no command
195 * byte, no error byte and no response fetch: one byte out, six
196 * bytes back.
197 *
198 * Entry: X = a six-byte buffer
199 * Exit:  buffer holds year-1900, month, day, hour, minute,
200 *        second; Z set when all six arrived
201 * -----
202 DWTIME    PSHS     X
203         JSR      FNWAIT
204         LDX      #DWTFF
205         LDY      #1
206         JSR      [DWWRITV]
207         PULS     X
208         LDY      #6
209         JMP      [DWREADV]
210
211 * -----
212 * the frames these routines keep to themselves

```

```

213 * -----
214 FNDRYF FCB OPFUJI,FNREADY
215 FNERRF FCB OPFUJI,FNERROR
216 FNRSPP FCB OPFUJI,FNRESP
217 FNC0F FCB OPFUJI,0
218 FNC1F FCB OPFUJI,0,0
219 NTERRF FCB OPNET,0,FNERROR,0,0
220 NTRSPF FCB OPNET,0,FNRESP,0,0
221 DWTF FCB OPTIME
222 FNSCRP RMB 1

```

fnnet.asm --- the five moves of the N: device

```

1 *****
2 * FNNET.ASM --- the five moves of the N: device in 6809
3 *
4 * Open, close, status, read, write. Every network program in
5 * this book is these five in some order. INCLUDE FN.INC and
6 * FNLOW.ASM before this file.
7 *
8 * One channel at a time: NTUNI remembers the unit that NTOPE
9 * parsed out of the devicespec, and the other four use it. To
10 * run two channels at once, keep your own unit bytes and call
11 * NTXACT, NTGERR and NTGRSP directly.
12 *****
13
14 * -----
15 * NTOPE --- instantiate a protocol and connect.
16 *
17 * Entry: X = the devicespec, NUL-terminated, under 256 bytes
18 *        A = the open mode (OMREAD, OMWRITE, OMRW, OMDIR, ...)
19 *        B = the translation mode (TRNONE, TRCR, ...)
20 * Exit: A = the error byte, Z set on success; NTUNI is set
21 *
22 NTOPE STA NTOFRM+3      aux1 is the open mode
23       STB NTOFRM+4      aux2 is the translation mode
24       PSHS X
25       JSR NTUNIT        B = the unit named in the spec
26       STB NTOFRM+1
27       STB NTUNI
28       LDX #NTOSPC       the spec field is a fixed 256
29       LDY #SZSPEC       bytes and must be NUL-padded
30
31 NT01  CLR ,X+
32       LEAY -1,Y
33       BNE NT01
34       PULS X
35
36 NT02  LDY #NTOSPC
37       LDA ,X+
38       STA ,Y+
39       BNE NT02
40       LDX #NTOFRM
41       LDY #NTOLEN
42       LDB NTUNI
43       JMP NTXACT
44
45 * -----
46 * NTCLOS --- close the channel and destroy the protocol.
47 *
48 * Exit: A = the error byte, Z set on success
49 *
50 NTCLOS LDB NTUNI
51        STB NTCFRM+1
52        LDX #NTCFRM
53        LDY #5
54        JMP NTXACT

```

```

55 * NTSTAT --- how much is waiting, and are we still connected?
56 *
57 * Entry: X = a four-byte buffer
58 * Exit: X+0,X+1 bytes waiting, high byte first
59 *        X+2      1 while the connection stands
60 *        X+3      the channel's own error byte
61 *        A = the error byte from the STATUS command itself
62 * -----
63 NTSTAT PSHS X
64        LDB NTUNI
65        STB NTSFRM+1
66        LDX #NTSFRM
67        LDY #5
68        JSR NTXACT
69        PULS X
70        LDB NTUNI
71        LDY #4
72        JMP NTGRSP
73
74 * -----
75 * NTREAD --- take bytes out of the channel.
76 *
77 * Ask for no more than NTSTAT said was waiting. Asking for more
78 * makes the adapter pad the reply with zeros.
79 *
80 * Entry: X = buffer, Y = count
81 * Exit: Z set when every byte arrived
82 *
83 NTREAD PSHS X,Y
84        LDB NTUNI
85        STB NTRFRM+1
86        TFR Y,D          the count rides in the aux word,
87        STD NTRFRM+3      high byte first
88        LDX #NTRFRM
89        LDY #5
90        LDB NTUNI
91        JSR NTXACT
92        PULS X,Y
93        LDB NTUNI
94        JMP NTGRSP
95
96 * -----
97 * NTWRIT --- put bytes into the channel.
98 *
99 * The header goes out first, then the payload as a second
100 * DriveWire write. The adapter is already counting.
101 *
102 * Entry: X = buffer, Y = count
103 * Exit: A = the error byte, Z set on success
104 * -----
105 NTWRIT PSHS X,Y
106        LDB NTUNI
107        STB NTWFRM+1
108        TFR Y,D

```

```

109     STD     NTWFRM+3
110     LDX     #NTWFRM
111     LDY     #5
112     JSR     FNWAIT
113     JSR     [DWWRTIV]      the header
114     PULS    X,Y
115     JSR     [DWWRTIV]      the payload
116     LDB     NTUNI
117     JMP     NTGERR
118
119 * -----
120 * the frames, and the unit they all share
121 * -----
122 NTOFRM  FCB     OPNET,0,NCOPEN,0,0
123 NTOSPC  RMB     SZSPEC
124 NTOLEN  EQU     *-NTOFRM
125 NTCFRM  FCB     OPNET,0,NCCLOSE,0,0
126 NTSFRM  FCB     OPNET,0,NCSTAT,0,0
127 NTRFRM  FCB     OPNET,0,NCREAD,0,0
128 NTWFRM  FCB     OPNET,0,NCWRITE,0,0
129 NTUNI   FCB     1

```

C.3 The console

cocoio.asm --- screen and keyboard

```

1 *****
2 * COCOIO.ASM --- screen and keyboard, the plain way
3 *
4 * Color BASIC leaves two vectors in RAM for us.  POLCAT at
5 * $A000 returns the key that is down, or zero.  CHROUT at $A002
6 * prints the character in A.
7 *
8 * The ROM routines are documented as preserving everything but
9 * A, and mostly they do.  OUTCH saves the rest anyway: a
10 * corrupted X halfway through a string is a long evening.
11 *****
12
13 POLCAT  EQU     $A000      keyboard poll vector
14 CHROUT  EQU     $A002      console out vector
15
16 FF      EQU     12        CHROUT clears the screen on this
17 CR      EQU     13
18 BS      EQU     8
19 BREAK   EQU     3
20
21 * -----
22 * OUTCH --- print A.  Everything else comes back untouched.
23 * -----
24 OUTCH   PSHS     B,X,Y,U
25         JSR     [CHROUT]
26         PULS    B,X,Y,U,PC
27
28 * -----
29 * CLRSCR --- clear the screen
30 * -----
31 CLRSCR  LDA     #FF
32         BRA     OUTCH
33
34 * -----
35 * CRLF --- end the line
36 * -----
37 CRLF    LDA     #CR
38         BRA     OUTCH
39
40 * -----

```

```

41 * PUTS --- print the NUL-terminated string at X.  X is kept.
42 * -----
43 PUTS    PSHS     A,X
44 PUTS1   LDA      ,X+
45         BEQ     PUTS9
46         JSR     OUTCH
47         BRA     PUTS1
48 PUTS9   PULS     A,X,PC
49
50 * -----
51 * PUTBUF --- print D bytes starting at X, controls and all.
52 * -----
53 PUTBUF  PSHS     D,X
54         TSTA
55         BNE     PUTB1
56         TSTB
57         BEQ     PUTB9
58 PUTB1   LDA      ,X+
59         JSR     OUTCH
60         SUBD    #1
61         BNE     PUTB1
62 PUTB9   PULS     D,X,PC
63
64 * -----
65 * PUTDEC --- print A as one to three decimal digits
66 * -----
67 PUTDEC  PSHS     A,B
68         CLRB
69 PD100   CMPA     #100
70         BLO     PD100X
71         SUBA    #100
72         INCB
73         BRA     PD100
74 PD100X  TSTB
75         BEQ     PD10
76         PSHS    A
77         TFR     B,A
78         ADDA    #'0'
79         JSR     OUTCH
80         PULS    A

```

```

81 PD10 CLRB
82 PD10L CMPA #10
83 BLO PD10X
84 SUBA #10
85 INCB
86 BRA PD10L
87 PD10X PSHS A
88 TFR B,A
89 ADDA #'0'
90 JSR OUTCH
91 PULS A
92 ADDA #'0'
93 JSR OUTCH
94 PULS A,B,PC
95
96 * -----
97 * PUTHEX --- print A as two hexadecimal digits
98 * -----
99 PUTHEX PSHS A
100 LSRA
101 LSRA
102 LSRA
103 LSRA
104 JSR PUTNIB
105 PULS A
106 PSHS A
107 ANDA #$0F
108 JSR PUTNIB
109 PULS A,PC
110 PUTNIB ANDA #$0F
111 CMPA #9
112 BLS PUTN1
113 ADDA #7
114 PUTN1 ADDA #'0'
115 JMP OUTCH
116
117 * -----
118 * GETLIN --- read a line into the buffer at X, echoing as it
119 * goes. BACKSPACE rubs out. ENTER ends it and a
120 * terminating NUL is written. BREAK gives an empty
121 * line and leaves the carry set.
122 * -----
123 GETLIN PSHS A,X,U
124 LEAU ,X
125 GL1 JSR [POLCAT]
126 TSTA
127 BEQ GL1
128 CMPA #CR
129 BEQ GL9
130 CMPA #BREAK
131 BEQ GL8
132 CMPA #BS
133 BNE GL2
134 CMPU 1,S anything left to rub out?
135 BEQ GL1
136 LEAU -1,U
137 JSR OUTCH
138 BRA GL1
139 GL2 CMPA #' '
140 BLO GL1 ignore the other control keys
141 STA ,U+
142 JSR OUTCH
143 BRA GL1
144 GL8 LDU 1,S BREAK --- give back nothing
145 CLR ,U
146 JSR CRLF
147 PULS A,X,U
148 ORCC #1 carry set: the user gave up
149 RTS
150 GL9 CLR ,U
151 JSR CRLF

```

```

152 PULS A,X,U
153 ANDCC #$FE carry clear: a line was read
154 RTS
155
156 * -----
157 * SCOPY --- copy the NUL-terminated string at X to Y, leaving Y
158 * on the terminator so the next copy runs straight on.
159 * -----
160 SCOPY PSHS A,X
161 SCOPY1 LDA ,X+
162 BEQ SCOPY9
163 STA ,Y+
164 BRA SCOPY1
165 SCOPY9 CLR ,Y
166 PULS A,X,PC

```

C.4 First contact

fcdemo.asm

```
1 *****
2 * FCDEMO.ASM --- First Contact
3 *
4 * Asks the adapter who it is, and prints the answer. Three
5 * commands' worth of code, and the whole shape of every other
6 * command is in it.
7 *
8 * lwasn --6809 --format=decB -o FCDEMO.BIN fcdemo.asm
9 * LOADM"FCDEMO":EXEC
10 *****
11
12     ORG     $3F00
13
14     INCLUDE "fn.inc"
15
16 START JSR     CLRSCR
17       LDX     #MSIGN
18       JSR     PUTS
19
20 * --- ask for the extended adapter configuration -----
21       LDA     #FCADCFX
22       JSR     FNCMD0      $E2 $C4, then collect the verdict
23       BNE     FAILED      A came back as something else
24
25       LDX     #CFG
26       LDY     #SZADCFX
27       JSR     FNGRSP      240 bytes land in CFG
28       BNE     FAILED
29
30 * --- and print the three fields worth having -----
31       LDX     #MSSID
32       JSR     PUTS
33       LDX     #CFG+0      ssid[33]
34       JSR     PUTS
35       JSR     CRLF
36
37       LDX     #MADDR
38       JSR     PUTS
39       LDX     #CFG+140     sLocalIP[16] --- already dotted
40       JSR     PUTS
41       JSR     CRLF
42
43       LDX     #MFIRM
44       JSR     PUTS
45       LDX     #CFG+125     fn_version[15]
46       JSR     PUTS
47       JSR     CRLF
48       RTS
49
50 FAILED PSHS     A
51       LDX     #MFAIL
52       JSR     PUTS
53       PULS     A
54       JSR     PUTDEC
55       JMP     CRLF
56
57 MSIGN  FCC      "FUJINET FIRST CONTACT"
58       FCB      CR,0
59 MSSID  FCC      "NETWORK.. "
60       FCB      0
61 MADDR  FCC      "ADDRESS.. "
62       FCB      0
63 MFIRM  FCC      "FIRMWARE. "
64
65 MFAIL  FCC      "NO ANSWER, ERROR "
66       FCB      0
67
68       INCLUDE "cocoio.asm"
69       INCLUDE "fnlow.asm"
70
71 CFG    RMB      SZADCFX
72
73       END      START
```

fcdemo.c

```
1  /*
2  * FCDEMO.C --- First Contact
3  *
4  * The same program as FCDEMO.ASM, in C.  fujinet-lib does the
5  * frame building; what is left is the part you came for.
6  *
7  *   cmoc -o FCDEMO.BIN fcdemo.c -lfujinet
8  *   LOADM"FCDEMO":EXEC
9  */
10
11 #include <cmoc.h>
12 #include <coco.h>
13 #include <fujinet-fuji.h>
14
15 int main(void)
16 {
17     AdapterConfigExtended ac;
18
19     putchar(12);
20     printf("FUJINET FIRST CONTACT\n");
21
22     if (!fuji_get_adapter_config_extended(&ac))
23     {
24         printf("NO ANSWER\n");
25         return 1;
26     }
27
28     printf("NETWORK.. %s\n", ac.ssid);
29     printf("ADDRESS.. %s\n", ac.sLocalIP);  /* already dotted text */
30     printf("FIRMWARE. %s\n", ac.fn_version);
31
32     return 0;
33 }
```

C.5 Netcat

netcat.asm

```
1  *****
2  * NETCAT.ASM --- a wire, in both directions
3  *
4  * Opens a TCP channel, sends what you type, prints what comes
5  * back.  BREAK hangs up.  It is the smallest program that uses
6  * all five moves of the N: device, which is why every one of
7  * these handbooks ends with it.
8  *
9  *   lwasm --6809 --format=decB -I../fnlib -o NETCAT.BIN netcat.asm
10 *   LOADM"NETCAT":EXEC
11 *
12 * Somewhere to point it:
13 *   HOST? fujinet.online   PORT? 6502
14 *****
15
16 RXMAX  EQU      200          how much we take in one gulp
17
18          ORG      $3F00
19
20          INCLUDE  "fn.inc"
21
22 START   JSR      CLRSCR
23          LDX      #MSIGN
24          JSR      PUTS
25
26 * --- ask where to go -----
27          LDX      #MHOST
28          JSR      PUTS
29          LDX      #HOSTB
30          JSR      GETLIN
31          LBCS     QUIT
32          LDX      #MPORT
33          JSR      PUTS
34          LDX      #PORTB
35          JSR      GETLIN
36          LBCS     QUIT
37
38 * --- build "N:TCP://host:port/" -----
39          LDY      #SPEC
40          LDX      #MPFX
41          JSR      SCOPY
42          LDX      #HOSTB
43          JSR      SCOPY
44          LDA      #' ':' '
45          STA      ,Y+
46          LDX      #PORTB
```

```

47      JSR      SCOPY
48
49  * --- open it -----
50      LDX      #MOPEN
51      JSR      PUTS
52      LDX      #SPEC
53      LDA      #OMRW          read and write
54      LDB      #TRNONE       leave the bytes alone
55      JSR      NTOPEN
56      BNE      BADOPN
57      LDX      #MREADY
58      JSR      PUTS
59
60  * --- the loop -----
61  LOOP JSR      [POLCAT]      anything typed?
62      TSTA
63      BEQ      LRECV
64      CMPA     #BREAK
65      BEQ      BYE
66      STA      KEYB
67      LDX      #KEYB
68      LDY      #1
69      JSR      NTWRIT        one byte, up the wire
70      BNE      BADIO
71
72  LRECV LDX      #STBUF
73      JSR      NTSTAT
74      BNE      BADIO
75      LDD      STBUF        bytes waiting, high byte first
76      BEQ      LCONN
77      CMPD     #RXMAX
78      BLS      LREAD
79      LDD      #RXMAX        take only what we can hold
80  LREAD PSHS     D
81      LDX      #RXBUF
82      TFR      D,Y
83      JSR      NTREAD
84      PULS     D
85      LDX      #RXBUF
86      JSR      PUTBUF
87      BRA      LOOP
88
89  LCONN LDA      STBUF+2      still connected?
90      BNE      LOOP
91      LDX      #MDROP
92      JSR      PUTS
93
94  BYE   JSR      NTCLOS
95      LDX      #MBYE
96      JSR      PUTS
97  QUIT RTS
98
99  BADOPN PSHS     A
100      LDX      #MNOOPN
101      JSR      PUTS
102      BRA      BADX
103  BADIO PSHS     A
104      LDX      #MNOIO
105      JSR      PUTS
106  BADX  PULS     A
107      JSR      PUTDEC
108      JSR      CRLF
109      JSR      NTCLOS
110      RTS
111
112  MSIGN FCC      "FUJINET NETCAT"
113      FCB      CR,0
114  MHOST FCC      "HOST? "
115      FCB      0
116  MPORT FCC      "PORT? "
117      FCB      0

```

```

118  MOPEN FCC      "OPENING..."
119      FCB      CR,0
120  MREADY FCC      "CONNECTED. BREAK HANGS UP."
121      FCB      CR,0
122  MDROP  FCB      CR
123      FCC      "THE OTHER END CLOSED."
124      FCB      CR,0
125  Mbye   FCC      "CHANNEL CLOSED."
126      FCB      CR,0
127  MNOOPN FCC      "COULD NOT OPEN, ERROR "
128      FCB      0
129  MNOIO  FCB      CR
130      FCC      "CHANNEL ERROR "
131      FCB      0
132  MPFX   FCC      "N:TCP://"
133      FCB      0
134
135      INCLUDE "cocoio.asm"
136      INCLUDE "fnlow.asm"
137      INCLUDE "fnnet.asm"
138
139  KEYB   RMB      1
140  STBUF  RMB      4
141  HOSTB  RMB      64
142  PORTB  RMB      8
143  SPEC   RMB      SZSPEC
144  RXBUF  RMB      RXMAX
145
146      END      START

```

netcat.c

```
1  /*
2  * NETCAT.C --- a wire, in both directions
3  *
4  * The same program as NETCAT.ASM.  fujinet-lib builds the
5  * frames; the five moves are still plainly visible.
6  *
7  * cmoc -I$(FNDIR) -o NETCAT.BIN netcat.c -lfujinet
8  * LOADM"NETCAT":EXEC
9  */
10
11 #include <cmoc.h>
12 #include <coco.h>
13 #include <fujinet-fuji.h>
14 #include <fujinet-network.h>
15
16 #define RXMAX 200          /* how much we take in one gulp */
17 #define KBREAK 3
18 #define KENTER 13
19 #define KBS 8
20
21 static char spec[256];
22 static char host[64];
23 static char port[8];
24 static uint8_t rx[RXMAX];
25
26 /* Read a line, echoing as it goes.  BACKSPACE rubs out. */
27 static void getline_(char *buf, byte max)
28 {
29     byte n = 0, k;
30
31     for (;;)
32     {
33         k = waitkey(1);
34         if (k == KENTER)
35             break;
36         if (k == KBS)
37         {
38             if (n) { n--; putchar(KBS); }
39             continue;
40         }
41         if (k < ' ' || n >= max - 1)
42             continue;
43         buf[n++] = k;
44         putchar(k);
45     }
46     buf[n] = '\0';
47     putchar('\n');
48 }
49
50 int main(void)
51 {
52     uint16_t bw, i;
53     uint8_t conn, err, k;
54     int16_t n;
55
56     cls(1);
57     printf("FUJINET NETCAT\n");
58
59     printf("HOST? "); getline_(host, sizeof(host));
60     printf("PORT? "); getline_(port, sizeof(port));
61
62     strcpy(spec, "N:TCP://");
63     strcat(spec, host);
64     strcat(spec, ":");
65     strcat(spec, port);
66
67     printf("OPENING...\n");
68     if (network_open(spec, OPEN_MODE_RW, OPEN_TRANS_NONE) != FN_ERR_OK)
69     {
70         printf("COULD NOT OPEN, ERROR %u\n", fn_device_error);
71         return 1;
72     }
73     printf("CONNECTED.  BREAK HANGS UP.\n");
74
75     for (;;)
76     {
77         k = inkey();          /* anything typed? */
78         if (k == KBREAK)
79             break;
80         if (k)
81             if (network_write(spec, &k, 1) != FN_ERR_OK)
82                 goto ioerr;
83
84         if (network_status(spec, &bw, &conn, &err) != FN_ERR_OK)
85             goto ioerr;
86
87         if (bw)
88         {
89             if (bw > RXMAX)
90                 bw = RXMAX;          /* only what we can hold */
91             n = network_read(spec, rx, bw);
92             if (n < 0)
93                 goto ioerr;
94             for (i = 0; i < (uint16_t) n; i++)
95                 putchar(rx[i]);
96         }
97         else if (!conn)
98         {
99             printf("\nTHE OTHER END CLOSED.\n");
100             break;
101         }
102     }
103
104     network_close(spec);
105     printf("CHANNEL CLOSED.\n");
106     return 0;
107
108 ioerr:
109     printf("\nCHANNEL ERROR %u\n", fn_device_error);
110     network_close(spec);
111     return 1;
112 }
```

C.6 Mounter

mounter.asm

```

1 *****
2 * MOUNTER.ASM --- host slots, a directory, and a disk in a drive
3 *
4 * The mounting sequence by hand: read the host slots, mount
5 * one, open its directory, read it, put a file into a device
6 * slot, mount it. What CONFIG does, with the menus taken off.
7 *
8 * lwasmb --6809 --format=decb -I../fnlib -o MOUNTER.BIN mounter.asm
9 * LOADM"MOUNTER":EXEC
10 *****
11
12 MAXENT EQU 16 as many names as we will hold
13 NAMELEN EQU 37
14 NHOSTS EQU 8
15 NDRIVES EQU 4 the CoCo has four, not eight
16
17 ORG $3F00
18
19 INCLUDE "fn.inc"
20
21 START JSR CLRSCR
22 LDX #MSIGN
23 JSR PUTS
24
25 * --- the eight host slots -----
26 LDA #FCRDHS
27 JSR FNCMD0
28 LBNE ENOHS
29 LDX #HOSTS
30 LDY #SZHOSTS 8 slots, 32 bytes each
31 JSR FNGRSP
32 LBNE ENOHS
33
34 SHOW1 CLR IDX
35 LDA IDX
36 INCA
37 JSR PUTDEC
38 LDA #' '
39 JSR OUTCH
40 JSR HSPTR X = HOSTS + 32*IDX
41 LDA ,X
42 BNE SHOW2
43 LDX #MEMPTY nothing in this slot
44 SHOW2 JSR PUTS
45 JSR CRLF
46 INC IDX
47 LDA IDX
48 CMPA #NHOSTS
49 BLO SHOW1
50
51 * --- pick one -----
52 LDX #MHOST
53 LDA #1
54 LDB #NHOSTS
55 JSR ASKDIG
56 TSTA
57 LBEQ QUIT
58 DECA
59 STA HS
60
61 * --- mount it and open its root -----
62 LDA #FCMNTHS
63 LDB HS
64 JSR FNCMD1 $E2 $F9 slot
65 LBNE ENOMH
66
67 * the path field is one 256-byte block holding
68 * "/" NUL "*.dsk" NUL --- path, then pattern
69 LDX #ODPATH
70 LDY #SZSPEC
71 ODCLR CLR ,X+
72 LEAY -1,Y
73 BNE ODCLR
74 LDY #ODPATH
75 LDX #MROOT
76 JSR SCOPY
77 LEAY 1,Y step over the NUL
78 LDX #MFILT
79 JSR SCOPY
80
81 LDA HS
82 STA ODFRM+2
83 LDX #ODFRM
84 LDY #ODLEN
85 JSR FNAXCT
86 LBNE ENODIR
87
88 * --- read it -----
89 CLR NENT
90 LDU #NAMES
91 DIR1 LDX #RDFRM $E2 $F6 maxlen aux2 --- two
92 LDY #4 parameters, so FNCMD1 will not do
93 JSR FNAXCT
94 BNE DIR9
95 LDX #ENTRY
96 LDY #NAMELEN-1
97 JSR FNGRSP
98 BNE DIR9
99 LDA ENTRY
100 CMPA #$7F two $7F bytes end the directory
101 BEQ DIR9
102 LDA NENT
103 INCA
104 JSR PUTDEC
105 LDA #' '
106 JSR OUTCH
107 LDX #ENTRY
108 JSR PUTS
109 JSR CRLF
110 LDX #ENTRY
111 LDY #NAMELEN
112 CPY1 LDA ,X+
113 STA ,U+
114 BNE CPY2
115 LEAX -1,X keep filling with NULs
116 CPY2 LEAY -1,Y
117 BNE CPY1
118 INC NENT
119 LDA NENT
120 CMPA #MAXENT
121 BLO DIR1
122
123 DIR9 LDA #FCCLSDR
124 JSR FNCMD0
125 LDA NENT
126 LBEQ ENONE

```

```

127
128 * --- pick a file and a drive -----
129     LDX     #MFILE
130     LDA     #1
131     LDB     NENT
132     CMPB    #9
133     BLS     ASK1
134     LDB     #9
135     JSR     ASKDIG
136     TSTA
137     LBEQ    QUIT
138     DECA
139     STA     IDX
140
141     LDX     #MDRIVE
142     LDA     #1
143     LDB     #NDRIVES
144     JSR     ASKDIG
145     TSTA
146     LBEQ    QUIT
147     DECA
148     STA     DS
149
150 * --- write the name into the slot -----
151 *     mode 0 stores the name without opening anything
152     LDX     #SDNAME
153     LDY     #SZSPEC
154     SDCLR   CLR     ,X+
155             LEAY    -1,Y
156             BNE     SDCLR
157             LDA     #NAMELEN
158             LDB     IDX
159             MUL
160             LDX     #NAMES
161             LEAX    D,X
162             LDY     #SDNAME
163             JSR     SCOPY
164
165             LDA     DS
166             STA     SDFRM+2
167             LDA     HS
168             STA     SDFRM+3
169             CLR     SDFRM+4
170             LDY     #SDFRM
171             LDY     #SDLEN
172             JSR     FNXACT
173             LBNE    ENOSET
174
175 * --- and mount it -----
176 *     1 is read-only, 2 is read and write
177     LDA     DS
178     STA     MIFRM+2
179     LDA     #2
180     STA     MIFRM+3
181     LDX     #MIFRM
182     LDY     #4
183     JSR     FNXACT
184     LBNE    ENOMNT
185
186     JSR     CRLF
187     LDX     #MOK
188     JSR     PUTS
189     LDA     DS
190     JSR     PUTDEC
191     JSR     CRLF
192     QUIT    RTS
193
194     ENOHS   LDX     #MENOHS
195             BRA     FAIL
196     ENOMH   LDX     #MENOMH
197             BRA     FAIL

```

```

198     ENODIR  LDX     #MENODR
199             BRA     FAIL
200     ENOSET  LDX     #MENOST
201             BRA     FAIL
202     ENOMNT  LDX     #MENOMT
203             BRA     FAIL
204     ENONE   LDX     #MENONE
205             JSR     PUTS
206             RTS
207     FAIL    PSHS    A
208             JSR     PUTS
209             PULS    A
210             JSR     PUTDEC
211             JMP     CRLF
212
213 * -----
214 * ASKDIG --- prompt with the string at X and take one digit
215 *           between A and B. Returns A = the digit, or zero
216 *           if BREAK was pressed.
217 * -----
218     ASKDIG  PSHS    B,X
219             STA     ADLO
220             STB     ADHI
221     AD1     LDX     1,S
222             JSR     PUTS
223     AD2     JSR     [POLCAT]
224             TSTA
225             BEQ     AD2
226             CMPA    #BREAK
227             BEQ     AD8
228             JSR     OUTCH
229             JSR     CRLF
230             SUBA    #'0'
231             CMPA    ADLO
232             BLO     AD1
233             CMPA    ADHI
234             BHI     AD1
235             PULS    B,X,PC
236     AD8     CLRA
237             JSR     CRLF
238             PULS    B,X,PC
239
240 * -----
241 * HSPTR --- X = HOSTS + 32*IDX
242 * -----
243     HSPTR   PSHS    D
244             LDA     #32
245             LDB     IDX
246             MUL
247             LDX     #HOSTS
248             LEAX    D,X
249             PULS    D,PC
250
251     MSIGN   FCC     "FUJINET MOUNTER"
252             FCB     CR,CR,0
253     MEMPTY  FCC     "(EMPTY)"
254             FCB     0
255     MHOST   FCB     CR
256             FCC     "HOST? "
257             FCB     0
258     MFILE   FCB     CR
259             FCC     "FILE? "
260             FCB     0
261     MDRIVE  FCC     "DRIVE (1-4)? "
262             FCB     0
263     MROOT   FCC     "/"
264             FCB     0
265     MFILT   FCC     "*.dsk"
266             FCB     0
267     MOK      FCC     "READY ON DRIVE "
268             FCB     0

```

```

269 MENOHS FCC "CANNOT READ HOST SLOTS, ERROR "
270 FCB 0
271 MENOMH FCC "CANNOT MOUNT THAT HOST, ERROR "
272 FCB 0
273 MENODR FCC "CANNOT OPEN DIRECTORY, ERROR "
274 FCB 0
275 MENOST FCC "CANNOT SET THE SLOT, ERROR "
276 FCB 0
277 MENOMT FCC "CANNOT MOUNT IT, ERROR "
278 FCB 0
279 MENONE FCC "NOTHING THERE."
280 FCB CR,0
281
282 * --- the frames with payloads -----
283 ODFRM FCB OPFUJI,FCOPNDR,0
284 ODPATH RMB SZSPEC
285 ODLEN EQU *-ODFRM
286 SDFRM FCB OPFUJI,FCSDVFP,0,0,0
287 SDNAME RMB SZSPEC
288 SDLEN EQU *-SDFRM
289 MIFRM FCB OPFUJI,FCMNTIM,0,0
290 RDFRM FCB OPFUJI,FCRDDIR,NAMELEN-1,0
291
292 INCLUDE "cocoio.asm"
293 INCLUDE "fnlow.asm"
294
295 ADLO RMB 1
296 ADHI RMB 1
297 IDX RMB 1
298 HS RMB 1
299 DS RMB 1
300 NENT RMB 1
301 ENTRY RMB NAMELEN
302 HOSTS RMB SZHOSTS
303 NAMES RMB MAXENT*NAMELEN
304
305 END START

```

mounter.c

```

1 /*
2  * MOUNTER.C --- host slots, a directory, and a disk in a drive
3  *
4  * Walks the whole mounting sequence by hand: read the host
5  * slots, mount one, open its directory, read it, put a file
6  * into a device slot, mount it. This is what CONFIG does,
7  * with the menus taken off.
8  *
9  * cmoc -I$(FNDIR) -o MOUNTER.BIN mounter.c -lfujinet
10 * LOADM"MOUNTER":EXEC
11 */
12
13 #include <cmoc.h>
14 #include <coco.h>
15 #include <fujinet-fuji.h>
16
17 #define MAXENT 40 /* as many names as we will hold */
18 #define NAMELEN 37
19 #define NHOSTS 8
20 #define NDRIVES 4 /* the CoCo has four, not eight */
21
22 static HostSlot hosts[NHOSTS];
23 static DeviceSlot drives[NDRIVES];
24 static char names[MAXENT][NAMELEN];
25 static char entry[NAMELEN];
26 static char root[] = "/";
27 static char filter[] = "*.disk";

```

```

28 static char empty[] = "(EMPTY)";
29
30 /* Ask for a number between lo and hi. Returns 0 if BREAK. */
31 static byte ask(const char *prompt, byte lo, byte hi)
32 {
33     byte k;
34
35     for (;;)
36     {
37         printf("%s", prompt);
38         k = waitkey(1);
39         if (k == 3) { printf("\n"); return 0; }
40         putchar(k);
41         printf("\n");
42         if (k >= '0' + lo && k <= '0' + hi)
43             return k - '0';
44         printf("BETWEEN %u AND %u, PLEASE.\n", lo, hi);
45     }
46 }
47
48 int main(void)
49 {
50     byte i, hs, ds, n;
51
52     cls(1);
53     printf("FUJINET MOUNTER\n\n");
54

```

```

55  /* ---- the eight host slots ----- */
56  if (!fuji_get_host_slots((uint8_t *) hosts, NHOSTS))
57  {
58      printf("CANNOT READ HOST SLOTS\n");
59      return 1;
60  }
61  for (i = 0; i < NHOSTS; i++)
62      printf("%u %s\n", i + 1, hosts[i][0] ? (char *) hosts[i] :
empty);
63
64  hs = ask("\nHOST? ", 1, NHOSTS);
65  if (!hs)
66      return 0;
67  if (!hosts[hs - 1][0])
68  {
69      printf("THAT SLOT IS EMPTY.\n");
70      return 1;
71  }
72
73  /* ---- mount it and read its root ----- */
74  if (!fuji_mount_host_slot(hs - 1))
75  {
76      printf("CANNOT MOUNT THAT HOST\n");
77      return 1;
78  }
79  if (!fuji_open_directory2(hs - 1, root, filter))
80  {
81      printf("CANNOT OPEN THE DIRECTORY\n");
82      return 1;
83  }
84
85  n = 0;
86  while (n < MAXENT)
87  {
88      if (!fuji_read_directory(NAMELEN - 1, 0, entry))
89          break;
90      if ((byte) entry[0] == 0x7F) /* two $7F bytes end it */
91          break;
92      strcpy(names[n], entry);
93      printf("%u %s\n", n + 1, names[n]);
94      n++;
95  }
96  fuji_close_directory();
97
98  if (!n)
99  {
100      printf("NOTHING THERE.\n");
101      return 1;
102  }
103
104  /* ---- pick a file and a drive ----- */
105  i = ask("\nFILE? ", 1, n > 9 ? 9 : n);
106  if (!i)
107      return 0;
108  ds = ask("DRIVE (1-4)? ", 1, NDRIVES);
109  if (!ds)
110      return 0;
111
112  /* mode 0 writes the name into the slot without opening it */
113  if (!fuji_set_device_filename(0, hs - 1, ds - 1, names[i - 1]))
114  {
115      printf("CANNOT SET THE SLOT\n");
116      return 1;
117  }
118  /* 1 is read-only, 2 is read and write */
119  if (!fuji_mount_disk_image(ds - 1, 2))
120  {
121      printf("CANNOT MOUNT IT\n");
122      return 1;
123  }
124

```

```

125  /* ---- show the drives as they now stand ----- */
126  printf("\n");
127  if (fuji_get_device_slots(drives, NDRIVES))
128      for (i = 0; i < NDRIVES; i++)
129          printf("%u%c %s\n", i,
130              (drives[i].mode & 0x40) ? '*' : ' ',
131              drives[i].file[0] ? (char *) drives[i].file : empty);
132
133  printf("\nDRIVE %u IS READY.\n", ds - 1);
134  return 0;
135  }

```

C.7 Weather

weather.asm

```
1 *****
2 * WEATHER.ASM --- a JSON channel, end to end
3 *
4 * Opens an HTTPS channel, puts the JSON parser on it, and asks
5 * three questions of the document. The adapter keeps the
6 * document; the CoCo only ever holds the answer.
7 *
8 * lwasmm --6809 --format=decB -I../fnlib -o WEATHER.BIN weather.asm
9 * LOADM"WEATHER":EXEC
10 *****
11
12 ANSMAX EQU 64
13
14 ORG $3F00
15
16 INCLUDE "fn.inc"
17
18 START JSR CLRSCR
19 LDX #MSIGN
20 JSR PUTS
21
22 * --- open the channel -----
23 LDX #SPEC
24 LDA #OMHGET HTTP GET
25 LDB #TRNONE
26 JSR NTOPE
27 LBNE ENOOPN
28
29 * --- put the JSON parser on it -----
30 * The mode rides in aux2. fujinet-lib puts it in aux1,
31 * which is why its network_json_parse() quietly selects
32 * no parser at all on a current adapter.
33 LDA NTUNI
34 STA SPFRM+1
35 LDX #SPFRM
36 LDY #5
37 LDB NTUNI
38 JSR NTXACT
39 LBNE ENOPAR
40
41 * --- and parse what is waiting -----
42 LDA NTUNI
43 STA PAFRM+1
44 LDX #PAFRM
45 LDY #5
46 LDB NTUNI
47 JSR NTXACT
48 LBNE ENOPAR
49
50 * --- three questions -----
51 LDX #MTIME
52 LDY #QTIME
53 JSR ASK
54 LDX #MTEMP
55 LDY #QTEMP
56 JSR ASK
57 LDX #MWIND
58 LDY #QWIND
59 JSR ASK
60
61 JSR NTCLOS
62 RTS
63
64 ENOOPN LDX #MENOOP
65 BRA FAIL
66 ENOPAR LDX #MENOPA
67 FAIL PSHS A
68 JSR PUTS
69 PULS A
70 JSR PUTDEC
71 JSR CRLF
72 JSR NTCLOS
73 RTS
74
75 * -----
76 * ASK --- print the label at X, then the answer to the query at Y
77 *
78 * A query sets the channel's contents to the value it selected.
79 * STATUS then says how many bytes that is, and READ takes them.
80 * -----
81 ASK PSHS X,Y
82 JSR PUTS the label
83 * copy the query into the frame's 256-byte field
84 LDX #QYSTR
85 LDY #SZSPEC
86 ASKCLR CLR ,X+
87 LEAY -1,Y
88 BNE ASKCLR
89 LDX 2,S the query string
90 LDY #QYSTR
91 JSR SCOPY
92 LDA NTUNI
93 STA QYFRM+1
94 LDX #QYFRM
95 LDY #QYLEN
96 LDB NTUNI
97 JSR NTXACT
98 BNE ASKBAD
99 * how much did it select?
100 LDX #STBUF
101 JSR NTSTAT
102 BNE ASKBAD
103 LDD STBUF
104 BEQ ASKBAD
105 CMPD #ANSMAX
106 BLS ASKRD
107 LDD #ANSMAX
108 ASKRD PSHS D
109 LDX #ANS
110 TFR D,Y
111 JSR NTREAD
112 PULS D
113 LDX #ANS
114 JSR PUTBUF
115 JSR CRLF
116 ASKBAD PULS X,Y,PC
117 LDX #MHUH
118 JSR PUTS
119 PULS X,Y,PC
120
121 MSIGN FCC "FUJINET WEATHER"
122 FCB CR,CR,0
123 MTIME FCC "TIME.. "
124 FCB 0
125 MTEMP FCC "TEMP.. "
126 FCB 0
```

```

127 MWIND FCC "WIND.. "
128 FCB 0
129 MHUH FCC "?"
130 FCB CR,0
131 MENOOP FCC "CANNOT OPEN, ERROR "
132 FCB 0
133 MENOPA FCC "CANNOT PARSE, ERROR "
134 FCB 0
135
136 SPEC FCC "N:HTTPS://api.open-meteo.com/v1/forecast"
137 FCC "?latitude=39.10&longitude=-94.58"
138 FCC "&current=temperature_2m,wind_speed_10m"
139 FCB 0
140 QTIME FCC "/current/time"
141 FCB 0
142 QTEMP FCC "/current/temperature_2m"
143 FCB 0
144 QWIND FCC "/current/wind_speed_10m"
145 FCB 0
146
147 SPFRM FCB OPNET,0,NCSPARS,0,PMJSON
148 PAFRM FCB OPNET,0,NCPARSE,0,0
149 QYFRM FCB OPNET,0,NCQUERY,0,0
150 QYSTR RMB SZSPEC
151 QYLEN EQU *-QYFRM
152
153 INCLUDE "cocoio.asm"
154 INCLUDE "fnlow.asm"
155 INCLUDE "fnnet.asm"
156
157 STBUF RMB 4
158 ANS RMB ANSMAX
159
160 END START

```

weather.c

```

1  /*
2  * WEATHER.C --- a JSON channel, end to end
3  *
4  * Opens an HTTPS channel, puts the JSON parser on it, and asks
5  * three questions of the document without ever holding the
6  * whole thing in the CoCo's memory. That is the point of the
7  * parser: the adapter keeps the document, you keep the answer.
8  *
9  * cmoc -I$(FNDIR) -o WEATHER.BIN weather.c -lfujinet
10 * LOADM"WEATHER":EXEC
11 */
12
13 #include <cmoc.h>
14 #include <coco.h>
15 #include <fujinet-fuji.h>
16 #include <fujinet-network.h>
17
18 /*
19 * fujinet-lib keeps the bus primitives to itself, but they are
20 * in the library all the same. We need them for exactly one
21 * frame; see json_mode() below.
22 */
23 extern void bus_ready(void);
24 extern byte dwwrite(byte *s, int l);
25 extern uint8_t network_get_error(uint8_t unit);
26
27 #define OP_NET 0xE3
28 #define NC_SET_PARSER 0xFC
29 #define NC_PARSER 'P'
30 #define PARSE_JSON 1

```

```

31 static char spec[] =
32 "N:HTTPS://api.open-meteo.com/v1/forecast"
33 "?latitude=39.10&longitude=-94.58"
34 "&current=temperature_2m,wind_speed_10m";
35
36 static char q_time[] = "/current/time";
37 static char q_temp[] = "/current/temperature_2m";
38 static char q_wind[] = "/current/wind_speed_10m";
39
40 static char ans[64];
41
42 /*
43 * Put the JSON parser on the channel and parse what is there.
44 *
45 * network_json_parse() would do this, but it puts the parser
46 * mode in aux1 and the firmware reads it out of aux2, so on a
47 * current adapter it quietly selects no parser at all. Two
48 * frames by hand cost less than the afternoon that discovery
49 * costs.
50 */
51 static uint8_t json_mode(uint8_t unit)
52 {
53     struct _f
54     {
55         uint8_t op, unit, cmd, aux1, aux2;
56     } f;
57     uint8_t e;
58     f.op = OP_NET;
59
60

```

```

61     f.unit = unit;
62     f.cmd  = NC_SET_PARSER;
63     f.aux1 = 0;
64     f.aux2 = PARSE_JSON;      /* aux2, not aux1 */
65
66     bus_ready();
67     dwwrite((byte *) &f, sizeof(f));
68     e = network_get_error(unit);
69     if (e != FN_ERR_OK)
70         return e;
71
72     f.cmd  = NC_PARSE;
73     f.aux1 = 0;
74     f.aux2 = 0;
75
76     bus_ready();
77     dwwrite((byte *) &f, sizeof(f));
78     return network_get_error(unit);
79 }
80
81 static void show(const char *label, char *query)
82 {
83     printf("%s", label);
84     if (network_json_query(spec, query, ans) > 0)
85         printf("%s\n", ans);
86     else
87         printf("? \n");
88 }
89
90 int main(void)
91 {
92     uint8_t unit;
93
94     cls(1);
95     printf("FUJINET WEATHER\n\n");
96
97     if (network_open(spec, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) !=
FN_ERR_OK)
98     {
99         printf("CANNOT OPEN, ERROR %u\n", fn_device_error);
100         return 1;
101     }
102
103     unit = network_unit(spec);
104
105     if (json_mode(unit) != FN_ERR_OK)
106     {
107         printf("CANNOT PARSE, ERROR %u\n", fn_device_error);
108         network_close(spec);
109         return 1;
110     }
111
112     show("TIME.. ", q_time);
113     show("TEMP.. ", q_temp);
114     show("WIND.. ", q_wind);
115
116     network_close(spec);
117     return 0;
118 }

```

C.8 OS-9

dwport.h

```

1  #ifndef DWPORT_H
2  #define DWPORT_H

```

```

3
4 #include <cmoc.h>
5
6 /*
7  * coco.h is a Disk BASIC header and we are not using it here,
8  * so the one typedef we want from it comes along by hand.
9  */
10 typedef unsigned char byte;
11
12 /* Read l bytes into s. Returns 1 when every byte arrived. */
13 byte dwwread(byte *s, int l);
14
15 /* Write l bytes from s. Returns 0 on success. */
16 byte dwwwrite(byte *s, int l);
17
18 #endif /* DWPORT_H */

```

dwport.c — DWRead and DWWrite for a memory-mapped port

```

1 /*
2  * DWPORT.C --- DWRead and DWWrite for OS-9
3  *
4  * Under Disk BASIC the ROM hands you these two at [$D93F] and
5  * [$D941], and every other program in this book calls them.
6  * Under OS-9 that ROM is not mapped and you must supply them.
7  *
8  * These are for the memory-mapped ports: the Becker port, the
9  * CoCo 3 FPGA's DriveWire window, and the high-speed UART
10 * cartridge. Status at $FF41, data at $FF42. There is no
11 * timing to get right --- you poll a bit and move a byte.
12 *
13 * For the bit-banger port, which is what a FujiNet cartridge
14 * plugs into on a stock machine, DWRead and DWWrite are
15 * cycle-counted routines that live in the DriveWire and
16 * NitroOS-9 sources. Take them from there, put them behind
17 * these two names, and nothing above this file changes. That
18 * is the whole argument for building on the vectors.
19 *
20 * Two details that are easy to miss:
21 *
22 * The interrupt mask. These ports hold one byte. An OS-9
23 * clock tick in the wrong place loses it. Frames are short
24 * and the mask is shorter than the transfer.
25 *
26 * The timeout. The ROM's DWRead gives up after about a
27 * second, and FN_WAIT depends on that: it asks again when the
28 * adapter is busy. A poll loop with no timeout would hang
29 * there forever instead, so this one counts.
30 */
31
32 #include <cmoc.h>
33 #include "dwport.h"
34
35 byte dwwread(byte *s, int l)
36 {
37     asm
38     {
39         pshs    cc,x,y,u
40         orcc    #$50          ; the port holds exactly one byte
41         ldx     :s
42         ldy     :l
43         beq     @ok
44         @byte   ldu     #$2000      ; about a tenth of a second
45         @wait   lda     $FF41      ; DriveWire status
46         bita    #$02          ; a byte waiting?
47         bne     @got
48         leau    -1,u
49
50         bne     @wait
51         bra     @timeout
52         lda     $FF42          ; DriveWire data
53         sta     ,x+
54         leay    -1,y
55         bne     @byte
56         @ok     puls    cc,x,y,u
57         ldb     #1            ; every byte arrived
58         bra     @exit
59         @timeout puls    cc,x,y,u
60         clrb
61         @exit   clrb          ; it went quiet on us
62     }
63 }
64
65 byte dwwwrite(byte *s, int l)
66 {
67     asm
68     {
69         pshs    cc,x,y
70         orcc    #$50
71         ldx     :s
72         ldy     :l
73         beq     @done
74         @more   lda     ,x+
75         sta     $FF42
76         leay    -1,y
77         bne     @more
78         @done   puls    cc,x,y
79         clrb
80     }
81 }

```

fnbus.h

```
1  #ifndef FNBUS_H
2  #define FNBUS_H
3
4  #include <cmoc.h>
5  #include "dwport.h"
6
7  #define FN_OK 1
8
9  void fn_wait(void);
10 byte fn_error(void);
11 byte fn_send(byte *frame, int len);
12 byte fn_response(byte *buf, int len);
13
14 #endif /* FNBUS_H */
```

fnbus.c --- the four moves, above your own transport

```
1  /*
2  * FNBUS.C --- the four moves, rebuilt above your own transport
3  *
4  * Fujinet-lib's Color Computer build calls the Disk BASIC ROM
5  * directly, so it cannot be linked into an OS-9 module. This
6  * file is the part of it that matters, written against DWPORT
7  * instead. Compare it with FNLOW.ASM: same four routines,
8  * same order, same three-step handshake.
9  */
10
11 #include <cmoc.h>
12 #include "dwport.h"
13 #include "fnbus.h"
14
15 #define OP_FUJI      0xE2
16 #define FN_READY     0x00
17 #define FN_RESPONSE  0x01
18 #define FN_ERROR     0x02
19
20 #define E_GENERAL    144
21
22 /* Block until the adapter answers. It is allowed to be busy. */
23 void fn_wait(void)
24 {
25     byte frame[2];
26     byte r;
27
28     frame[0] = OP_FUJI;
29     frame[1] = FN_READY;
30
31     do
32     {
33         dwwrite(frame, sizeof(frame));
34     }
35     while (!dwread(&r, 1));
36 }
37
38 /* Ask how the last command went. 1 means it went well. */
39 byte fn_error(void)
40 {
41     byte frame[2];
42     byte e;
43
44     frame[0] = OP_FUJI;
45     frame[1] = FN_ERROR;
```

```
46
47     fn_wait();
48     dwwrite(frame, sizeof(frame));
49     return dwread(&e, 1) ? e : E_GENERAL;
50 }
51
52 /* Send a request frame and collect the verdict. */
53 byte fn_send(byte *frame, int len)
54 {
55     fn_wait();
56     dwwrite(frame, len);
57     return fn_error();
58 }
59
60 /* Fetch whatever the last command left waiting. */
61 byte fn_response(byte *buf, int len)
62 {
63     byte frame[2];
64
65     frame[0] = OP_FUJI;
66     frame[1] = FN_RESPONSE;
67
68     fn_wait();
69     dwwrite(frame, sizeof(frame));
70     return dwread(buf, len) ? 1 : E_GENERAL;
71 }
```

fnstat.c

```
1  /*
2  * FNSTAT.C --- who is on the other end of the wire, under OS-9
3  *
4  * The same question FCDEMO asks under Disk BASIC, and the same
5  * two commands answer it. Only DWRead and DWWrite changed.
6  *
7  * cmoc --os9 -o fnstat fnstat.c fnbus.c dwport.c
8  * os9 copy fnstat /dd/CMD5/fnstat
9  */
10
11 #include <cmoc.h>
12 #include "fnbus.h"
13
14 #define FUJI_ADAPTERCONFIG_EXTENDED 0xC4
15 #define CFGLEN 240
16
17 /* AdapterConfigExtended, by offset */
18 #define O_SSID 0 /* char[33] */
19 #define O_VERSION 125 /* char[15] */
20 #define O_LOCALIP 140 /* char[16], already dotted */
21
22 static byte cfg[CFGLEN];
23
24 int main(void)
25 {
26     byte frame[2];
27     byte e;
28
29     frame[0] = 0xE2;
30     frame[1] = FUJI_ADAPTERCONFIG_EXTENDED;
31
32     e = fn_send(frame, sizeof(frame));
33     if (e != FN_OK)
34     {
35         printf("fnstat: no answer, error %u\n", e);
36         return 1;
37     }
38
39     e = fn_response(cfg, CFGLEN);
40     if (e != FN_OK)
41     {
42         printf("fnstat: short reply, error %u\n", e);
43         return 1;
44     }
45
46     printf("network %s\n", (char *) &cfg[O_SSID]);
47     printf("address %s\n", (char *) &cfg[O_LOCALIP]);
48     printf("firmware %s\n", (char *) &cfg[O_VERSION]);
49
50     return 0;
51 }
```

C.9 The Makefiles

listings/common.mk

```
1 # common.mk --- shared settings for every listing in this book
2 #
3 # fujinet-lib ships as a flat directory: the headers, the .inc
4 # files and one .lib. Point FNDIR at wherever you unpacked it.
5 # The default is where an installed cmoc keeps its headers.
6
7 FNDIR ?= /usr/share/cmoc/include
8 ifeq ($(wildcard $(FNDIR)/fujinet-fuji.h),)
9 FNDIR := $(HOME)/Workspace/fujinet-lib
10 endif
```

```

11
12 CC      := cmoc
13 AS      := lwasm
14
15 CFLAGS  := -I$(FNDIR)
16 LDLIBS  := -lfujinet
17
18 ASFLAGS := --6809 --format=decb -I../fnlib
19
20 %.BIN: %.c
21 $(CC) $(CFLAGS) -o $@ $< $(LDLIBS)

```

listings/netcat/Makefile

```

1 include ../common.mk
2
3 all: NETCAT.BIN NETCAT-C.BIN
4
5 NETCAT.BIN: netcat.asm ../fnlib/fn.inc ../fnlib/fnlow.asm ../fnlib/fnnet.asm ../fnlib/cocoio.asm
6 $(AS) $(ASFLAGS) -o $@ netcat.asm
7
8 NETCAT-C.BIN: netcat.c
9 $(CC) $(CFLAGS) -o $@ netcat.c $(LDLIBS)
10
11 clean:
12 rm -f *.BIN *.dsk
13
14 .PHONY: all clean

```

listings/os9/Makefile

```

1 include ../common.mk
2
3 # An OS-9 module, not a LOADM binary.  fujinet-lib is not linked:
4 # its Color Computer build calls the Disk BASIC ROM, which is not
5 # mapped under OS-9.  FNBUS.C is the part we need, rebuilt above
6 # a transport we supply ourselves.
7
8 all: fnstat
9
10 fnstat: fnstat.c fnbus.c dwport.c fnbus.h dwport.h
11 $(CC) --os9 -o $@ fnstat.c fnbus.c dwport.c
12
13 clean:
14 rm -f fnstat *.o
15
16 .PHONY: all clean

```

APPENDIX D

TROUBLESHOOTING

Every row here is something that has actually happened to somebody, and most of them cost an evening.

Symptom	Remedy
The program hangs before printing anything.	FNWAIT loops forever by design — it is waiting for an adapter that may simply be busy. Before you know one is there, send the seven-byte probe from Chapter 3: \$A5 and seven bytes back. It gives up in a second and a half.
Nothing works after the first command, and the first command worked.	The bus is out of step. Something sent fewer bytes than a command declared — an unpadded devicespec, an app key write shorter than 64 bytes, a base64 output with no length word — or you sent a command the control device does not dispatch and it left your payload in the buffer. Only a reset recovers it.
A command reports success and does nothing.	It is probably not dispatched. The control device clears its error to <i>success</i> before looking for a handler, so an unrecognised command answers \$01. Check it against Chapter 12.12.
<code>network_json_parse()</code> succeeds and every query is empty.	The library sends the parser mode in aux1; the firmware reads aux2. Send the five-byte frame yourself — Chapter 18 does.
Reads come back padded with zeros.	You asked for more bytes than STATUS said were waiting. The adapter pads the reply to the length you named. Ask for what is there.
The read loop never ends.	You are testing for zero bytes waiting instead of for error 136 or a clear connected byte. A quiet socket has nothing waiting and is perfectly healthy.
The count register is wrong after a read.	DWRead returns the block's checksum in Y. Push it.
The program branches on the carry after a read and behaves oddly.	It is the zero flag that says the read worked. The carry is a framing error, which is a rarer and different complaint.
A two-byte value comes out byte-swapped.	Almost everything on this bus is big-endian, which is the 6809's own order — so you should not be swapping at all. The four exceptions are GET_DIRECTORY_POSITION, RANDOM_NUMBER, the network TELL, and the write-sector checksum.

Symptom	Remedy
MOUNT_IMAGE succeeds but the drive is empty.	The name was never written into the slot. SET_DEVICE_FULLPATH comes first, and MOUNT_HOST before that.
The directory opens once and never again.	There is one directory handle for the whole adapter and a previous program exited without calling CLOSE_DIRECTORY. Send one.
COPY_FILE wedges the bus.	It is broken in the firmware: the handler never reads its payload. Do not send it. Chapter 12.5 has the detail.
Setting the timezone hangs the adapter.	Also a firmware bug — the length is byte-swapped twice, so a three-byte name arrives as 768. Use the web interface.
Everything worked yesterday and the adapter is silent today.	Check the model switches. They set the baud rate, and a CoCo 1 at 38,400 and a CoCo 3 at 115,200 do not talk to each other.
Lowercase comes out as inverse video.	That is the MC6847 doing exactly what it has always done. It is not a FujiNet problem and there is no fixing it from this side; either uppercase your output or use a hi-res text screen.



INDEX

Subject	Page
AdapterConfigExtended.	14
App keys.	24
Assembler library.	16
Aux bytes.	12
Base64.	24
Baud rate.	10
Becker port.	28
Big-endian.	12
Bit-banger port.	10
Boot modes.	26
CMOC.	8, 19
CONFIG disk.	26
CP/M device.	75
Checksum.	83
Clock device.	75
Command reference, Fuji.	30
Command reference, network.	60
DEVICE_READY.	12
DWPORT.C.	28
DWRead.	10
DWWrite.	10
Desynchronisation.	12
Detail block.	24
Device slots.	24
Devices, the list of.	6
Directory, reading a.	24
DriveWire protocol.	6
End of file.	22
Error codes.	101

Subject	Page
Media types.	26
Model switches (DIP).	10
Modem (not reachable).	75
Mounting.	26
N: devicespec.	22
NETCAT.	90
NEW_DISK.	26
Named object mount.	83
Network Protocol Handbook.	98
Network device.	22
Network device commands.	60
NitrOS-9.	28
OP_DWINIT.	83
OP_JEFF.	83
OP_READEX.	83
OP_WRITE.	83
OS-9.	28, 96
Open modes.	22
Parsers.	22
Prefix (network).	22
Printer device.	75
Program listings.	103
Programs, MOUNTER.	92
Programs, NETCAT.	90
Programs, WEATHER.	94
Programs, fnstat.	96
QR code.	24
Quick reference.	99
SEND_ERROR.	12

FCDEMO.....	14	SEND_RESPONSE.....	12
FNGERR.....	16	SET_DEVICE_FULLPATH.....	26
FNGRSP.....	16	Sector protocol.....	83
FNWAIT.....	16	Status codes.....	101
FNXACT.....	16	Status, network.....	22
FN_ERR codes.....	19	Timezone.....	75
First Contact.....	14	Tools, installing.....	8
Frames, command.....	12	Toolshed.....	8
Fuji device commands.....	30	Translation modes.....	22
FujiNet cartridge.....	6	Transports.....	28
Further reading.....	98	Troubleshooting.....	125
GET_ADAPTERCONFIG_EXTENDED.....	14	Vectors, DriveWire.....	10
HDB-DOS.....	26	Virtual serial channels.....	83
Hashing.....	24	WEATHER.....	94
Host slots.....	24	fn.inc.....	16
Inline assembly (CMOC).....	19	fn_device_error.....	19
JSON.....	94	fnlow.asm.....	16
LWTOOLS.....	8	fnstat.....	96
Listings.....	103	fujinet-lib.....	8, 19
MOUNTER.....	92	lwasm.....	8
MOUNT_IMAGE.....	26		



FUJINET

THE SOURCES WIN

Every opcode, command byte, parameter and reply length in this book was transcribed from the FujiNet firmware and checked against a client that speaks to it. Firmware moves. When this book and the sources disagree, the sources are right and this book has a bug. Report it and the next edition will be better.

CHAT

The Discord. Somebody will answer a question about a desynchronised bus at two in the morning.

SOURCES

github.com/FujiNetWIFI — the firmware, the client library, the applications and the hardware.

LIBRARY

fujinet.online — the project, the documentation and a great many disk images.